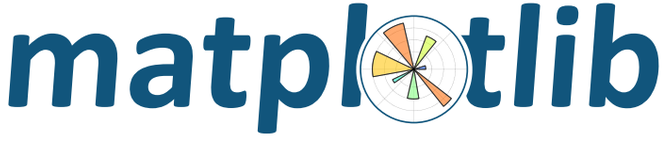
SpiralTrain
developer courses





Data Analysis with Python





General Course Remarks

- Course times
- Drinks
- Meals
- Toilettes
- Smoking
- Mobile Phones
- Manual
- Software

Everything should be made as simple as possible, but not simpler

Albert Einstein

Ja, was ich hier geschrieben habe, macht im Einzelnen überhaupt nicht den Anspruch auf Neuheit; und darum gebe ich auch keine Quellen an, weil es mir gleichgültig ist, ob das was ich gedacht habe, vor mir schon ein anderer gedacht hat.

Ludwig Wittgenstein

Prerequisites and Setup

- **Prerequisites :**
  - Experience with a programming language or an analysis tool such as SPSS, Matlab or VBA
  - Python basics : variables, lists, dictionaries, loops and functions
- **Setup, from the course folder :**

```
uv sync --frozen
uv run jupyter lab
```

- **Course folder :**
  - `exercises/` **: one notebook per exercise**
  - `solutions/` **: solutions to the exercises**
  - `datasets/` **: the data files demos/exercise might need**
  - `demos/` **: scripts shown on the slides/run**
- **The first cell of every exercise notebook :**

```python
from pathlib import Path
import pandas as pd

DATA = Path.cwd().parents[1] / "datasets" / "exercises"
```

- `Path.cwd()` **is the notebook's folder ; two levels up is the course folder**

SpiralTrain developer courses

- ▶ **NumPy**
- Pandas Intro
- Selecting and Filtering
- Cleaning and Transforming
- Advanced Indexing
- Rearranging Data
- Combining Data
- Grouping Data
- Matplotlib
- Time Series
- Integration Case
- Scikit-Learn Essentials
- Data Formats and Tools

APPENDIX

- Reading Other Sources

- What is NumPy?
- NumPy in the Python Ecosystem
- NumPy Speed
- Vectorization
- Basic Operations
- Broadcasting
- Broadcasting Rules
- Broadcasting Limits
- NumPy Data Types
- Integers and Scalars in NumPy 2
- ndarray
- Multidimensional Array
- Axis and Dimensions
- Shape and Storage Checks
- NumPy Functions
- Unique, Sort and Diff
- Reshape and Transpose
- Indexing and Slicing
- Boolean Masks
- Missing Values in NumPy
- Ones and Zeros
- Creating Arrays
- Random Number Generation
- Array Copy
- Views and Copies
- Identity and Eye Function
- Ravel and Stack
- Growing and Joining Arrays
- Optional Extensions
- Python Integers
- Python versus C Integers
- NumPy Array versus Python List
- Comparing Array Performance
- Iterating NumPy Arrays
- Memory Layout
- C and F Order

SpiralTrain developer courses

What is NumPy?

- **NumPy is the Python module for numerical computing :**
  - Python extension written in C
  - Used for fast vector and matrix operations on large data
- **NumPy contains among other things :**
  - Powerful N-dimensional array object
  - Broadcasting rules for combining arrays of different shapes
  - Linear algebra, Fourier transform and random number capabilities
  - Tools for integrating C, C++ and Fortran code
- **Core of the NumPy package is the `ndarray` object :**
  - Encapsulates n-dimensional arrays of homogeneous data types
  - Many operations are performed in compiled code for performance
- **NumPy is imported under the conventional alias `np` :**

```
import numpy as np
print(np.__version__)   # 2.5.3 in the course environment
```

SpiralTrain developer courses

NumPy in the Python Ecosystem

- **NumPy grew out of two earlier array packages :**
  - Numeric (1995) and its successor numarray served different user groups
  - Travis Oliphant merged both into NumPy 1.0 in 2006
  - NumPy 2.0 (June 2024) cleaned up the Python API and type promotion
- **Most of the scientific Python stack builds on NumPy arrays :**
  - pandas stores numeric `Series` and `DataFrame` columns in NumPy arrays
  - SciPy, Matplotlib and scikit-learn accept and return `ndarray` objects
  - `df.to_numpy()` hands pandas data to any NumPy-based library
- **NumPy arrays differ from standard Python sequences :**
  - NumPy arrays have a fixed size at creation ; Python lists grow dynamically
  - Elements in a NumPy array all have the same data type, and so the same size
  - NumPy arrays allow fast mathematical operations on large data

demo01
numpy_intro

SpiralTrain developer courses

NumPy Speed

- **Sequence size and speed are important in scientific computing**
- **Multiplying the elements of two sequences in plain Python :**
  - Gives the correct answer, but every step runs through the interpreter
  - Each element and each product is a full Python object
- **The same loop in C saves the interpreter and object overhead :**
  - At the expense of the benefits gained from coding in Python

```python
a = [1, 2, 3]
b = [10, 20, 30]
c = []
for i in range(len(a)):
    c.append(a[i] * b[i])
```

```c
for (i = 0; i < n; i++) {
    c[i] = a[i] * b[i];
}
```

- **NumPy gives the best of both worlds :**
  - Arithmetic operators work element by element on whole arrays
  - The loop over the elements runs in pre-compiled C code

```python
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
c = a * b       # [10 40 90]
```

demo02
Time Comparison

Vectorization

- **Vectorization is the absence of explicit looping and indexing in code :**
  - Looping takes place behind the scenes in optimized, pre-compiled C code
  - Operators and functions apply to every element of an array at once
- **Benefits of vectorized code :**
  - Vectorized code is more concise and easier to read
  - Fewer lines of code generally means fewer bugs
  - Code more closely resembles standard mathematical notation

```
x = np.linspace(0, 1, 5)          # [0.   0.25 0.5  0.75 1.  ]
y = 3 * x**2 + 2 * x + 1          # evaluated for all five values
print(y)                          # [1.     1.6875 2.75   4.1875 6.    ]
```

- **Each operation allocates a new full-size array :**
  - `a * b + c` first builds a temporary array for `a * b`, then adds `c`
  - In-place operators such as `a *= b` reuse the existing array
  - Keep this in mind for arrays with many millions of elements

Basic Operations

- **Arithmetic operators on arrays apply elementwise :**
  - A new array is created and filled with the result
  - Universal functions such as `np.sin` and `np.sqrt` work the same way
- **The product operator `*` also operates elementwise :**
  - The matrix product uses the `@` operator, or the `dot` function or method

```
a = np.array([20, 30, 40, 50])
b = np.arange(4)
print(b)              # [0 1 2 3]
c = a - b
print(c)              # [20 29 38 47]
b = b**2
print(b)              # [0 1 4 9]
print(10 * np.sin(a))
# [ 9.12945251 -9.88031624
#   7.4511316  -2.62374854]
```

```
A = np.array([[1, 1], [0, 1]])
B = np.array([[2, 0], [3, 4]])
print(A * B)   # elementwise product
print(A @ B)   # matrix product, A.dot(B)
```

```
[[2 0]
 [0 4]]
[[5 4]
 [3 4]]
```

Broadcasting

- **Broadcasting lets elementwise operations combine different shapes :**
  - A scalar combines with every element of an array
  - A row or a column combines with every row or column of a matrix
  - The smaller operand is reused without being copied

```
a = np.array([1, 2, 3])
print(a * 10)         # [10 20 30]

X = np.arange(6).reshape(2, 3)
print(X)
print(X + 100)
```

```
[[0 1 2]
 [3 4 5]]
[[100 101 102]
 [103 104 105]]
```

- **The scalar `100` behaves as an array of shape `(2, 3)` filled with `100`**
- **Broadcasting replaces the loops that would otherwise repeat the smaller operand**

SpiralTrain developer courses

Broadcasting Rules

- **Compare shapes starting from the last (trailing) dimension :**
  - Two dimensions are compatible if they are equal or one of them is 1
  - A missing leading dimension counts as 1
  - A size-1 dimension stretches to the other dimension's size

```
X = np.arange(6).reshape(2, 3)  # shape (2, 3)
row = np.array([10, 20, 30])    # (3,) -> (1, 3)
col = np.array([[100], [200]])  # shape (2, 1)
print(X + row)                  # shape (2, 3)
print(X + col)                  # shape (2, 3)
```

```
[[10 21 32]
 [13 24 35]]
[[100 101 102]
 [203 204 205]]
```

- **Incompatible shapes raise an error, here 3 versus 2 :**

```
np.ones((2, 3)) + np.ones(2)
# ValueError: operands could not be broadcast together with shapes (2,3) (2,)
```

SpiralTrain developer courses

Broadcasting Limits

- **Broadcasting describes elementwise operations only :**
  - `A @ B` (matrix product) needs matching inner dimensions instead
  - Reductions remove an axis : `X.sum(axis=0)` turns shape `(2, 3)` into `(3,)`
- **The result has the full broadcast shape :**
  - Shapes `(10000, 1)` and `(1, 10000)` give 100 million elements
  - As `float64` that result needs 800 MB of memory
- **`np.broadcast_shapes` predicts the result shape without computing it :**

```
print(np.broadcast_shapes((2, 1), (3,)))          # (2, 3)
print(np.broadcast_shapes((256, 256, 3), (3,)))  # (256, 256, 3)
```

- **`np.newaxis` turns a 1-D array into a row or a column explicitly :**

```
v = np.array([1, 2, 3])
print(v[:, np.newaxis].shape)   # (3, 1)
print(v[np.newaxis, :].shape)   # (1, 3)
```

NumPy Data Types

- Data type object `dtype` is an instance of the `numpy.dtype` class :
  - Can be created with `numpy.dtype`

```
i16 = np.dtype(np.int16)
print(i16)

lst = [ [3.4, 8.7, 9.9],
        [1.1, -7.8, -0.7],
        [4.1, 12.3, 4.8] ]
A = np.array(lst, dtype=i16)
print(A)
```

- Executing the code prints :

```
int16
[[ 3  8  9]
 [ 1 -7  0]
 [ 4 12  4]]
```

- Floats converted to an integer `dtype` are truncated toward zero
- Other data types in NumPy are :
  - `int8`, `int32`, `int64`, `float16`, `float32`, `float64`, `bool`, `object`

SpiralTrain developer courses

Integers and Scalars in NumPy 2

- **Default integer is `int64` on all 64-bit platforms :**
  - Before NumPy 2, Windows used `int32`, so older output may differ
- **A Python scalar's kind (`int`, `float`, `complex`) participates in promotion :**
  - Its precision is ignored ; the NumPy `dtype` can stay unchanged within a kind

```
np.int8(100) + 1       # np.int8(101): stays int8
np.float32(3) + 3.0    # np.float32(6.0): stays float32
np.int8(100) + 1.5     # np.float64(101.5): float kind promotes the integer
```

- **Fixed-width integers can overflow :**

```
np.array([127], dtype=np.int8) + 1  # array([-128], dtype=int8), no warning
np.int8(127) + 1                     # np.int8(-128) and a RuntimeWarning
np.int8(100) + 300
# OverflowError: Python integer 300 out of bounds for int8
```

- **Choose a `dtype` wide enough for the values, such as `int64` or `float64`**

SpiralTrain developer courses

ndarray

- **NumPy's `ndarray` object is a homogeneous multidimensional array :**
  - Dimensions are called axes ; the number of axes is `ndim` (sometimes "rank")
- **The following array has 2 axes :**
  - First dimension has a length of 2, second dimension has a length of 3

```
[[1, 0, 0], [0, 1, 2]]
```

- **Zero-dimensional array :**

```
x = np.array(42)
print(type(x))      # <class 'numpy.ndarray'>
print(np.ndim(x))   # 0
```

- **One-dimensional arrays :**

```
F = np.array([1, 1, 2, 3, 5, 8, 13, 21])
V = np.array([3.4, 6.9, 99.8, 12.8])
print(F.dtype)      # int64
print(V.dtype)      # float64
print(np.ndim(F))   # 1
print(np.ndim(V))   # 1
```

demo07
Zero_and_one_dimensional_array

Multidimensional Array

- `ndarray.shape` is a tuple of integers with the size of each dimension :
  - For a matrix with n rows and m columns, shape is `(n, m)`
  - First index or dimension counts the rows and is axis 0
  - Second index or dimension counts the columns and is axis 1

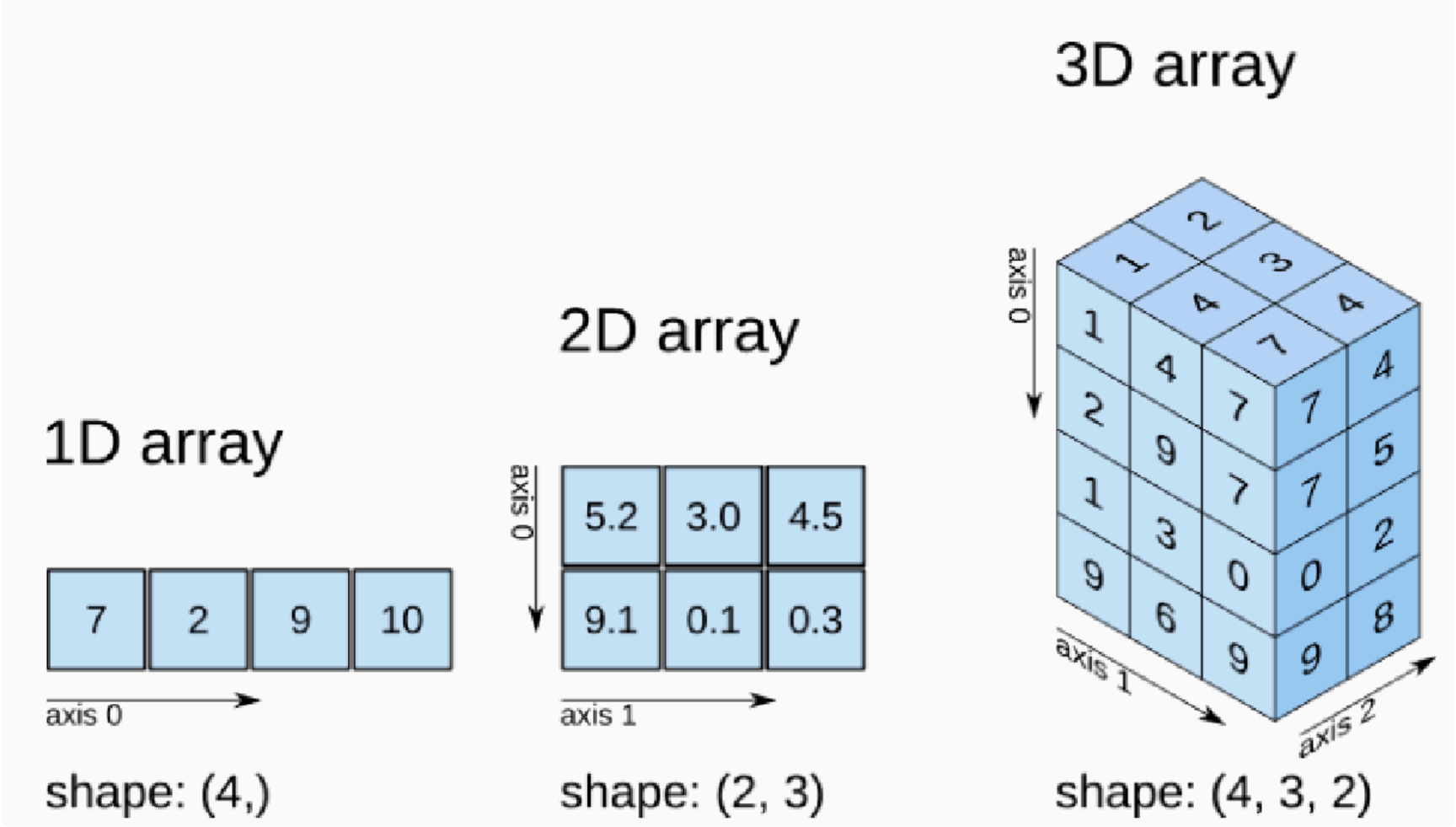
```
A = np.array([[3.4, 8.7, 9.9],
              [1.1, -7.8, -0.7],
              [4.1, 12.3, 4.8]])
print(A)
print(A.ndim)   # 2
print(A.shape)  # (3, 3)
```

```
B = np.array([[[111, 112], [121, 122]],
              [[211, 212], [221, 222]],
              [[311, 312], [321, 322]]])
print(B)
print(B.ndim)   # 3
print(B.shape)  # (3, 2, 2)
```

- A shape lists the sizes from the outermost to the innermost dimension

demo08
Multi dimensional array

Axis and Dimensions



Shape and Storage Checks

```
samples = np.arange(6, dtype=np.int64).reshape(2, 3)
print(samples.shape)     # (2, 3): two rows, three columns
print(samples.size)      # 6: number of elements
print(samples.itemsize)  # 8: bytes per int64 element
print(samples.nbytes)    # 48: bytes used by the element data
```

- `nbytes` excludes array metadata and any separately referenced Python objects
- Predict the shape and result size before broadcasting large operands
- Check shapes and storage when validating a numerical calculation

NumPy Functions

- **Aggregations such as `min`, `max`, `sum` and `std` reduce an array to a value :**
  - `axis=0` reduces down the columns, `axis=1` along the rows

```
a = np.array([1, 2, 3])
print(a.min())          # 1
print(a.max())          # 3
print(a.sum())          # 6

a = np.array([(1, 2, 3), (3, 4, 5)])
print(a.sum(axis=0))    # [4 6 8]
print(a.sum(axis=1))    # [ 6 12]
```

```
print(np.sqrt(a))       # elementwise
print(np.std(a))        # 1.2909944487358056

# divide the interval 1 to 3
# into 10 equally spaced points
a = np.linspace(1, 3, 10)
print(a)
```

- **`np.argmin` and `np.argmax` return the position of the extreme value :**
  - With `m = np.array([[3, 9, 2], [8, 1, 7]])`, `m.argmax(axis=0)` gives `[1 0 1]` : the row of each column's maximum
  - `m.argmax(axis=1)` gives `[1 0]` : the column of each row's maximum
- **`np.any` and `np.all` test whether some or all elements are `True`**

demo09
Numpy Functions

Unique, Sort and Diff

SpiralTrain
developer courses

```
a = np.array([3, 1, 3, 2, 1, 3])
print(np.unique(a))                          # [1 2 3]
values, counts = np.unique(a, return_counts=True)
print(values, counts)                        # [1 2 3] [2 1 3]
print(np.sort(a))                            # [1 1 2 3 3 3]
print(np.diff(np.array([1, 4, 9, 16])))   # [3 5 7]
```

- `np.unique` returns the distinct values, sorted ; `return_counts=True` also counts them
- `np.sort` returns a sorted copy ; `a.sort()` sorts `a` itself
- `np.diff` gives the differences between neighbouring elements, one fewer than the input

demo23
Unique sort diff

SpiralTrain developer courses

Reshape and Transpose

- **`numpy.reshape` gives an array a new shape without changing its data :**
  - Only possible if the total number of elements remains the same
  - `-1` lets NumPy compute one dimension : `a.reshape(-1, 2)`
- **`numpy.transpose` reverses the dimensions of the array :**
  - Turns rows into columns and columns into rows for a 2-D array
  - Returns a view with permuted axes whenever possible
  - Also available as the `T` attribute

```
a = np.array([(8, 9, 10), (11, 12, 13)])
print(a.shape)      # (2, 3)
b = a.transpose()   # or a.T
print(b)
print(b.shape)      # (3, 2)
```

```
a = a.reshape(3, 2)
print(a)
print(a.shape)      # (3, 2)
a = a.reshape(2, 3)
print(a.shape)      # (2, 3)
```

Indexing and Slicing

- **One-dimensional arrays can be indexed, sliced and iterated over :**
  - Much like lists and other Python sequences
- **Multidimensional arrays take one index or slice per axis :**
  - Separated by commas inside a single pair of square brackets

```
F = np.array([1, 1, 2, 3, 5, 8, 13, 21])
print(F[0])       # 1: first element
print(F[-1])      # 21: last element
print(F[2:5])     # [2 3 5]
print(F[::2])     # [ 1  2  5 13]
print(F[::-1])    # reversed
```

```
B = np.array([[[111, 112], [121, 122]],
              [[211, 212], [221, 222]],
              [[311, 312], [321, 322]]])
print(B[0, 1, 0])   # 121
print(B[0][1][0])   # 121, slower
print(B[:, 0, 0])   # [111 211 311]
```

- `B[0, 1, 0]` indexes once ; `B[0][1][0]` creates two intermediate arrays
- A step of `-1` walks backwards : `F[::-1]` reverses the array
- Assignment uses the same indexing : `F[0] = 100` changes the first element

Boolean Masks

```
a = np.array([4, 15, 8, 23, 16, 42])
mask = a > 10
print(mask)                     # [False  True False  True  True  True]
print(a[mask])                  # [15 23 16 42]
print(np.nonzero(mask)[0])      # [1 3 4 5]
print(np.count_nonzero(mask))   # 4
print(a[(a > 10) & (a < 30)])   # [15 23 16]
print(np.where(a > 10, a, 0))   # [ 0 15  0 23 16 42]
```

- A comparison gives a Boolean array ; indexing with it keeps the `True` elements
- `np.nonzero(mask)` returns the positions of the `True` elements, one array per axis
- `&`, `|` and `~` combine masks ; put each comparison in parentheses
- `np.where(condition, x, y)` picks from `x` where the condition holds, else from `y`
- A Boolean index returns a copy, not a view

demo21
Boolean masks

Missing Values in NumPy

```
temps = np.array([17.4, 15.9, np.nan, 14.1])
print(temps.mean())                  # nan
print(np.nanmean(temps).round(2))    # 15.8
print(np.isnan(temps))               # [False False  True False]
print(np.isnan(temps).sum())         # 1
print(np.nan == np.nan)              # False
```

- `np.nan` (Not a Number) marks a missing float value
- Ordinary reductions return `nan` when any input is missing
- `np.nanmean`, `np.nansum`, `np.nanmax` and others skip the missing values
- `np.isnan` finds them; `nan == nan` is `False`, so never compare with `==`
- pandas builds its missing-value handling on this (Module 4)

demo24
Missing values

Ones and Zeros

- `np.ones(shape)` takes a shape tuple and fills the array with ones :
  - The default data type is `float64` ; `dtype` selects another one
- `np.zeros(shape)` does the same with zeros

```
E = np.ones((2, 3))
print(E)    # [[1. 1. 1.]
            #  [1. 1. 1.]]
F = np.ones((3, 4), dtype=int)
print(F)    # [[1 1 1 1]
            #  [1 1 1 1]
            #  [1 1 1 1]]
Z = np.zeros((2, 4))
print(Z)    # [[0. 0. 0. 0.]
            #  [0. 0. 0. 0.]]
```

- `np.empty(shape)` allocates an array without initializing it :
  - Fast, but the content is whatever was in memory
  - Assign every element before reading it

Creating Arrays

```
print(np.arange(2, 11).reshape(3, 3))   # [[ 2  3  4]
                                        #  [ 5  6  7]
                                        #  [ 8  9 10]]
print(np.arange(0, 1, 0.25))            # [0.   0.25 0.5  0.75]
print(np.full((2, 3), 7))               # [[7 7 7]
                                        #  [7 7 7]]
print(np.asarray((1, 2, 3)))            # [1 2 3]
f = np.array([1, 2, 3, 4], dtype=np.float64)
print(f / 2)                            # [0.5 1.  1.5 2. ]
```

- `np.arange(start, stop, step)` excludes `stop`, like `range`; the step can be a float
- `np.full(shape, value)` fills a new array with one value
- `np.array` and `np.asarray` convert lists and tuples; `asarray` does not copy an existing array
- `dtype=` fixes the element type when creating the array

demo20
Creating arrays

Random Number Generation

- `np.random.default_rng(seed)` creates a `Generator` object :
  - A seed makes the sequence reproducible
  - Each generator has its own state ; the legacy `np.random.seed` is global

```
rng = np.random.default_rng(42)
print(rng.random((2, 3)))             # floats in [0.0, 1.0)
print(rng.integers(0, 10, size=5))    # integers 0 to 9
print(rng.normal(loc=0, scale=1, size=3))
print(rng.choice([10, 20, 30, 40], size=2, replace=False))
```

- The first call prints :

```
[[0.77395605 0.43887844 0.85859792]
 [0.69736803 0.09417735 0.97562235]]
```

Array Copy

- Simple assignments make no copy of array objects or data :

```
a = np.arange(12)
b = a            # no new object is created
print(b is a)  # True: a and b are two names for the same ndarray object
```

- For numeric arrays, `.copy()` creates independent array data :

```
x = np.array([[42, 22, 12], [44, 53, 66]])
y = x.copy()
x[0, 0] = 1001
print(x[0, 0])                    # 1001
print(y[0, 0])                    # 42
print(np.shares_memory(x, y))   # False
```

- Copy when later edits must leave the original unchanged
- Object arrays copy references ; nested mutable objects stay shared

Views and Copies

```
a = np.arange(6)
s = a[1:4]                          # basic slice: a view
s[0] = 100
print(a)                            # [  0 100   2   3   4   5]
print(np.shares_memory(a, s))       # True
print(s.base is a)                  # True: s looks into the data of a

f = a[[1, 2, 3]]                    # integer-array index: a copy
f[0] = -1
print(a)                            # [  0 100   2   3   4   5]
print(np.shares_memory(a, f))       # False
```

- Basic slices are views ; integer-array and boolean indexing return copies
- Use `.copy()` when an independent array is needed
- pandas differs : under Copy-on-Write (default in pandas 3) a selection acts as a copy

demo16
Array Copy

Identity and Eye Function

- **`np.identity(n)` returns the identity array :**
  - Square array with ones on the main diagonal
- **`np.eye(n, m, k)` returns a 2-D array :**
  - Ones on a diagonal and zeros elsewhere
  - `k` shifts the diagonal up (positive) or down (negative)
  - `m` sets the number of columns, so the array need not be square

```
print(np.identity(4))
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

```
print(np.eye(3, k=1))
```

```
[[0. 1. 0.]
 [0. 0. 1.]
 [0. 0. 0.]]
```

demo17
Identity Function

demo18
Eye Function

SpiralTrain developer courses

Ravel and Stack

- `ravel` flattens an array to one dimension :
  - Returns a view whenever possible, otherwise a copy
  - `flatten` does the same but always returns a copy
- `np.vstack` stacks arrays vertically, row after row
- `np.hstack` stacks arrays horizontally, column after column

```
x = np.array([(1, 2, 3), (3, 4, 5)])
y = np.array([(1, 2, 3), (3, 4, 5)])
print(x.ravel())   # [1 2 3 3 4 5]
print(np.vstack((x, y)))
print(np.hstack((x, y)))
```

```
[[1 2 3]
 [3 4 5]
 [1 2 3]
 [3 4 5]]
[[1 2 3 1 2 3]
 [3 4 5 3 4 5]]
```

demo19
Ravel and Stack

Growing and Joining Arrays

SpiralTrain
developer courses

```
a = np.array([10, 20, 30])
b = np.append(a, [40, 50])
print(a)                          # [10 20 30]
print(b)                          # [10 20 30 40 50]
print(np.concatenate([a, b]))     # [10 20 30 10 20 30 40 50]

values = []
for x in range(3):
    values.append(x * 10)
print(np.array(values))           # [ 0 10 20]
```

- An array has a fixed size : `np.append` returns a new, longer array and leaves `a` unchanged
- `np.concatenate` joins a list of arrays along an existing axis
- Each append copies all elements ; to grow a collection, collect values in a Python list first, then convert once

demo22
Growing and joining

SpiralTrain developer courses

Summary : NumPy

- **NumPy is the Python module for scientific computing :**
  - Python extension written in C used for fast vector and matrix operations
  - SciPy, pandas and Matplotlib all build on it
- **Core of the NumPy package is the `ndarray` object :**
  - Encapsulates n-dimensional arrays of homogeneous data types
  - Dimensions are called axes ; the number of axes is `ndim`
- **NumPy supports vectorization and broadcasting :**
  - Vectorization replaces explicit Python loops with compiled whole-array operations
  - Broadcasting combines shapes whose trailing dimensions are equal or 1
- **Basic slices are views ; use `.copy()` for an independent array**
- **Boolean masks select elements ; `np.nan` marks missing values, `np.nan*` functions skip them**
- **The optional extensions that follow cover internals and memory layout**

exercise1
Numpy

Optional Extensions

- **These extensions are outside the introductory analysis path :**
  - CPython objects and array storage explain representation choices
  - Measured comparisons of Python loops and NumPy on the same data
  - Element iteration and C/F memory order for numerical workloads
- **Demo03, Demo04 and Demo11 to Demo13 belong to these extensions**

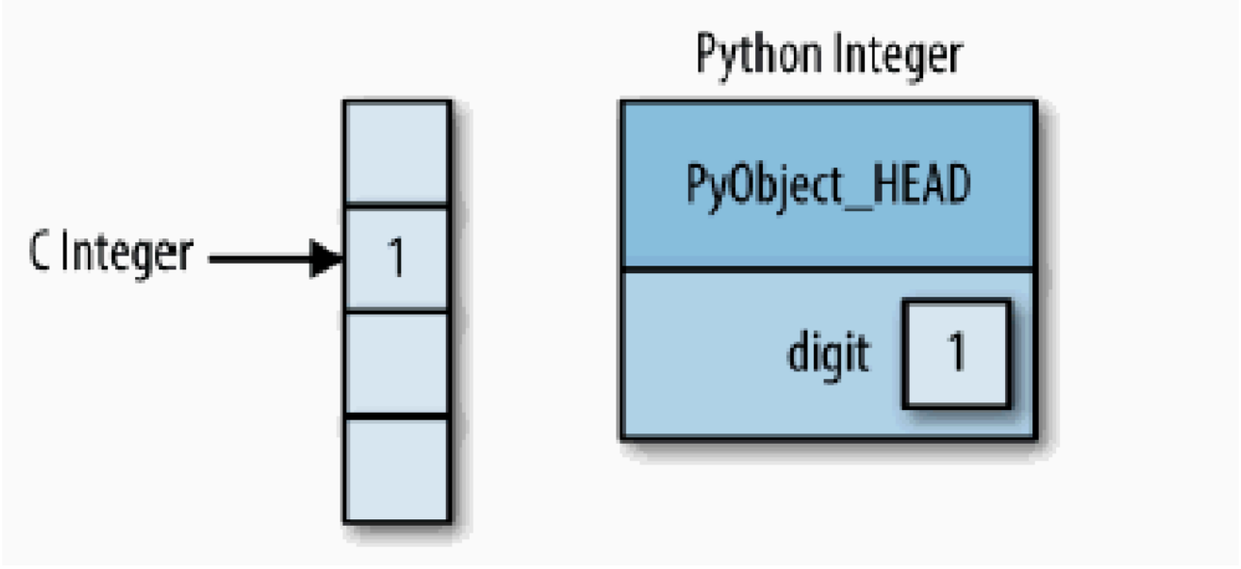
SpiralTrain developer courses

Python Integers

- **A Python `int` is a complete object (a "boxed" value) :**
  - Object header with reference count and type, followed by the value
  - Arbitrary precision : larger values use more memory
  - A variable or list element holds a reference to that object
- **A C integer, or one element of a NumPy `int64` array, is raw bytes :**
  - Fixed width (8 bytes for `int64`), interpreted according to the `dtype`

```
import sys
import numpy as np
print(sys.getsizeof(1))           # 28 (bytes, 64-bit CPython 3.14)
print(sys.getsizeof(10**100))     # 72
print(np.arange(1000).itemsize)   # 8
```

Python versus C Integers

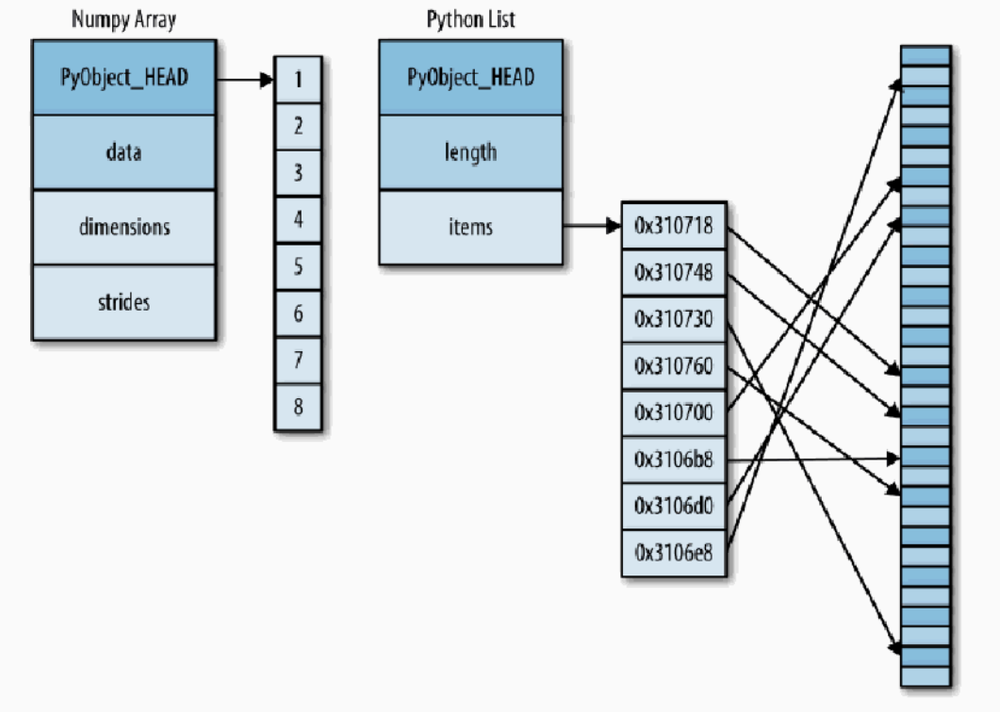


- **Conceptual picture only :**
  - `PyObject_HEAD` stands for the reference count, type and other header fields
  - The exact CPython layout is an internal detail that changes between versions

SpiralTrain developer courses

NumPy Array versus Python List

- **NumPy array :**
  - Metadata (`dtype` , `shape` , `strides`) plus a pointer to a buffer of raw values
  - A new array usually has one contiguous buffer
  - A slice can be a strided view into the buffer of another array
  - Arrays with `dtype=object` store references to Python objects, like a list
- **Python list :**
  - Pointer to a block of pointers, each pointing to a full Python object



demo03
numpyversusarraysize

SpiralTrain developer courses

Comparing Array Performance

- **Demo04 adds the same integers using Python ranges and NumPy arrays :**
  - Its inputs are created before timing ; setup and conversion are excluded
  - Demo02 does the same for a loop over two lists
- **Before reusing a timing :**
  - Verify that both versions give equal results
  - Account for the setup each version needs
  - Repeat measurements on a relevant workload
- **The speed-up a demo prints is an observation on that machine, not a fixed factor**

demo04
numpyversusarrayperformance.

Iterating NumPy Arrays

- Iterating over a NumPy array can be done using `for` :

```
ca = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
for i in range(len(ca)):
    for j in range(len(ca[i])):
        print(ca[i][j], end=' ')
```

- For calculations, prefer whole-array operations such as `ca.sum()` or `ca * 2`
- To visit every element, iterator object `numpy.nditer` is an option :
  - Order of iteration is chosen to match the memory layout of an array

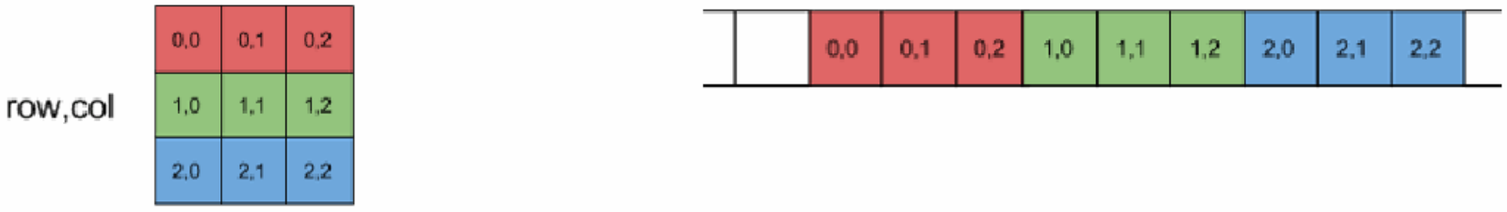
```
a = np.arange(12).reshape(3, 4)
for x in np.nditer(a):
    print(x, end=' ')       # 0 1 2 3 4 5 6 7 8 9 10 11
```

demo11
Iterating over Numpy Arrays

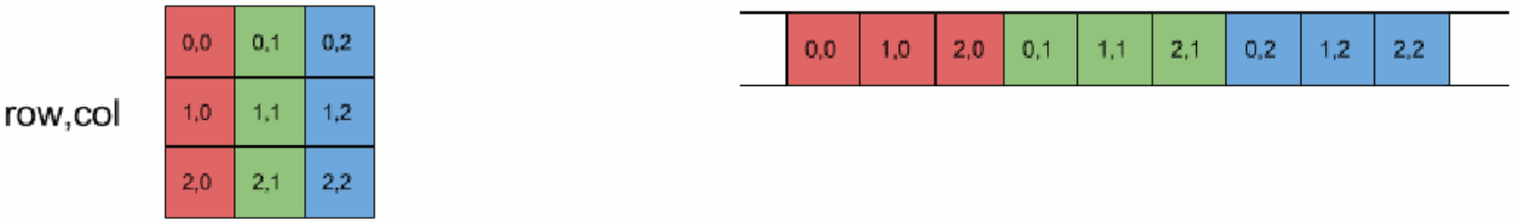
SpiralTrain developer courses

Memory Layout

- **Computer memory is a linear one-dimensional structure :**
  - Multi-dimensional data can be mapped onto it in several ways
- **Row-major layout stores the rows one after another :**
  - Used in C and the default in NumPy ; column indices change the fastest



- **Column-major layout stores the columns one after another :**
  - Used in Fortran and available in NumPy ; row indices change the fastest



SpiralTrain developer courses

C and F Order

- **Memory layout of NumPy arrays can be specified at creation time :**
  - `'C'` indicates row major, `'F'` indicates column major, default is `'C'`
- **`nditer` provides the `order` parameter to control iteration order :**
  - Default `order='K'` keeps the existing memory order
  - `order='C'` and `order='F'` force C or Fortran order

```
a = np.arange(6).reshape(2, 3)
c = np.array(a, order='C')
f = np.array(a, order='F')
for x in np.nditer(c):
    print(x, end=' ')   # 0 1 2 3 4 5
print()
for x in np.nditer(f):
    print(x, end=' ')   # 0 3 1 4 2 5
print()
for x in np.nditer(f, order='C'):
    print(x, end=' ')   # 0 1 2 3 4 5
```

- **The traversal order affects performance :**
  - **Faster when subsequent accesses are near each other in memory**
  - **Demo13 uses 2000 × 2000 by default ; `--large` runs the original 10000 × 10000**

demo12
Iteration Order

demo13
CandF Order Performance

- NumPy
- **Pandas Intro**
- Selecting and Filtering
- Cleaning and Transforming
- Advanced Indexing
- Rearranging Data
- Combining Data
- Grouping Data
- Matplotlib
- Time Series
- Integration Case
- Scikit-Learn Essentials
- Data Formats and Tools

APPENDIX

- Reading Other Sources

- What is pandas?
- pandas History
- Key Features of pandas
- Series
- DataFrames from a Dictionary
- DataFrames from Lists
- Broadcasting a New Column
- Broadcasting with a Dictionary
- Change Index and Columns
- Reading a CSV File
- head() and tail()
- info() and Missing Values
- describe()
- Selecting a Column
- A Saved Row Index
- Sunspot CSV
- Dataset Problems
- Declare a Headerless CSV
- Name the Columns
- Declare the Missing-Value Sentinel
- More read_csv Options
- Assemble Dates
- Check the Resulting Schema
- Use Dates as the Index
- Trim Redundant Columns
- Write Files

What is pandas?

- **pandas provides labeled tables and analysis operations for Python :**
  - Two core data structures : the one-dimensional `Series` and the two-dimensional `DataFrame`
  - The name derives from ”panel data”, a term for multidimensional data sets
  - Built on NumPy ; numeric columns are stored in NumPy arrays
- **pandas is well suited for :**
  - Tabular data with columns of different types, as in an SQL table or Excel sheet
  - Time series data, with or without a fixed frequency
  - Any other form of observational or statistical data set
- **Data can be loaded from CSV, Excel, JSON, SQL databases and more :**
  - Rows and columns get meaningful labels instead of bare positions

```
import pandas as pd
print(pd.__version__)   # 3.0.6 in the course environment
```

SpiralTrain developer courses

pandas History

- **Wes McKinney started pandas in 2008 at the investment firm AQR :**
  - Financial time series needed labeled data, alignment and missing-value handling
  - Released as open source in 2009 ; now a NumFOCUS sponsored project
- **Major releases changed defaults that older code relies on :**
  - pandas 1.0 (2020) stabilized the API
  - pandas 2.0 (2023) added Arrow-backed data types
  - pandas 3.0 (2026) made Copy-on-Write and the `str` text dtype the default
- **Older tutorials and answers online often show pandas 1 or 2 behavior :**
  - Text columns that used to print as `object` now print as `str`
  - Some assignment patterns that used to work no longer change the frame (Module 3)

SpiralTrain developer courses

Key Features of pandas

- **Fast, labeled containers for data analysis :**
  - Easy handling of missing data
  - Columns can be inserted and deleted
- **Automatic alignment on labels in arithmetic**
- **Selecting, filtering and transforming columns and rows**
- **Merging and joining data sets, as in SQL**
- **Group by : split-apply-combine operations**
- **Reshaping and pivoting**
- **Time series : date ranges, resampling, moving windows**
- **Reading and writing many file formats**

SpiralTrain developer courses

Series

- A `Series` is one labeled column of values :
  - The `index` holds the labels, shown on the left
  - All values share one data type, the `dtype`
  - `name` labels the column itself

```
visitors = pd.Series([139, 237, 326], index=['Sun', 'Mon', 'Tue'], name='visitors')
print(visitors)
print(visitors.index.tolist())
print(visitors.to_list())
print(visitors.dtype)
```

```
Sun    139
Mon    237
Tue    326
Name: visitors, dtype: int64
['Sun', 'Mon', 'Tue']
[139, 237, 326]
int64
```

- Without `index=`, pandas numbers the rows 0, 1, 2 and so on

demo01
Series

SpiralTrain developer courses

DataFrames from a Dictionary

- A `DataFrame` is a table of columns that share one row index :
  - Each dictionary key becomes a column label, each list a column
  - All lists must have the same length
  - Each column has its own `dtype`

```
data = {'weekday': ['Sun', 'Sun', 'Mon', 'Mon'],
        'city': ['Paris', 'Berlin', 'Paris', 'Berlin'],
        'visitors': [139, 237, 326, 456],
        'signups': [7, 12, 3, 5]}
users = pd.DataFrame(data)
print(users)
```

```
  weekday    city  visitors  signups
0     Sun   Paris       139        7
1     Sun  Berlin       237       12
2     Mon   Paris       326        3
3     Mon  Berlin       456        5
```

demo02
From a dictionary

SpiralTrain developer courses

DataFrames from Lists

- `zip` pairs a list of labels with a list of columns :
  - `dict` turns the pairs into the dictionary that `DataFrame` expects
  - The columns appear in the order of the labels

```
cities = ['Paris', 'Berlin', 'Paris', 'Berlin']
signups = [7, 12, 3, 5]
visitors = [139, 237, 326, 456]
weekdays = ['Sun', 'Sun', 'Mon', 'Mon']
labels = ['city', 'signups', 'visitors', 'weekday']
columns = [cities, signups, visitors, weekdays]
users = pd.DataFrame(dict(zip(labels, columns)))
print(users)
```

```
     city  signups  visitors weekday
0   Paris        7       139     Sun
1  Berlin       12       237     Sun
2   Paris        3       326     Mon
3  Berlin        5       456     Mon
```

demo03
From lists

Broadcasting a New Column

- **Assigning a scalar to a new column label creates the column :**
  - The scalar is broadcast to every row
  - The new column is added at the right

```
users['fees'] = 0
print(users)
```

```
     city  signups  visitors weekday  fees
0   Paris        7       139     Sun     0
1  Berlin       12       237     Sun     0
2   Paris        3       326     Mon     0
3  Berlin        5       456     Mon     0
```

- **Assigning a list with one value per row also works :** `users['fees'] = [1, 2, 3, 4]`

demo04
Broadcasting new column

Broadcasting with a Dictionary

- **A scalar value in the dictionary is broadcast to the length of the lists :**
  - `'sex': 'M'` fills all seven rows

```
heights = [59.0, 65.2, 62.9, 65.4, 63.7, 65.7, 64.1]
results = pd.DataFrame({'height': heights, 'sex': 'M'})
print(results)
```

```
   height sex
0    59.0   M
1    65.2   M
2    62.9   M
3    65.4   M
4    63.7   M
5    65.7   M
6    64.1   M
```

demo05
Broadcasting dictionary

SpiralTrain developer courses

Change Index and Columns

- Assigning a list to `columns` or `index` replaces all labels at once :
  - The list must have one label per column or row
- `rename` changes selected labels and returns a new frame

```
results.columns = ['height (in)', 'sex']
results.index = list('ABCDEFG')
print(results.head(3))

results = results.rename(columns={'height (in)': 'height_in'})
print(results.columns.tolist())
```

```
   height (in) sex
A         59.0   M
B         65.2   M
C         62.9   M
['height_in', 'sex']
```

- Assign the result of `rename` ; the original frame is not changed

demo06
Change index and columns

SpiralTrain developer courses

Reading a CSV File

- `pd.read_csv` builds a `DataFrame` from a comma-separated file :
  - The header line gives the column labels
  - `AAPL.csv` holds weekly Apple stock prices since 2022, one row per week

```
aapl = pd.read_csv('AAPL.csv')
print(aapl.shape)
print(aapl.columns.tolist())
aapl = aapl.set_index('Date')
print(aapl.shape)
```

```
(249, 6)
['Date', 'Open', 'High', 'Low', 'Close', 'Volume']
(249, 5)
```

- `shape` gives the number of rows and columns
- `set_index('Date')` makes the dates the row labels and returns a new frame :
  - Assign the result to keep the change

demo07
Read CSV

SpiralTrain developer courses

head() and tail()

- `head(n)` returns the first `n` rows, `tail(n)` the last ; the default is 5

```
print(aapl.head(3))
print(aapl.tail(3))
```

```
                Open    High     Low   Close     Volume
Date
2022-01-03  177.83  182.94  171.03  172.17  482162350
2022-01-10  169.08  177.18  168.17  173.07  422655620
2022-01-17  171.51  172.54  162.30  162.41  400253140
                Open    High     Low   Close     Volume
Date
2026-09-21  335.28  345.34  333.05  341.07  162105450
2026-09-28  340.37  342.99  325.81  333.69  190872350
2026-10-05  332.82  341.57  330.62  336.64  172211370
```

- Comparing `head` and `tail` shows the range the data covers
- Prices are in US dollars ; `Volume` counts the shares traded in the week

demo08
head and tail

info() and Missing Values

```
aapl.info()
print(aapl.isna().sum())
```

- `info()` shows each column's non-missing count and `dtype`
- `isna()` marks missing values ; `sum()` counts them per column
- This file has no missing values

```
<class 'pandas.DataFrame'>
Index: 249 entries, 2022-01-03 to 2026-10-05
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Open    249 non-null    float64
 1   High    249 non-null    float64
 2   Low     249 non-null    float64
 3   Close   249 non-null    float64
 4   Volume  249 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 14.1 KB
Open      0
High      0
Low       0
Close     0
Volume    0
dtype: int64
```

demo09
info

SpiralTrain developer courses

describe()

- `describe()` summarizes each numeric column :
  - Count, mean, standard deviation, minimum, quartiles and maximum

```
print(aapl[['Close', 'Volume']]
      .describe().round(2))
```

- A list of labels selects those columns
- `count` excludes missing values
- `std` is the sample standard deviation
- The 50% row is the median

	Close	Volume
count	249.00	2.490000e+02
mean	208.03	2.983184e+08
std	51.49	1.072001e+08
min	129.62	1.056462e+08
25%	169.58	2.238178e+08
50%	195.27	2.730243e+08
75%	242.84	3.501599e+08
max	341.07	6.886707e+08

demo10
describe

Selecting a Column

```
close = aapl['Close']
print(type(close).__name__)
print(close.head(3))

prices = close.to_numpy()
print(type(prices).__name__, prices[:3])
```

```
Series
Date
2022-01-03    172.17
2022-01-10    173.07
2022-01-17    162.41
Name: Close, dtype: float64
ndarray [172.17 173.07 162.41]
```

- One column label in square brackets gives a `Series` :
  - It keeps the row index ; the column label becomes its `name`
- `.to_numpy()` returns the values as a NumPy array, without the labels
- Module 3 covers selecting rows, columns and single values in detail

demo11
Select a column

SpiralTrain developer courses

A Saved Row Index

- `users.csv` was written with its row index as the first, unnamed column :

```
,weekday,city,visitors,signups
0,Sun,Paris,139,7
1,Sun,Berlin,237,12
```

- `index_col=0` turns that first column back into the row index :

```
users = pd.read_csv('users.csv', index_col=0)
print(users)
```

```
  weekday    city  visitors  signups
0     Sun   Paris       139        7
1     Sun  Berlin       237       12
2     Mon   Paris       326        3
3     Mon  Berlin       456        5
```

- Without `index_col`, the old index appears as an extra column named `Unnamed: 0`

demo12
index_col

Sunspot CSV

- **Real files need inspection before parsing :**
  - `ISSN_D_tot.csv` holds daily sunspot counts since 1818

```
filepath = 'ISSN_D_tot.csv'
sunspots = pd.read_csv(filepath)
print(sunspots.iloc[10:13])
```

```
    1818  01  01.1  1818.004   -1  1
10  1818   1    12  1818.034   -1  1
11  1818   1    13  1818.037   22  1
12  1818   1    14  1818.040   -1  1
```

- `iloc[10:13]` shows rows 10 to 12 by position (Module 3)
- The file has no header : the default read uses its first data row as column names
- pandas makes duplicate names unique, hence `01.1`

demo13
Sunspot CSV file

Dataset Problems

- **The CSV file has no column headers :**
  - Columns 0-2 : Gregorian date (year, month, day)
  - Column 3 : date as fraction of a year
  - Column 4 : daily total sunspot number
  - Column 5 : definitive/provisional indicator (1 or 0)
- **Missing values in column 4 are indicated by `-1`**
- **Date representation is inconvenient :**
  - Three separate numeric columns instead of one date
- **The following slides fix each problem with a `read_csv` keyword or one step after reading**

Declare a Headerless CSV

- `header=None` reads the first line as data :
  - Columns are numbered from 0
  - Row 10 now holds 11 January, which the default read lost

```
sunspots = pd.read_csv(filepath, header=None)
print(sunspots.iloc[10:13])
```

```
       0  1   2         3   4  5
10  1818  1  11  1818.031  -1  1
11  1818  1  12  1818.034  -1  1
12  1818  1  13  1818.037  22  1
```

demo14
header keyword

SpiralTrain developer courses

Name the Columns

- `names` supplies the column labels for a headerless file :
  - One name per column, in file order

```
col_names = ['year', 'month', 'day', 'dec_date', 'sunspots', 'definite']
sunspots = pd.read_csv(filepath, header=None, names=col_names)
print(sunspots.iloc[10:13])
```

```
    year  month  day  dec_date  sunspots  definite
10  1818      1   11  1818.031        -1         1
11  1818      1   12  1818.034        -1         1
12  1818      1   13  1818.037        22         1
```

- For a file that does have a header line, `header=0, names=[...]` replaces it

demo15
names keyword

SpiralTrain developer courses

Declare the Missing-Value Sentinel

- `na_values` lists the values to read as missing :
  - A dictionary restricts each sentinel to one column

```
sunspots = pd.read_csv(filepath, header=None, names=col_names,
                       na_values={'sunspots': [' -1']})
print(sunspots.iloc[10:13])
print(sunspots['sunspots'].isna().sum(), len(sunspots))
```

```
    year  month  day  dec_date  sunspots  definite
10  1818      1   11  1818.031       NaN         1
11  1818      1   12  1818.034       NaN         1
12  1818      1   13  1818.037      22.0         1
3247 71861
```

- This CSV stores a leading space in the sentinel token ; match the actual input
- `sunspots` becomes `float64` because the missing marker `NaN` is a float
- 3247 of 71861 days, about 4.5%, have no sunspot count

demo16
na_values keyword

More read_csv Options

```
Weather station export, De Bilt
Generated 2026-10-01; values in ...
date;max_temp;rain
2026-09-28;17.4;0.0
2026-09-29;15.9;3.2
# sensor maintenance, no reading ...
2026-10-01;14.1;7.8
```

```
readings = pd.read_csv('station-readings.txt', sep=';',
                       skiprows=2, comment='#')
print(readings)
```

```
         date  max_temp  rain
0  2026-09-28      17.4   0.0
1  2026-09-29      15.9   3.2
2  2026-10-01      14.1   7.8
```

- `sep=';'` sets the field separator ; `delimiter=` is the same option, `sep='\t'` reads tabs
- `skiprows=2` skips the two introduction lines before the header
- `header=2` gives the same result here : the header is line 2, counting from 0
- `comment='#'` ignores everything after `#`, including whole comment lines
- Open the file in an editor first : the extension does not guarantee the separator

demo17
More read options

SpiralTrain developer courses

Assemble Dates

- `pd.to_datetime` assembles one date from `year`, `month` and `day` columns
- `drop(columns=...)` removes the three columns once the date exists

```
sunspots['date'] = pd.to_datetime(sunspots[['year', 'month', 'day']])
sunspots = sunspots.drop(columns=['year', 'month', 'day'])
print(sunspots.iloc[10:13])
print(sunspots['date'].iloc[[0, -1]].tolist())
```

```
    dec_date  sunspots  definite       date
10  1818.031       NaN         1 1818-01-11
11  1818.034       NaN         1 1818-01-12
12  1818.037      22.0         1 1818-01-13
[Timestamp('1818-01-01 00:00:00'), Timestamp('2014-09-30 00:00:00')]
```

- The first and last dates confirm the range : 1818 to 2014
- Older code passed nested lists to `parse_dates` ; pandas 3 removed that form

demo18
Assemble dates

Check the Resulting Schema

- **Check the dtypes after each conversion :**

```
print(sunspots.dtypes)
```

```
dec_date           float64
sunspots           float64
definite             int64
date         datetime64[us]
dtype: object
```

- **`datetime64[us]` stores timestamps with microsecond resolution**
- **Typical dtypes :**
  - `int64` and `float64` for numbers, `bool` for True/False
  - `str` for text, `datetime64` for timestamps
  - `category` for a few repeated values (Module 8)

SpiralTrain developer courses

Use Dates as the Index

- **Check the dates before making them the index :**
  - Unique dates give one row per label
  - Increasing dates make date ranges reliable

```
sunspots = sunspots.set_index('date')
print(sunspots.index.is_unique, sunspots.index.is_monotonic_increasing)
print(sunspots.iloc[10:13])
```

```
True True
            dec_date  sunspots  definite
date
1818-01-11  1818.031       NaN         1
1818-01-12  1818.034       NaN         1
1818-01-13  1818.037      22.0         1
```

- `set_index` **moves the column into the index ; its name becomes the index name**

demo19
Dates as index

Trim Redundant Columns

- `dec_date` repeats the information in the index :
  - Selecting a list of columns keeps only those columns

```
sunspots = sunspots[['sunspots', 'definite']]
print(sunspots.iloc[10:13])
```

```
            sunspots  definite
date
1818-01-11       NaN         1
1818-01-12       NaN         1
1818-01-13      22.0         1
```

demo20
Trimming redundant columns

Write Files

```
sunspots.to_csv('sunspots.csv')
sunspots.to_csv('sunspots.tsv', sep='\t')
sunspots.to_excel('sunspots.xlsx')

summary = pd.DataFrame({
    'statistic': ['mean', 'std'],
    'sunspots': [sunspots['sunspots'].mean(),
                 sunspots['sunspots'].std()]})
summary.to_csv('summary.csv', index=False,
               float_format='%.2f')
```

```
statistic,sunspots
mean,53.76
std,51.11
```

- The row index is written by default ; `index=False` leaves it out
- `sep='\t'` writes tab-separated values ; `float_format` rounds floats in the file
- `to_excel` needs the `openpyxl` package, which is in the course environment
- CSV keeps no dtypes : parse dates again when reading it back
- `restored.equals(original)` is `True` when two frames have the same labels, values and dtypes

demo21
Writing files

SpiralTrain developer courses

Summary : Pandas Intro

- **pandas offers the labeled `Series` and `DataFrame` :**
  - Build them from dictionaries and lists, or read them from files
  - A scalar is broadcast to every row of a new column
- **Inspect every new table :**
  - `shape`, `head`, `tail`, `info`, `describe`, `isna().sum()`
- **Read files deliberately :**
  - Look at the raw file first
  - Declare `header`, `names`, `sep`, `skiprows`, `comment` and `na_values` as needed
  - Assemble and check dates, then choose the index
- **Write results with `to_csv` and `to_excel`**

- NumPy
- Pandas Intro
- ▶ **Selecting and Filtering**
  - The Sales Table
  - Selecting a Value with .loc
  - Slices with .loc
  - Lists with .loc
  - Selecting by Position with .iloc
  - Lists and Negative Positions with .iloc
  - Selecting Columns
  - Series versus One-Column DataFrame
  - Selecting from a Series
  - Bracket and Attribute Access
  - Creating a Boolean Series
  - Filtering with a Boolean Series
  - Combining Filters
  - isin and between
  - The Shop Orders Dataset
  - Sorting
  - Largest and Smallest
  - Counting Values
  - unique, nunique and median
- Cleaning and Transforming
- Advanced Indexing
- Rearranging Data
- Combining Data
- Grouping Data
- Matplotlib
- Time Series
- Integration Case
- Scikit-Learn Essentials
- Data Formats and Tools

APPENDIX

- Reading Other Sources

SpiralTrain developer courses

The Sales Table

- **Most examples in this module use a small sales table :**
  - Six months as row labels, three products as columns
  - May has no recorded salt sales : `NaN`

```
import pandas as pd

df = pd.read_csv('sales.csv', index_col='month')
print(df)
print(df.shape)
```

```
       eggs  salt  spam
month
Jan      47  12.0    17
Feb     110  50.0    31
Mar     221  89.0    72
Apr      77  87.0    20
May     132   NaN    52
Jun     205  60.0    55
(6, 3)
```

- **`index_col='month'` makes the months the row labels**
- **Selection uses those labels, or positions**

demo01
Sales table

SpiralTrain developer courses

Selecting a Value with .loc

- `.loc[row, column]` selects by labels :
  - Row label first, column label second
  - Both inside one pair of square brackets

```
print(df.loc['May', 'spam'])   # 52
print(df.loc['Jan', 'salt'])   # 12.0
```

- `.loc` is the main way to select, and the way to assign (Module 4)
- A label that does not exist raises a `KeyError`

demo02
loc single value

Slices with .loc

- A slice selects a range of labels ; `:` alone selects everything

```
# some rows, all columns
print(df.loc['Jan':'Apr', :])
```

```
       eggs  salt  spam
month
Jan      47  12.0    17
Feb     110  50.0    31
Mar     221  89.0    72
Apr      77  87.0    20
```

```
# all rows, some columns
print(df.loc[:, 'eggs':'salt'])
```

```
       eggs  salt
month
Jan      47  12.0
Feb     110  50.0
Mar     221  89.0
Apr      77  87.0
May     132   NaN
Jun     205  60.0
```

- Label slices include both endpoints : `'Jan':'Apr'` includes April
- A step of `-1` walks backwards : `df.loc['Apr':'Jan':-1]` lists April down to January

demo03
loc slices

SpiralTrain developer courses

Lists with .loc

- **A list selects exactly the listed labels, in that order :**
  - Lists and slices can be mixed in one selection

```
print(df.loc[['Jan', 'Jun'], ['eggs', 'spam']])
```

```
       eggs  spam
month
Jan      47    17
Jun     205    55
```

```
print(df.loc['Feb':'Apr', ['spam', 'eggs']])
```

```
       spam  eggs
month
Feb      31   110
Mar      72   221
Apr      20    77
```

- **Note the double brackets : the outer pair indexes, the inner pair is a list**

demo04
loc lists

Selecting by Position with .iloc

- `.iloc[row, column]` selects by zero-based positions :
  - Row 4 is the fifth row, May ; column 2 is the third column, spam
  - Position slices exclude the stop position, as in Python lists

```
print(df.iloc[4, 2])
print(df.iloc[2:5, 1:])
print(df.loc['Mar':'May', 'salt':'spam'])
```

```
52
       salt  spam
month
Mar    89.0    72
Apr    87.0    20
May     NaN    52
```

```
       salt  spam
month
Mar    89.0    72
Apr    87.0    20
May     NaN    52
```

- Both select the same block ; positions change when rows are sorted or filtered

demo05
iloc

Lists and Negative Positions with .iloc

```
print(df.iloc[[0, 4, 5], [0, 2]])
```

```
       eggs  spam
month
Jan      47    17
May     132    52
Jun     205    55
```

```
print(df.iloc[-2:, :])
```

```
       eggs  salt  spam
month
May     132   NaN    52
Jun     205  60.0    55
```

- Lists of positions select specific rows or columns
- Negative positions count from the end : `-1` is the last row
- Use `.loc` when the labels mean something ; use `.iloc` for ”the first n” or ”the last n”

demo06
iloc lists

Selecting Columns

```
print(df['eggs'])
```

```
month
Jan     47
Feb    110
Mar    221
Apr     77
May    132
Jun    205
Name: eggs, dtype: int64
```

```
print(df[['salt', 'eggs']])
```

```
       salt  eggs
month
Jan    12.0    47
Feb    50.0   110
Mar    89.0   221
Apr    87.0    77
May     NaN   132
Jun    60.0   205
```

- **Square brackets with a column label are the short form for whole columns :**
  - `df['eggs']` is the same as `df.loc[:, 'eggs']`
  - A list of labels gives a `DataFrame` with those columns, in that order

demo07
Selecting columns

SpiralTrain developer courses

Series versus One-Column DataFrame

- A single label gives a `Series` ; a list with one label gives a `DataFrame`

```
print(type(df['eggs']).__name__, df['eggs'].shape)       # Series (6,)
print(type(df[['eggs']]).__name__, df[['eggs']].shape)   # DataFrame (6, 1)
```

- A `Series` is one-dimensional : its shape has one number
- A one-column `DataFrame` is still a table : rows and one column
- Choose the form the next step expects :
  - Arithmetic and statistics on one column : `Series`
  - A table to print, write or merge : `DataFrame`

demo08
Series versus DataFrame

SpiralTrain developer courses

Selecting from a Series

- A `Series` has the same `.loc` and `.iloc`, with one position per selection

```
eggs = df['eggs']
print(eggs.loc['Feb':'Apr'])
print(eggs.iloc[0], eggs.iloc[-1])
```

```
month
Feb    110
Mar    221
Apr     77
Name: eggs, dtype: int64
47 205
```

- `.loc` uses labels, including integer labels : `s.loc[1]` means the label 1
- `.iloc` always uses positions : `s.iloc[1]` means the second value

demo09
Selecting from a Series

SpiralTrain developer courses

Bracket and Attribute Access

- Older code and online answers often use two other forms :

```
# column, then row: two steps
print(df['salt']['Jan'])                                    # 12.0
# column as an attribute
print(df.eggs['Mar'])                                       # 221
print(df.loc['Jan', 'salt'], df.loc['Mar', 'eggs'])         # 12.0 221
```

- Both read the same values as `.loc`, but :
  - `df['salt']['Jan']` selects in two steps ; assigning through it does not change `df`
  - Attribute access fails for labels with spaces or names such as `count`
- Read them, but write `.loc` in your own code

demo10
Shortcuts

Creating a Boolean Series

```
mask = df['salt'] > 60
print(mask)
```

```
month
Jan    False
Feb    False
Mar     True
Apr     True
May    False
Jun    False
Name: salt, dtype: bool
```

- A comparison on a column compares every value :
  - The result is a Boolean `Series` with the same index
  - Such a `Series` is called a mask
- Comparisons : `==` , `!=` , `<` , `<=` , `>` , `>=`
- A missing value compares as `False` :
  - May has no salt value, so its mask value is `False`

demo11
Boolean Series

Filtering with a Boolean Series

```
print(df[df['salt'] > 60])

enough_salt = df['salt'] > 60
print(df.loc[enough_salt, ['salt', 'spam']])
```

```
       eggs  salt  spam
month
Mar     221  89.0    72
Apr      77  87.0    20
       salt  spam
month
Mar    89.0    72
Apr    87.0    20
```

- A mask inside square brackets keeps the rows where it is `True`
- `.loc[mask, columns]` filters rows and selects columns in one step
- A named mask makes the condition readable and reusable
- The mask aligns with the frame by index label

demo12
Filtering

SpiralTrain developer courses

Combining Filters

- `&` (and), `|` (or) and `~` (not) combine masks element by element :
  - Each comparison needs its own parentheses, because `&` binds tighter than `>=`
  - Python's `and`, `or` and `not` raise an error on a `Series`

```
print(df[(df['salt'] >= 50) & (df['eggs'] < 200)])
print(df[~(df['salt'] >= 50)])
```

```
       eggs  salt  spam
month
Feb     110  50.0    31
Apr      77  87.0    20
       eggs  salt  spam
month
Jan      47  12.0    17
May     132   NaN    52
```

- Both conditions : February and April
- `~` keeps the rows where the condition is `False` :
  - January has too little salt
  - May has no salt value, so `>= 50` is `False` and `~` makes it `True`
- `|` keeps rows where either condition holds

demo13
Combining filters

isin and between

```
print(df[df['eggs'].between(100, 210)])

first_quarter = df.index.isin(['Jan', 'Feb', 'Mar'])
print(df[first_quarter])
```

```
       eggs  salt  spam
month
Feb     110  50.0    31
May     132   NaN    52
Jun     205  60.0    55
       eggs  salt  spam
month
Jan      47  12.0    17
Feb     110  50.0    31
Mar     221  89.0    72
```

- `between(low, high)` tests a range :
  - Both ends are included by default
  - The same as `(s >= low) & (s <= high)`
- `isin(list)` tests membership in a list of values :
  - Works on a column or on the index
  - Replaces a chain of `==` conditions joined by `|`

demo14
isin and between

The Shop Orders Dataset

```
shop = pd.read_csv('shop.csv')
print(shop.head())
shop.info()
```

```
  order_id        date        city product  units  price
0     A001  2026-03-02   Amsterdam  Coffee      3   4.50
1     A002  2026-03-02     Utrecht     Tea      1   3.20
2     A003  2026-03-03   amsterdam  Coffee      2   4.50
3     A004  2026-03-03   Rotterdam    Cake      4   5.75
4     A005  2026-03-04     Utrecht  Coffee      5   4.50
```

```
RangeIndex: 12 entries, 0 to 11
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   order_id  12 non-null     str
 1   date      12 non-null     str
 2   city      12 non-null     str
 3   product   12 non-null     str
 4   units     12 non-null     int64
 5   price     12 non-null     float64
```

- Twelve orders from a small coffee chain, one row per order
- No `index_col` : the rows keep the default 0 to 11 labels

demo15
Shop table

Sorting

```
print(shop.sort_values('units', ascending=False).head(3))
print(shop.sort_values(['product', 'units']).head(4))
```

```
  order_id        date        city product  units  price
7     A008  2026-03-05  Amsterdam     Cake      6   5.75
4     A005  2026-03-04     Utrecht  Coffee      5   4.50
3     A004  2026-03-03   Rotterdam    Cake      4   5.75
   order_id        date        city product  units  price
11     A012  2026-03-07     Utrecht    Cake      2   5.75
3      A004  2026-03-03   Rotterdam    Cake      4   5.75
7      A008  2026-03-05  Amsterdam     Cake      6   5.75
6      A007  2026-03-05   Rotterdam  Coffee      1   4.50
```

- `sort_values(column)` sorts the rows by a column ; `ascending=False` reverses it
- A list sorts by the first column, then by the next within ties
- The row labels move with the rows ; `sort_index()` sorts by the labels instead

demo16
Sorting

Largest and Smallest

```
print(shop.nlargest(3, 'units'))
print(shop['units'].idxmax(), shop['units'].max())
print(shop.loc[shop['units'].idxmax(), 'order_id'])
```

```
  order_id        date        city product  units  price
7     A008  2026-03-05  Amsterdam     Cake      6   5.75
4     A005  2026-03-04     Utrecht  Coffee      5   4.50
3     A004  2026-03-03   Rotterdam    Cake      4   5.75
7 6
A008
```

- `nlargest(n, column)` and `nsmallest` give the top or bottom rows directly
- `idxmax()` and `idxmin()` return the label of the largest or smallest value :
  - Combine it with `.loc` to look up other columns of that row

demo17
Largest

Counting Values

```
print(shop['product'].value_counts())
print(shop['product']
      .value_counts(normalize=True).round(2))
```

```
product
Coffee    6
Tea       3
Cake      3
Name: count, dtype: int64
product
Coffee    0.50
Tea       0.25
Cake      0.25
Name: proportion, dtype: float64
```

```
print(shop['city'].value_counts())
```

```
city
Utrecht       4
Rotterdam     3
Amsterdam     2
 amsterdam    1
AMSTERDAM     1
Amsterdam     1
Name: count, dtype: int64
```

- Four spellings of Amsterdam ; Module 4 cleans them

- `value_counts()` counts each distinct value, most frequent first
- `normalize=True` gives shares instead of counts

demo18
Value counts

SpiralTrain developer courses

unique, nunique and median

```
print(shop['product'].unique().tolist())   # ['Coffee', 'Tea', 'Cake']
print(shop['product'].nunique())           # 3
print(shop['units'].median(), shop['units'].mean())
# 2.5 2.9166666666666665
```

- `unique()` lists the distinct values in order of appearance
- `nunique()` counts them
- Summary methods on a column : `sum` , `mean` , `median` , `min` , `max` , `std` , `count`
- The median is the middle value : one large order moves the mean, not the median

demo19
unique

SpiralTrain developer courses

Summary : Selecting and Filtering

- `.loc[rows, columns]` selects by labels ; label slices include the end
- `.iloc[rows, columns]` selects by positions ; position slices exclude the end
- `df['col']` gives a `Series`, `df[['a', 'b']]` a `DataFrame`
- Filter rows with Boolean masks :
  - Combine them with `&`, `|` and `~`, with parentheses around each comparison
  - `isin` and `between` express common conditions
- Explore with `sort_values`, `nlargest`, `idxmax`, `value_counts`, `unique` and `nunique`

- NumPy
- Pandas Intro
- Selecting and Filtering
- ▶ **Cleaning and Transforming**
  - Zeros and NaNs
  - all and any
  - Finding Missing Values
  - Dropping Missing Values
  - Filling Missing Values
  - Missing Values and Integer Columns
  - Text and Nullable dtypes
  - Modifying Values with .loc
  - Chained Assignment Does Not Work
  - Copy-on-Write in pandas 3
  - Vectorized Arithmetic
  - NumPy Functions on Columns
  - Applying a Python Function
  - Rows and Single Values
  - Mapping Values with a Dictionary
  - Changing Types
  - Renaming and Dropping Columns
  - Cleaning Text Columns
  - Testing Text
  - Storing a Transformation
- Advanced Indexing
- Rearranging Data
- Combining Data
- Grouping Data
- Matplotlib
- Time Series
- Integration Case
- Scikit-Learn Essentials
- Data Formats and Tools

APPENDIX

- Reading Other Sources

Zeros and NaNs

```
df = pd.read_csv('sales.csv', index_col='month')
df2 = df.copy()
df2['bacon'] = [0, 0, 50, 60, 70, 80]
print(df2)
```

```
       eggs  salt  spam  bacon
month
Jan      47  12.0    17      0
Feb     110  50.0    31      0
Mar     221  89.0    72     50
Apr      77  87.0    20     60
May     132   NaN    52     70
Jun     205  60.0    55     80
```

- The sales table from Module 3, plus a `bacon` column
- `df.copy()` gives an independent frame to modify
- A list with one value per row creates a new column
- `df2` holds two kinds of ”empty” values :
  - `bacon` has zeros in January and February
  - `salt` has a missing value in May
- A zero is a measured value ; `NaN` is an unknown value

demo01
Zeros and NaNs

all and any

```
print(df2.all())
print(df2.loc[:, df2.all()])
```

```
eggs      True
salt      True
spam      True
bacon    False
dtype: bool
       eggs  salt  spam
month
Jan      47  12.0    17
...
```

- `all()` is `True` for a column without zeros
- `any()` is `True` for a column with at least one nonzero value
- A Boolean `Series` over the columns selects columns with `.loc[:, mask]`
- `all()` and `any()` skip `NaN`, so `salt` passes `all()`

demo02
all and any

Finding Missing Values

```
print(df2.isna().sum())
print(df2.loc[:, df2.isna().any()])
print(df2[df2['salt'].isna()])
```

```
eggs     0
salt     1
spam     0
bacon    0
dtype: int64
       salt
month
Jan    12.0
...
May     NaN
Jun    60.0
       eggs  salt  spam  bacon
month
May     132   NaN    52     70
```

- `isna()` marks missing values, `notna()` present values
- `isna().sum()` counts the missing values per column
- `isna().any()` selects the columns with a gap
- A mask on one column shows the rows with a gap
- `isnull()` and `notnull()` are older aliases
- Use `isna()`, not `== np.nan` : `NaN` is never equal to anything, not even itself

demo03
Finding missing values

Dropping Missing Values

```
print(df.dropna())
print(df.dropna(subset=['eggs', 'spam']).shape)
print(df.dropna(axis='columns').columns.tolist())
print(df.dropna(axis='columns', thresh=5)
        .columns.tolist())
```

```
       eggs  salt  spam
month
Jan      47  12.0    17
Feb     110  50.0    31
Mar     221  89.0    72
Apr      77  87.0    20
Jun     205  60.0    55
(6, 3)
['eggs', 'spam']
['eggs', 'salt', 'spam']
```

- `dropna()` drops each row with a missing value ; `how='all'` only all-missing rows
- `subset=` only looks at the listed columns
- `axis='columns'` drops columns instead of rows
- `thresh=n` keeps rows or columns with at least `n` non-missing values :
  - `salt` has 5 values, so it survives `thresh=5`
- Dropping May also loses its eggs and spam sales

demo04
Dropping missing values

Filling Missing Values

```
print(df.fillna(0).loc['May'])
print(df['salt'].fillna(df['salt'].median()).loc['May'])
```

```
eggs    132.0
salt      0.0
spam     52.0
Name: May, dtype: float64
60.0
```

- `fillna(value)` replaces missing values ; it returns a new frame or `Series`
- **Each fill is a claim about the data :**
  - `0` claims that no salt was sold in May
  - The median claims that May was a typical month
- **Choose a fill only when it is justified, and report how many values were filled**

demo05
Filling missing values

Missing Values and Integer Columns

```
visits = pd.DataFrame({'city': ['Paris', 'Berlin', 'Rome'],
                       'visitors': [139, 237, 326]})
print(visits['visitors'].dtype)
visits.loc[1, 'visitors'] = float('nan')
print(visits['visitors'].dtype)
print(visits)

counts = pd.Series([139, None, 326], dtype='Int64')
print(counts)
```

```
int64
float64
     city  visitors
0   Paris     139.0
1  Berlin       NaN
2    Rome     326.0
0     139
1    <NA>
2     326
dtype: Int64
```

- `NaN` is a float : a missing value turns an `int64` column into `float64`
- The nullable `Int64` dtype (capital I) keeps integers and marks gaps as `<NA>`

demo06
Missing values and dtypes

SpiralTrain developer courses

Text and Nullable dtypes

```
text = pd.Series(['Paris', None])
names = pd.Series(['Paris', None], dtype='string')
counts = pd.Series([7, None], dtype='Int64')
flags = pd.Series([True, None], dtype='boolean')
print(text.dtype, names.dtype, counts.dtype, flags.dtype)
print(text.isna().tolist(), counts.isna().tolist())
print((counts > 5).tolist())
```

```
str string Int64 boolean
[False, True] [False, True]
[True, <NA>]
```

- pandas 3 stores text as `str` by default ; older versions showed `object`
- `string` , `Int64` and `boolean` are nullable types : a gap is `pd.NA`
- A comparison with `pd.NA` gives `<NA>` , not `False` : unknown stays unknown
- `isna()` works the same for `NaN` and `pd.NA`

demo07
Text and nullable dtypes

Modifying Values with .loc

```
df2 = df.copy()
mask = df2['salt'] > 55
print(df2.loc[mask, 'eggs'])
df2.loc[mask, 'eggs'] += 5
print(df2)
```

```
month
Mar    221
Apr     77
Jun    205
Name: eggs, dtype: int64
```

```
       eggs  salt  spam
month
Jan      47  12.0    17
Feb     110  50.0    31
Mar     226  89.0    72
Apr      82  87.0    20
May     132   NaN    52
Jun     210  60.0    55
```

- One `.loc` assignment updates the frame itself
- Selected rows gain 5 ; other rows keep their values
- `df` itself is unchanged : the demo works on a copy

demo08
Modifying with loc

Chained Assignment Does Not Work

```
df2 = df.copy()
mask = df2['salt'] > 55
df2['eggs'][mask] += 5
print(df2['eggs'].tolist())
```

```
ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series
through chained assignment.
[47, 110, 221, 77, 132, 205]
```

- `df2['eggs']` returns a `Series` ; `[mask] += 5` changes that temporary object
- `df2` stays unchanged, and pandas warns with a `ChainedAssignmentError`
- Select rows and column in one `.loc` call :
  - `df2.loc[mask, 'eggs'] += 5` is the form that updates the frame

demo09
Chained assignment

Copy-on-Write in pandas 3

```python
df = pd.DataFrame({'visitors': [139, 237]})
column = df['visitors']
column.iloc[0] = 0
print(df['visitors'].tolist())   # [139, 237]
df.loc[0, 'visitors'] = 0
print(df['visitors'].tolist())   # [0, 237]
```

- Every selection behaves as an independent object :
  - Writing to `column` leaves `df` unchanged
  - pandas copies the data only when one of them is changed
- To change a frame, assign through `.loc` or `.iloc` on that frame
- This is the rule behind the previous slide

demo10
Copy-on-Write

Vectorized Arithmetic

```
print(df // 12)   # dozens
print(df['eggs'] + df['spam'])
print(df['eggs'].add(df['salt'], fill_value=0))
```

```
       eggs  salt  spam
month
Jan       3   1.0     1
Feb       9   4.0     2
...
month
Jan      64
Feb     141
...
month
...
May     132.0
Jun     265.0
dtype: float64
```

- Operators work on whole columns at once, in compiled code :
  - `+`, `-`, `*`, `/`, `//`, `**`
  - Two columns are matched by row label
- The method forms `add`, `sub`, `mul`, `div` and `floordiv` accept options :
  - `fill_value=0` treats a missing value as 0 for this calculation
  - May's eggs plus missing salt gives 132 instead of `NaN`

demo11
Vectorized arithmetic

NumPy Functions on Columns

```
import numpy as np

print(np.sqrt(df['eggs']).round(2))
print(np.log10(df).round(2))
```

```
month
Jan     6.86
Feb    10.49
...
       eggs  salt  spam
month
Jan    1.67  1.08  1.23
Feb    2.04  1.70  1.49
...
May    2.12   NaN  1.72
```

- NumPy functions accept a `Series` or `DataFrame` :
  - The result keeps the row and column labels
  - The loop runs in compiled code
- Useful for maths without a pandas method :
  - `np.log` , `np.log10` , `np.sqrt` , `np.exp` , `np.abs`
- A missing input gives a missing result

demo12
NumPy functions

Applying a Python Function

```
def dozens(n):
    return n // 12

print(df.apply(dozens))
print(df.apply(lambda column: type(column).__name__))
```

```
       eggs  salt  spam
month
Jan       3   1.0     1
Feb       9   4.0     2
...
eggs    Series
salt    Series
spam    Series
dtype: str
```

- `apply` calls a function once for each column :
  - The function receives a whole column as a `Series`
  - The results are assembled into a new table
- Here `n // 12` is vectorized arithmetic, so `df // 12` gives the same result
- `apply` suits logic that has no vectorized form
- `lambda` writes a small function inline

demo13
apply

Rows and Single Values

```
print(df.apply(lambda row: row.max() - row.min(), axis=1))
print(df.map(lambda value: f'{value:.0f}').loc['Jan'])
```

```
month
Jan     35.0
Feb     79.0
Mar    149.0
...
eggs    47
salt    12
spam    17
Name: Jan, dtype: str
```

- `apply(f, axis=1)` calls `f` once per row ; each row arrives as a `Series`
- `df.map(f)` calls `f` once per value, for every value in the frame
- Both run Python once per row or value : slower than vectorized code on large tables
- `np.vectorize` accepts arrays, but still loops in Python

demo14
Row apply and map

SpiralTrain developer courses

Mapping Values with a Dictionary

```
shop = pd.read_csv('shop.csv')
kind = {'Coffee': 'drink', 'Tea': 'drink'}
shop['kind'] = shop['product'].map(kind)
print(shop[['product', 'kind']].head(4))
```

```
  product   kind
0  Coffee  drink
1     Tea  drink
2  Coffee  drink
3    Cake    NaN
```

- `Series.map(dictionary)` looks up each value in the dictionary
- A value that is not a key, or a missing value, maps to `NaN`
- Missing results show which values the mapping does not cover :
  - Safer than a rule that silently labels every other value

demo15
map with a dictionary

Changing Types

```
raw = pd.DataFrame({'units': ['3', '1', '2'],
                    'price': ['4.50', '3.20', 'n/a']})
print(raw.dtypes)
raw['units'] = raw['units'].astype('int64')
raw['price'] = pd.to_numeric(raw['price'],
                             errors='coerce')
print(raw.dtypes)
print(raw)
```

```
units    str
price    str
dtype: object
units      int64
price    float64
dtype: object
   units  price
0      3    4.5
1      1    3.2
2      2    NaN
```

- **`astype(dtype)` converts a column : `'int64'`, `'float64'`, `'str'`, `'category'`**
  - It raises an error when a value cannot be converted
- **`pd.to_numeric(..., errors='coerce')` turns unconvertible values into `NaN` :**
  - Count the new `NaN` values : coercion can hide bad input

demo16
astype

SpiralTrain developer courses

Renaming and Dropping Columns

```
orders = shop.rename(columns={'units': 'quantity'}).drop(columns=['date'])
print(orders.columns.tolist())
print(shop.columns.tolist())
```

```
['order_id', 'city', 'product', 'quantity', 'price']
['order_id', 'date', 'city', 'product', 'units', 'price']
```

- `rename(columns={old: new})` renames selected columns
- `drop(columns=[...])` removes columns ; `drop(index=[...])` removes rows
- Both return a new frame ; `shop` keeps its original columns
- Methods that return frames can be chained in one expression

demo17
Rename and drop

Cleaning Text Columns

```
print(shop['city'].unique().tolist())
shop['city'] = shop['city'].str.strip().str.title()
print(shop['city'].value_counts())
```

```
['Amsterdam', 'Utrecht', ' amsterdam', 'Rotterdam', 'AMSTERDAM', 'Amsterdam ']
city
Amsterdam    5
Utrecht      4
Rotterdam    3
Name: count, dtype: int64
```

- The `.str` accessor applies a string method to every value of a text column :
  - `strip()` removes surrounding spaces ; `lower()` , `upper()` , `title()` fix case
  - `replace(old, new)` replaces text inside each value
- Four spellings of Amsterdam become one : five Amsterdam orders, not two
- Labels have `.str` too : `df.columns = df.columns.str.strip()` cleans column names
- `+` joins text columns value by value : `df['first'] + ' ' + df['last']`

demo18
Cleaning text

Testing Text

```
print(shop['product'].str.startswith('C').tolist())
print(shop.loc[shop['product'].str.contains('a'), 'product'].unique().tolist())
print(shop['order_id'].str.fullmatch(r'A\d{3}').all())
print(shop['order_id'].str[1:].str.isdigit().all())
print(shop['order_id'].str.lower().head(2).tolist())
```

```
[True, False, True, True, True, False, True, True, False, True, True, True]
['Tea', 'Cake']
True
True
['a001', 'a002']
```

- `contains`, `startswith` and `endswith` give a Boolean mask for filtering
- `fullmatch(pattern)` checks that the whole value matches a pattern : `A` plus 3 digits
- `isdigit()` checks that a value consists of digits only
- `.str[1:]` slices every value, like a Python string

demo19
Text tests

Storing a Transformation

```
shop['city'] = shop['city'].str.strip().str.title()
shop['revenue'] = shop['units'] * shop['price']
print(shop[['order_id', 'city', 'units', 'price', 'revenue']].head(4))
print(shop['revenue'].sum().round(2))
```

```
  order_id       city  units  price  revenue
0     A001  Amsterdam      3   4.50     13.5
1     A002    Utrecht      1   3.20      3.2
2     A003  Amsterdam      2   4.50      9.0
3     A004  Rotterdam      4   5.75     23.0
164.7
```

- A transformation does not change the frame until its result is assigned
- Assigning to a new label adds a column ; assigning to an existing label replaces it

demo20
Storing a transformation

SpiralTrain developer courses

Summary : Cleaning and Transforming

- **Missing data : find it with `isna()`, then decide :**
  - `dropna` with `how`, `subset`, `axis` and `thresh`, or `fillna` with a justified value
  - Nullable `Int64`, `boolean` and `string` keep their type with gaps
- **Change a frame with one `.loc` assignment ; chained assignment does not work**
- **Transform with vectorized operators and NumPy functions first :**
  - `apply` and `map` for logic without a vectorized form ; `map(dict)` for lookups
- **Fix types with `astype` and `pd.to_numeric`, names with `rename`, columns with `drop`**
- **Clean and test text with the `.str` accessor**

exercise4
Cleaning and transforming

- NumPy
- Pandas Intro
- Selecting and Filtering
- Cleaning and Transforming
- ▶ **Advanced Indexing**
- Rearranging Data
- Combining Data
- Grouping Data
- Matplotlib
- Time Series
- Integration Case
- Scikit-Learn Essentials
- Data Formats and Tools

APPENDIX

- Reading Other Sources

- Choose Labels from the Data
- Labels and Positions
- Replacing Index Labels
- Transforming All Labels
- Index and Column Names
- Arithmetic Aligns Labels
- Reindexing
- Duplicate Labels
- MultiIndex
- Core Indexing Checklist
- Set and Sort a MultiIndex
- Select an Outer Level
- Lists Within a Level
- Select an Inner Level

SpiralTrain developer courses

Choose Labels from the Data

- The row index supplies labels ; use a column that identifies the observations
- `set_index(column)` turns that column into the index

```
sales = pd.read_csv('sales.csv')
print(sales.head(3))
sales = sales.set_index('month')
print(sales.index.is_unique)
print(sales.loc['Feb', 'eggs'], sales.iloc[1, 0])
```

```
  month  eggs  salt  spam
0   Jan    47  12.0    17
1   Feb   110  50.0    31
2   Mar   221  89.0    72
True
110 110
```

- Labels come from the rows themselves : reordering the file cannot mislabel them
- Check `index.is_unique` when one label must identify one observation
- `reset_index()` turns the index back into an ordinary column

demo01
Labels from data

Labels and Positions

```
prices = pd.Series([10.70, 10.86, 10.74], index=['Mon', 'Tue', 'Wed'])
print(prices.loc['Tue'], prices.iloc[1])   # 10.86 10.86
print(prices.loc['Mon':'Tue'].tolist())    # [10.7, 10.86]
print(prices.iloc[0:2].tolist())           # [10.7, 10.86]

numbered = pd.Series([10.70, 10.86, 10.74], index=[3, 2, 1])
print(numbered.loc[1], numbered.iloc[1])   # 10.74 10.86
```

- `.loc` uses labels ; label slices include both endpoints
- `.iloc` uses positions ; the stop position is excluded
- An integer label is still a label : `numbered.loc[1]` is the last value here

Replacing Index Labels

```
shares = pd.Series([10.70, 10.86], index=['Mon', 'Tue'])
try:
    shares.index[0] = 'Monday'
except TypeError as error:
    print('TypeError:', error)

shares = shares.rename(index={'Mon': 'Monday',
                              'Tue': 'Tuesday'})
print(shares)

shares.index = ['M', 'T']
print(shares.index.tolist())
```

```
TypeError: Index does not support mutable
operations
Monday     10.70
Tuesday    10.86
dtype: float64
['M', 'T']
```

- An `Index` is immutable : single entries cannot be assigned
- `rename(index={old: new})` replaces selected labels
- Assigning a whole list to `.index` attaches the labels by position :
  - Derive labels from the data before rows are reordered

demo03
Replacing labels

Transforming All Labels

```
sales = pd.read_csv('sales.csv', index_col='month')
print(sales.index)
sales.index = sales.index.str.upper()
print(sales.index)
sales.index = sales.index.map(lambda month: month[0])
print(sales.index.tolist())
```

```
Index(['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'], dtype='str', name='month')
Index(['JAN', 'FEB', 'MAR', 'APR', 'MAY', 'JUN'], dtype='str', name='month')
['J', 'F', 'M', 'A', 'M', 'J']
```

- `index.str` applies string methods to every label, as for a text column
- `index.map(function)` applies any function to every label
- The result replaces the whole index in one assignment
- The first letters are not unique : check `is_unique` before using such labels

Index and Column Names

```
sales = pd.read_csv('sales.csv', index_col='month')
sales.index.name = 'MONTHS'
sales.columns.name = 'PRODUCTS'
print(sales.head(3))
print(sales.reset_index().columns.tolist())
```

```
PRODUCTS  eggs  salt  spam
MONTHS
Jan         47  12.0    17
Feb        110  50.0    31
Mar        221  89.0    72
['MONTHS', 'eggs', 'salt', 'spam']
```

- `index.name` names the row axis ; `columns.name` names the column axis
- The names describe the axes ; labels such as `Jan` and `eggs` identify single rows and columns
- `reset_index()` uses the index name as the new column label

demo05
Index names

Arithmetic Aligns Labels

```
sales = pd.Series([47, 110], index=['Jan', 'Feb'],
                  name='eggs')
adjustment = pd.Series([2, 1], index=['Feb', 'Jan'])
print(sales + adjustment)

partial = pd.Series([2], index=['Feb'])
print(sales + partial)
print(sales.add(partial, fill_value=0))
```

```
Feb    112
Jan     48
dtype: int64
Feb    112.0
Jan      NaN
dtype: float64
Feb    112.0
Jan     47.0
dtype: float64
```

- pandas matches labels, even when the operands have different row orders
- A label without a partner gives a missing result
- `fill_value=0` is a decision that a missing adjustment means zero ; make it deliberately

demo06
Alignment

Reindexing

```
sales = pd.read_csv('sales.csv', index_col='month')
print(sales.reindex(['Jun', 'Jan', 'Jul']))
print(sales['eggs'].reindex(['Jan', 'Jul'],
                            fill_value=0))
```

```
       eggs  salt  spam
month
Jun   205.0  60.0  55.0
Jan    47.0  12.0  17.0
Jul     NaN   NaN   NaN
month
Jan    47
Jul     0
Name: eggs, dtype: int64
```

- `reindex(labels)` returns the rows in the order of the given labels
- A label that does not exist gets a row of missing values
- `fill_value` fills those new rows ; Module 10 shows reindexing on dates

demo07
reindex

Duplicate Labels

```
sales = pd.DataFrame({
    'state': ['CA', 'NY', 'NY'],
    'month': ['Jan', 'Jan', 'Feb'],
    'eggs': [47, 221, 77]}).set_index('state')
print(sales.index.is_unique)
print(sales.loc['NY'])
print(sales.iloc[1])
try:
    sales.reindex(['CA', 'NY'])
except ValueError as error:
    print('ValueError:', error)
```

```
False
      month  eggs
state
NY      Jan   221
NY      Feb    77
month    Jan
eggs     221
Name: NY, dtype: object
ValueError: cannot reindex on an axis with
duplicate labels
```

- Repeated state labels are valid : these are different monthly observations
- A label lookup can return several rows ; `reindex` refuses to choose between them

demo08
Duplicate labels

MultiIndex

```
rows = pd.DataFrame({
    'state': ['CA', 'CA', 'NY'],
    'month': ['Jan', 'Feb', 'Jan'],
    'eggs': [47, 110, 221]})
by_state_month = rows.set_index(['state', 'month'])
print(by_state_month)
print(by_state_month.index.is_unique,
      by_state_month.index.names)
print(by_state_month.loc[('CA', 'Feb'), 'eggs'])
```

```
              eggs
state month
CA    Jan      47
      Feb     110
NY    Jan     221
True ['state', 'month']
110
```

- A list of columns builds a `MultiIndex` : each row label is a tuple
- Use it when several fields together identify a row
- Grouping and pivoting (Modules 6 and 8) produce MultiIndex results

demo09
MultiIndex

SpiralTrain developer courses

Core Indexing Checklist

- Build labels from data columns with `set_index` ; check `is_unique`
- `.loc` for labels, `.iloc` for positions
- Change labels with `rename` , `index.str` , `index.map` or a whole new index
- Arithmetic aligns labels ; unmatched labels give missing values
- `reindex` puts rows in a chosen order and adds missing labels
- A lookup on a duplicate label returns several rows
- A `MultiIndex` uses a tuple of labels per row
- Exercise 5 : the MultiIndex practice is optional

exercise5

Set and Sort a MultiIndex

```
stocks = pd.read_csv('stocks.csv')
print(stocks)
stocks = (stocks.set_index(['Symbol', 'Date'])
                .sort_index())
print(stocks)
```

- Closing prices of three stocks over three days
- The pair (`Symbol`, `Date`) identifies each row
- Sorting makes range selections reliable

```
         Date   Close    Volume Symbol
0  2026-10-05  112.82  18548140   CSCO
1  2026-10-05  332.89  34400850   AAPL
2  2026-10-05  525.18  26604760   MSFT
3  2026-10-06  333.63  30449010   AAPL
...
                       Close    Volume
Symbol Date
AAPL   2026-10-05  332.89  34400850
       2026-10-06  333.63  30449010
       2026-10-07  336.67  34147860
CSCO   2026-10-05  112.82  18548140
...
```

demo10
Set and sort MultiIndex

Select an Outer Level

```
print(stocks.loc['AAPL'])
print(stocks.loc[('CSCO', '2026-10-06'), 'Close'])
print(stocks.loc['CSCO':'MSFT', 'Close'])
```

- One outer label returns its rows with the remaining level as index
- A full tuple selects one row
- A sorted outer range includes both ends

```
                Close    Volume
Date
2026-10-05  332.89  34400850
2026-10-06  333.63  30449010
2026-10-07  336.67  34147860
117.94
Symbol  Date
CSCO    2026-10-05    112.82
        2026-10-06    117.94
        2026-10-07    117.39
MSFT    2026-10-05    525.18
...
```

demo11
Outer level

Lists Within a Level

```
print(stocks.loc[(['AAPL', 'MSFT'], '2026-10-07'), :])
print(stocks.loc[('CSCO', ['2026-10-07', '2026-10-05']), 'Close'])
```

```
                     Close    Volume
Symbol Date
AAPL   2026-10-07  336.67  34147860
MSFT   2026-10-07  529.76  16790230
Symbol  Date
CSCO    2026-10-07    117.39
        2026-10-05    112.82
Name: Close, dtype: float64
```

- A list inside the tuple selects several labels of one level
- Keep the explicit `, :` when selecting rows with a tuple and all columns

demo12
Lists within levels

Select an Inner Level

```
print(stocks.loc[(slice(None), '2026-10-06'), :])
print(stocks.xs('2026-10-06', level='Date'))
```

```
                     Close    Volume
Symbol Date
AAPL   2026-10-06  333.63  30449010
CSCO   2026-10-06  117.94  21037890
MSFT   2026-10-06  529.30  21107460
          Close    Volume
Symbol
AAPL     333.63  30449010
CSCO     117.94  21037890
MSFT     529.30  21107460
```

- `slice(None)` keeps every label of the outer level, like `:`
- `xs(label, level=name)` takes a cross-section and drops that level
- Alternative : keep the fields as columns and filter with a Boolean mask

demo13
Inner level

- NumPy
- Pandas Intro
- Selecting and Filtering
- Cleaning and Transforming
- Advanced Indexing
- **Rearranging Data**
- Combining Data
- Grouping Data
- Matplotlib
- Time Series
- Integration Case
- Scikit-Learn Essentials
- Data Formats and Tools

APPENDIX

- Reading Other Sources

- Long and Wide Data
- Reshape by Pivoting
- Pivot Multiple Value Columns
- Wide to Long with melt
- Create a MultiIndex
- Unstack an Index Level
- Stack with pandas 3
- Missing Combinations in a Round Trip
- Control Level Order and Row Order
- Duplicate Entries in pivot
- Aggregating with pivot_table
- Count Nonmissing Responses

SpiralTrain developer courses

Long and Wide Data

- **The same observations can be stored in two shapes :**
  - Long : one row per observation, every variable in its own column
  - Wide : one row per entity, one column per value of a variable
- **Long data suits filtering, grouping and plotting libraries such as Seaborn**
- **Wide data suits comparison tables and reports**

```
import pandas as pd
trials = pd.read_csv('trials_01.csv')
print(trials)
```

```
   id treatment gender  response
0   1         A      F         5
1   2         A      M         3
2   3         B      F         8
3   4         B      M         9
```

- `trials` **is long : four patients, one row each**
- `treatment` **and** `gender` **identify each response**
- **The next slides turn it into a wide table and back :**
  - `pivot` **and** `unstack` **make data wider**
  - `melt` **and** `stack` **make data longer**

SpiralTrain developer courses

Reshape by Pivoting

- `pivot` turns the values of one column into column labels :
  - `index` : the column whose values become row labels
  - `columns` : the column whose values become column labels
  - `values` : the column that fills the table

```
print(trials.pivot(index='treatment', columns='gender', values='response'))
```

```
gender     F  M
treatment
A          5  3
B          8  9
```

- Each treatment/gender pair must identify one response
- A long table keeps these variables as columns ; `pivot` creates a wide comparison

demo01
Reshaping by Pivoting

SpiralTrain developer courses

Pivot Multiple Value Columns

- Omitting `values` pivots every remaining column :
  - Both `id` and `response` become hierarchical columns
  - The outer column level holds the original column name
  - The inner column level holds the gender

```
print(trials.pivot(index='treatment', columns='gender'))
```

```
           id    response
gender      F  M        F  M
treatment
A           1  2        5  3
B           3  4        8  9
```

- `result['response']` selects the response table from the outer level

demo02
Pivoting Multiple Columns

Wide to Long with melt

```
wide = trials.pivot(index='treatment', columns='gender',
                    values='response').reset_index()
print(wide)

long = wide.melt(id_vars='treatment',
                 value_vars=['F', 'M'],
                 var_name='gender',
                 value_name='response')
print(long)
```

```
gender treatment  F  M
0              A  5  3
1              B  8  9
  treatment gender  response
0         A      F         5
1         B      F         8
2         A      M         3
3         B      M         9
```

- `melt` is the inverse of `pivot` : column labels become values of a new column
- `id_vars` stay as identifier columns ; `value_vars` are the columns to unpivot
- `var_name` and `value_name` name the two new columns
- Many downloads arrive wide, with one column per year or month ; `melt` makes them long

demo03
Melt

SpiralTrain developer courses

Create a MultiIndex

- `set_index` with a list of columns builds a row `MultiIndex` :
  - Each row label is a (treatment, gender) tuple
  - The remaining columns stay as data

```
indexed = trials.set_index(['treatment', 'gender'])
print(indexed)
```

```
                   id  response
treatment gender
A         F         1         5
          M         2         3
B         F         3         8
          M         4         9
```

- Use meaningful level names ; check unique combinations before reshaping

demo04
Creating Multi Level Index

SpiralTrain developer courses

Unstack an Index Level

- `unstack` moves a row-index level into the columns :
  - The selected level becomes the inner column level
  - The level can be named or given by position

```
wide = indexed.unstack(level='gender')
print(wide)
```

```
           id    response
gender      F  M        F  M
treatment
A           1  2        5  3
B           3  4        8  9
```

- The result equals `pivot` without `values` on the previous slides

demo05
Unstacking Multi Index

Stack with pandas 3

- `stack` is the inverse of `unstack` :
  - It moves a column level back into the row index

```
restored = wide.stack(level='gender')
print(restored)
```

```
                   id  response
treatment gender
A         F         1         5
          M         2         3
B         F         3         8
          M         4         9
```

- The pandas 3 implementation retains missing values :
  - Use plain `stack()` on this baseline
  - Do not set the legacy `dropna` , `sort` or `future_stack` options

demo06
Stacking DataFrames

Missing Combinations in a Round Trip

```
sparse = indexed.drop(index=('B', 'M'))
expanded = sparse.unstack('gender').stack('gender')
print(expanded)
print(len(sparse), len(expanded))
```

```
                   id  response
treatment gender
A         F       1.0       5.0
          M       2.0       3.0
B         F       3.0       8.0
          M       NaN       NaN
3 4
```

- `unstack` introduces the absent B/M combination ; `stack` retains its missing row
- The integers become floats to hold `NaN`
- An absent observation is not a zero ; preserve known keys to recover sparse data

Control Level Order and Row Order

```
swapped = indexed.swaplevel('treatment', 'gender').sort_index()
print(swapped)
```

```
                   id  response
gender treatment
F      A            1         5
       B            3         8
M      A            2         3
       B            4         9
```

- `swaplevel` changes the level order ; `sort_index` changes the row order
- Check names, keys, row counts and values separately
- A sorted index supports range slicing such as `swapped.loc['F']`

Duplicate Entries in pivot

```
more_trials = pd.read_csv('trials_03.csv')
print(more_trials)
```

```
   id treatment gender  response
0   1         A      F         5
1   2         A      M         3
2   3         A      M         8
3   4         A      F         9
4   5         B      F         1
5   6         B      M         8
6   7         B      F         4
7   8         B      F         6
```

- `trials_03.csv` has several patients per treatment/gender pair :
  - A/F appears twice, B/F three times

```
more_trials.pivot(index='treatment', columns='gender', values='response')
```

```
ValueError: Index contains duplicate entries, cannot reshape
```

- The error is intentional : choose a meaningful aggregation instead of silently discarding rows

demo08
Pivot requires unique pairs

SpiralTrain developer courses

Aggregating with pivot_table

- `pivot_table` combines duplicate pairs with an aggregation function :
  - `aggfunc='mean'` averages the responses of each pair
  - Other functions are `'sum'`, `'count'`, `'size'`, `'min'` and `'max'`

```
print(more_trials.pivot_table(index='treatment', columns='gender',
                              values='response', aggfunc='mean'))
```

```
gender              F    M
treatment
A           7.000000  5.5
B           3.666667  8.0
```

- A/F averages 5 and 9 to 7.0
- These means summarize observations ; aggregation cannot restore the original rows

demo09
Using Pivot Table

Count Nonmissing Responses

- `aggfunc='count'` shows how many responses each cell summarizes :

```
print(more_trials.pivot_table(index='treatment', columns='gender',
                              values='response', aggfunc='count'))
```

```
gender     F  M
treatment
A          2  2
B          3  1
```

- The B/M mean of 8.0 rests on a single patient
- `count` counts nonmissing responses ; use `size` to count all rows

demo10
Other Aggregations

exercise6
Rearranging data

- NumPy
- Pandas Intro
- Selecting and Filtering
- Cleaning and Transforming
- Advanced Indexing
- Rearranging Data
- **Combining Data**
- Grouping Data
- Matplotlib
- Time Series
- Integration Case
- Scikit-Learn Essentials
- Data Formats and Tools

APPENDIX

- Reading Other Sources

- Combining Data
- Orders and Customers Data
- Appending Monthly Files
- Source Labels in the Index
- Column Alignment
- Merging on a Key
- Join Types Compared
- Reporting Unmatched Keys
- Different Names and Suffixes
- Expected Cardinality
- Detecting Duplicate Lookup Keys
- Row Inflation in a Merge
- Removing Exact Duplicates
- Missing Values in Keys
- Data Types of Key Columns
- Join on an Existing Index
- Tidy Data
- Melt Wide into Long
- Pivoting Back to Wide
- Aggregating Duplicate Pairs
- Checklist for Combining Data
- Exercise 7

SpiralTrain developer courses

Combining Data

- **Real analyses combine several tables :**
  - Monthly exports that share one layout are appended with `pd.concat`
  - A lookup table adds attributes by matching keys with `merge`
  - Wide and long layouts of one table are converted with `melt` and `pivot`
- **Each combination can change the data without an error :**
  - Rows can be lost when keys do not match
  - Rows can multiply when a lookup repeats a key
  - Totals change with them, even when the code runs without error
- **This module checks every step :**
  - Retain the source of each appended row
  - Declare the expected cardinality of each join
  - Explain whether row counts and totals should change

SpiralTrain developer courses

Orders and Customers Data

- **Task : total ordered units per region**
- **The inputs and what one row means :**
  - Monthly files : one row per order, five orders, 150 units in total
  - Customer lookup : intended to hold one row per customer
  - Customer IDs are text : `001` is not the number `1`
- **The population is all five orders :**
  - Orders without a region stay in the audit
  - Dropping them would silently omit their units
- **Local synthetic data : `datasets/orders-*.csv`**
  - The data dictionary is `orders-README.md`

SpiralTrain developer courses

Appending Monthly Files

```
months = ['jan', 'feb', 'mar']
frames = [pd.read_csv(DATA / f'orders-{month}.csv',
                      dtype={'customer_id': str}) for month in months]
orders = pd.concat([frame.assign(source=month)
                    for month, frame in zip(months, frames)],
                   ignore_index=True)
print(orders[['source', 'order_id', 'customer_id', 'units']].to_string(index=False))
```

```
source order_id customer_id  units
   jan     o001         001     10
   jan     o002         002     20
   feb     o003         001     30
   feb     o004         003     40
   mar     o005         004     50
```

- `DATA` is the course's `datasets` folder ; `customer_id` is read as text (see later)
- `pd.concat` stacks the monthly frames row by row
- `ignore_index=True` creates new row labels 0 to 4
- `assign(source=month)` adds a column that keeps the file identity
- Read all files first, then call `concat` once

demo01
Concat months

Source Labels in the Index

```
indexed = pd.concat(frames, keys=months, names=['source', 'row'])
print(indexed.index.tolist())
print('Rows:', len(orders), 'Units:', orders['units'].sum())
```

```
[('jan', 0), ('jan', 1), ('feb', 0), ('feb', 1), ('mar', 0)]
Rows: 5 Units: 150
```

- `keys=` adds the source as an outer index level :
  - Each row label becomes a (source, row) tuple
  - The original row numbers restart in each file
- Choose either this index or an ordinary source column
- The row count and unit total match the appended frame : 5 rows, 150 units

demo01
Concat months

Column Alignment

```
combined = pd.concat([jan, feb], ignore_index=True)
print(combined.to_string(index=False))
shared = pd.concat([jan, feb], join='inner', ignore_index=True)
print('Shared columns:', shared.columns.tolist())
```

```
id  units channel
 a     10     NaN
 b     20   store
Shared columns: ['id', 'units']
```

- `concat` aligns columns by name, not by position
- February has a `channel` column that January lacks :
  - Default `join='outer'` retains all columns ; absent fields become `NaN`
  - `join='inner'` keeps only the shared columns and drops `channel`

demo02
Column alignment

SpiralTrain developer courses

Merging on a Key

- `merge` adds columns by matching key values :
  - `on='customer_id'` names the key column in both frames
  - `how=` decides which unmatched rows survive
- A separate, smaller example makes each join type easy to count :
  - `small_orders` : `001 → 10`, `002 → 20`, `004 → 30` units
  - `small_customers` : `001 → North`, `002 → South`, `003 → West`
  - The five monthly orders return on the cardinality slides
- The join types :
  - `inner` : keep matching keys only
  - `left` : keep every order
  - `outer` : keep keys from both inputs
- Predict the row counts and unit totals before running the next demo

Join Types Compared

```
for how in ['inner', 'left', 'outer']:
    result = small_orders.merge(small_customers, on='customer_id', how=how,
                                validate='one_to_one', indicator=True)
    print(how, 'rows:', len(result), 'units:', result['units'].sum())
```

```
inner rows: 2 units: 30
left rows: 3 units: 60
outer rows: 4 units: 60.0
```

- `inner` drops order `004` and its 30 units
- `left` keeps all three small orders and all 60 units
- `outer` adds a lookup-only row for customer `003` :
  - Its `units` value is `NaN`, so the total becomes a float
- `validate='one_to_one'` checks that keys are unique on both sides

demo03
Merge and audit

SpiralTrain developer courses

Reporting Unmatched Keys

```
audit = small_orders.merge(small_customers, on='customer_id', how='outer',
                           validate='one_to_one', indicator=True)
print(audit[['customer_id', '_merge']].to_string(index=False))
unmatched = audit.loc[audit['_merge'] != 'both']
print('Unmatched keys:', unmatched['customer_id'].tolist())
```

```
customer_id     _merge
        001       both
        002       both
        003 right_only
        004  left_only
Unmatched keys: ['003', '004']
```

- `indicator=True` adds a `_merge` column describing each match :
  - `both` : the key occurs in both inputs
  - `left_only` : an order without a lookup row
  - `right_only` : a lookup row without an order
- Keep this report beside the result ; every unmatched key needs a decision

demo03
Merge and audit

SpiralTrain developer courses

Different Names and Suffixes

```
left = small_orders.assign(status='ordered')
right = small_customers.rename(columns={'customer_id': 'id'}).assign(status='active')
named = left.merge(right, left_on='customer_id', right_on='id',
                   how='left', suffixes=('_order', '_customer'),
                   validate='one_to_one')
print(named[['customer_id', 'status_order', 'status_customer']].to_string(index=False))
```

```
customer_id status_order status_customer
        001      ordered          active
        002      ordered          active
        004      ordered             NaN
```

- `left_on` and `right_on` match keys that have different column names
- Both frames have a `status` column that is not a key :
  - `suffixes` renames the two versions instead of joining on them
  - Order status and customer status are different variables
- Name each key explicitly

demo03
Merge and audit

SpiralTrain developer courses

Expected Cardinality

- **Cardinality describes how many rows may match each key :**
  - `one_to_one` : keys unique on both sides
  - `many_to_one` : repeated orders, unique customer lookup
  - `one_to_many` : unique left keys, repeated right keys
  - `many_to_many` permits repetition ; it performs no uniqueness check
- **A left join with a unique lookup preserves the order count**
- **`validate=` checks this expectation before accepting the result :**
  - A violation raises `pandas.errors.MergeError`
  - It does not check missing matches, required columns or value ranges

SpiralTrain developer courses

Detecting Duplicate Lookup Keys

```
customers = pd.read_csv(DATA / 'orders-customers.csv', dtype={'customer_id': str})
try:
    orders.merge(customers, on='customer_id', how='left',
                 validate='many_to_one')
except pd.errors.MergeError:
    print('Rejected: customer lookup keys are not unique')
duplicates = customers.loc[customers.duplicated('customer_id', keep=False)]
print(duplicates.to_string(index=False))
```

```
Rejected: customer lookup keys are not unique
customer_id region
        001  North
        001  North
```

- `validate='many_to_one'` rejects a lookup that repeats a key
- `duplicated(keep=False)` lists every copy of a repeated key
- Inspect the conflicting records before deciding on a repair
- A merge that runs without validation proves little

demo04
Duplicate keys

SpiralTrain developer courses

Row Inflation in a Merge

```
bad = orders.merge(customers, on='customer_id', how='left')
print(bad[['order_id', 'customer_id', 'units']].to_string(index=False))
print('Before:', len(orders), orders['units'].sum())
print('After:', len(bad), bad['units'].sum())
```

```
order_id customer_id  units
    o001         001     10
    o001         001     10
    o002         002     20
    o003         001     30
    o003         001     30
    o004         003     40
    o005         004     50
Before: 5 150
After: 7 190
```

- **Without validation the merge multiplies rows :**
  - Two orders × two lookup rows for `001` give four matches
  - The extra rows add 10 + 30 units
- **The correct total is 150 ; 190 is an artifact of the join**

demo04
Duplicate keys

SpiralTrain developer courses

Removing Exact Duplicates

```
lookup = customers.drop_duplicates()  # documented exact repeated export
checked = orders.merge(lookup, on='customer_id', how='left',
                       validate='many_to_one', indicator=True)
assert len(checked) == len(orders)
assert checked['units'].sum() == orders['units'].sum()
missing = checked.loc[checked['_merge'] == 'left_only', 'customer_id']
print('Rows:', len(checked), 'Units:', checked['units'].sum())
print('Missing lookup:', missing.tolist())
```

```
Rows: 5 Units: 150
Missing lookup: ['004']
```

- `drop_duplicates()` removes exact repeated rows only :
  - Justified here because the export is documented to repeat a row
  - Conflicting regions for one customer would still fail validation
- The assertions confirm 5 rows and 150 units after the join
- The total is restored ; customer `004` still needs investigation

demo04
Duplicate keys

SpiralTrain developer courses

Missing Values in Keys

```
left = pd.DataFrame({'id': ['001', None], 'units': [10, 20]})
right = pd.DataFrame({'id': ['001', None], 'region': ['North', 'Unknown']})
matched = left.merge(right, on='id', validate='one_to_one', indicator=True)
print(matched.to_string(index=False))
```

```
id  units  region _merge
001     10   North   both
NaN     20 Unknown   both
```

- **pandas matches missing keys to missing keys :**
  - Ordinary SQL equality joins never match `NULL` keys
  - Two missing IDs do not show that the rows describe the same customer
- **Choose a policy before merging :**
  - Reject rows with missing IDs, or
  - Set them aside in an unmatched report and merge only non-missing keys

demo05
Key policies

Data Types of Key Columns

```
inferred = pd.read_csv(DATA / 'orders-jan.csv')
typed = pd.read_csv(DATA / 'orders-jan.csv', dtype={'customer_id': str})
print('Inferred IDs:', inferred['customer_id'].tolist())
print('Text IDs:', typed['customer_id'].tolist())
try:
    inferred.merge(typed, on='customer_id')
except ValueError:
    print('Rejected: integer and string merge keys')
```

```
Inferred IDs: [1, 2]
Text IDs: ['001', '002']
Rejected: integer and string merge keys
```

- Without a `dtype`, `read_csv` reads `001` as the integer `1`
- `merge` rejects keys of incompatible types with a `ValueError`
- Read both key columns as `str`:
  - Converting `1` to text later cannot recover `001`

demo05
Key policies

SpiralTrain developer courses

Join on an Existing Index

```
indexed_orders = small_orders.set_index('customer_id')
indexed_customers = small_customers.set_index('customer_id')
joined = indexed_orders.join(indexed_customers, how='left',
                             validate='one_to_one')
print(joined)
```

```
units region
customer_id
001              10  North
002              20  South
004              30    NaN
```

- `DataFrame.join` matches on the index by default :
  - Here both small frames use `customer_id` as their index
  - `join` defaults to `how='left'` ; `merge` defaults to `how='inner'`
- Prefer `merge` for explicit key columns and an `indicator` audit

demo06
Index join

SpiralTrain developer courses

Tidy Data

- **Tidy data has one variable per column and one observation per row**
- **Wide input : one row per customer**
  - Separate `Jan` and `Feb` columns hold the units
- **Long output : one customer/month measurement per row**
  - Two wide rows become four long rows
- **The reshaping methods :**
  - `melt` moves month column names into a variable column
  - `pivot` puts those month values back into columns
  - Rearranging Data covers `pivot`, `stack`, `unstack` and hierarchical indexes
- **Reshaping changes the layout ; check that the measurements stay intact**

SpiralTrain developer courses

Melt Wide into Long

```
long = wide.melt(id_vars='customer_id', value_vars=['Jan', 'Feb'],
                 var_name='month', value_name='units')
print(long.to_string(index=False))
print('Measurements:', len(long), 'Units:', long['units'].sum())
```

```
customer_id month  units
        001   Jan     10
        002   Jan     20
        001   Feb     30
        002   Feb     40
Measurements: 4 Units: 100
```

- `id_vars` stays in place as an identifier column
- `value_vars` names the columns to reshape
- `var_name` and `value_name` name the two new columns
- Four measurements and 100 units, before and after

demo07
Tidy reshape

SpiralTrain developer courses

Pivoting Back to Wide

```
restored = long.pivot(index='customer_id', columns='month', values='units')
print(restored)
```

```
month        Feb  Jan
customer_id
001           30   10
002           40   20
```

- `pivot` restores the wide layout
- Each customer/month pair must be unique, as in Rearranging Data
- The month columns come out sorted alphabetically : `Feb` before `Jan`

SpiralTrain developer courses

Aggregating Duplicate Pairs

```
repeated = pd.concat([long, long.iloc[[0]]], ignore_index=True)
try:
    repeated.pivot(index='customer_id', columns='month', values='units')
except ValueError:
    print('Pivot rejected duplicate customer/month pairs')
totals = repeated.pivot_table(index='customer_id', columns='month',
                              values='units', aggfunc='sum')
print(totals)
```

```
Pivot rejected duplicate customer/month pairs
month        Feb  Jan
customer_id
001           30   20
002           40   20
```

- A repeated measurement makes `pivot` fail
- `pivot_table` aggregates duplicates instead :
  - Its default is the mean ; state the rule explicitly with `aggfunc`
  - Customer `001` in January now sums to 20 : the repeated row is counted twice
- Aggregation is not data repair ; find out why the extra row exists

demo07
Tidy reshape

Checklist for Combining Data

- **Check every combination before using its result :**
  - Expected cardinality : are the right keys unique?
  - Row counts : which rows should be kept, added or removed?
  - Reconciled totals : can every change be explained?
  - Unmatched report : which keys still need a decision?
  - Key contract : compatible types, valid values, a missing-key policy
- **Keep these checks beside the transformation, before plotting a result**
- **For the orders example :**
  - `many_to_one`, 5 rows, 150 units, customer `004` without a region

SpiralTrain developer courses

Exercise 7

- **The same checks on real data : World Bank population files, 2024**
  - Append two downloads ; reject a membership list as a lookup
  - Separate economies from aggregates and explain a 23-million gap
  - Total the population and the 65+ share per income group
  - Optional : melt 66 year columns ; explain a nearly tenfold unchecked join
- **Guide :** `src/exercises/07-combining-data.md`
- **Notebook :** `exercises/07-combining-data/exercise.ipynb`

- NumPy
- Pandas Intro
- Selecting and Filtering
- Cleaning and Transforming
- Advanced Indexing
- Rearranging Data
- Combining Data
- **Grouping Data**
- Matplotlib
- Time Series
- Integration Case
- Scikit-Learn Essentials
- Data Formats and Tools

APPENDIX

- Reading Other Sources

- Split-Apply-Combine
- Boolean Filter and Count
- Aggregation and Reduction
- Groupby and Sum with Multiple Columns
- Groupby Mean with a Multi-Level Index
- Groupby and Sum by Series
- Categorical Data
- groupby with Multiple Aggregations
- Custom Aggregation
- A Small Frame with Missing Values
- Named Aggregation
- Group Sizes and Counts
- Missing Group Keys
- Unobserved Categories
- Fuel Economy Data
- Mean MPG by Year
- Rank the Groups
- groupby Object
- Iterating over Groups
- Selecting Groups in a Loop
- Groups in a Comprehension
- Boolean groupby
- The apply Method
- The filter Method
- Groupby and Transformation
- Standard Deviation Conventions
- Missing Values and Small Groups
- MPG z-score by Year
- Transform and Retain Row Metadata
- Plotting a Grouped Result

SpiralTrain developer courses

Split-Apply-Combine

- `groupby` performs three steps in one expression :
  - Split : the rows are divided into groups by the values of a key
  - Apply : a function runs on each group independently
  - Combine : the results are assembled into a new `Series` or `DataFrame`
- The apply step takes one of three forms :
  - Aggregation reduces each group to one value : `sum`, `mean`, `count`
  - Transformation returns one value per row : a z-score within the group
  - Filtration keeps or drops whole groups
- The examples use a small sales table :

```
sales = pd.DataFrame({
    'weekday': ['Sun', 'Sun', 'Mon', 'Mon'],
    'city': ['Austin', 'Dallas', 'Austin', 'Dallas'],
    'bread': [139, 237, 326, 456],
    'butter': [20, 45, 70, 98],
})
```

Boolean Filter and Count

- A Boolean filter selects one group by hand ; `groupby` handles every group at once

```
print(sales.loc[sales['weekday'] == 'Sun'].count())
```

```
weekday    2
city       2
bread      2
butter     2
dtype: int64
```

```
print(sales.groupby('weekday').count())
```

```
         city  bread  butter
weekday
Mon         2      2       2
Sun         2      2       2
```

- `count()` counts the non-missing values per column
- The group keys become the index of the result, sorted by default

demo01
Boolean Filter Count and Groupby

SpiralTrain developer courses

Aggregation and Reduction

- **An aggregation reduces each group to one value per column :**
  - `mean()`, `std()`, `sum()`
  - `first()`, `last()`
  - `min()`, `max()`
  - `count()` and `size()`
- **Select the columns before aggregating :**
  - `sales.groupby('weekday')['bread']` aggregates one column into a `Series`
  - `sales.groupby('weekday')[['bread', 'butter']]` aggregates several into a `DataFrame`
  - Text columns such as `city` cannot be summed or averaged
- **`agg` applies one or more functions in a single call :**
  - A list of functions gives a two-level column index
  - Named aggregation gives flat, descriptive column names

Groupby and Sum with Multiple Columns

SpiralTrain developer courses

```
print(sales.groupby('weekday')['bread'].sum())
```

```
weekday
Mon    782
Sun    376
Name: bread, dtype: int64
```

```
print(sales.groupby('weekday')[['bread', 'butter']].sum())
```

```
         bread  butter
weekday
Mon        782     168
Sun        376      65
```

```
   weekday    city  bread  butter
0      Sun  Austin    139      20
1      Sun  Dallas    237      45
2      Mon  Austin    326      70
3      Mon  Dallas    456      98
```

- One column label gives a `Series` per group key
- A list of labels gives a `DataFrame` with one column per label
- Monday bread : 326 + 456 = 782

demo02
Groupby and Sum

Groupby Mean with a Multi-Level Index

- **A list of keys groups by every combination of their values :**
  - The result has a `MultiIndex` with one level per key
  - Each combination occurs once here, so each mean is a single value

```
print(sales.groupby(['city', 'weekday'])
      [['bread', 'butter']].mean())
```

```
                bread  butter
city   weekday
Austin Mon      326.0    70.0
       Sun      139.0    20.0
Dallas Mon      456.0    98.0
       Sun      237.0    45.0
```

```
   weekday    city  bread  butter
0      Sun  Austin    139      20
1      Sun  Dallas    237      45
2      Mon  Austin    326      70
3      Mon  Dallas    456      98
```

- **`.loc['Austin']` selects the outer level ; `reset_index()` turns the levels into columns**

demo03
Groupby and Mean

Groupby and Sum by Series

- The key can be a `Series` that is not a column of the frame :
  - It aligns with the rows by index label
  - Here row 0 belongs to Dave, rows 1 and 3 to Alice, row 2 to Bob

```
customers = pd.Series(['Dave', 'Alice', 'Bob', 'Alice'])
print(sales.groupby(customers)['bread'].sum())
```

```
Alice    693
Bob      326
Dave     139
Name: bread, dtype: int64
```

	weekday	city	bread	butter
0	Sun	Austin	139	20
1	Sun	Dallas	237	45
2	Mon	Austin	326	70
3	Mon	Dallas	456	98

- Alice bought bread twice : 237 + 456 = 693

Categorical Data

```
print(sales['weekday'].unique())
sales['weekday'] = sales['weekday'].astype('category')
print(sales['weekday'])
```

```
<ArrowStringArray>
['Sun', 'Mon']
Length: 2, dtype: str
0    Sun
1    Sun
2    Mon
3    Mon
Name: weekday, dtype: category
Categories (2, str): ['Mon', 'Sun']
```

- A `category` column stores each distinct value once :
  - Each row holds a small integer code
  - Saves memory when few values repeat often
- Categories can carry an explicit order :
  - `pd.Categorical(values, categories=[...], ordered=True)`
- Grouping by a categorical uses the categories as group keys

demo05
Categorical Data

groupby with Multiple Aggregations

```
print(sales.groupby('city')[['bread', 'butter']].max())
```

```
        bread  butter
city
Austin    326      70
Dallas    456      98
```

```
print(sales.groupby('city')[['bread', 'butter']]
      .agg(['max', 'sum']))
```

```
       bread      butter
         max  sum    max  sum
city
Austin   326  465     70   90
Dallas   456  693     98  143
```

- `agg` with a list applies every function to every column
- The result has a two-level column index :
  - Column label on the outer level
  - Function name on the inner level
- `result['bread', 'sum']` selects one column
- Named aggregation avoids the two levels

demo06
groupby Multiple Aggregations

Custom Aggregation

- `agg` accepts a Python function that reduces a `Series` to one value :
  - A dictionary assigns a different function to each column

```
def data_range(series):
    return series.max() - series.min()
```

```
print(sales.groupby('weekday')
      [['bread', 'butter']].agg(data_range))
```

```
         bread  butter
weekday
Mon        130      28
Sun         98      25
```

```
print(sales.groupby(customers)
      [['bread', 'butter']]
      .agg({'bread': 'sum', 'butter': data_range}))
```

```
       bread  butter
Alice    693      53
Bob      326       0
Dave     139       0
```

- A custom function runs in Python once per group ; built-in names such as `'sum'` run faster

demo07
Custom Aggregation

SpiralTrain developer courses

A Small Frame with Missing Values

- **A second frame, `sales_na`, adds missing values :**
  - Monday's bread value in Dallas is missing
  - The last row has no weekday

```
import numpy as np
import pandas as pd

sales_na = pd.DataFrame({
    'weekday': ['Sun', 'Sun', 'Mon', 'Mon', np.nan],
    'city': ['Austin', 'Dallas', 'Austin', 'Dallas', 'Austin'],
    'bread': [139, 237, 326, np.nan, 210],
    'butter': [20, 45, 70, 98, 33],
})
print(sales_na)
```

```
  weekday    city  bread  butter
0     Sun  Austin  139.0      20
1     Sun  Dallas  237.0      45
2     Mon  Austin  326.0      70
3     Mon  Dallas    NaN      98
4     NaN  Austin  210.0      33
```

SpiralTrain developer courses

Named Aggregation

- `agg(name=(column, function))` names each output column :
  - Flat, descriptive names replace a two-level column index
  - Routine summaries need no custom function
- `size` counts rows ; `count` counts non-missing values

```
print(sales_na.groupby('city').agg(
    rows=('bread', 'size'),
    bread_count=('bread', 'count'),
    bread_total=('bread', 'sum'),
    butter_mean=('butter', 'mean'),
))
```

```
        rows  bread_count  bread_total  butter_mean
city
Austin     3            3        675.0         41.0
Dallas     2            1        237.0         71.5
```

- Dallas has two rows but one bread value ; its total covers one day only

demo08
Named Aggregation

Group Sizes and Counts

- The `sales_na` frame with missing values :
  - Monday's bread value in Dallas is missing ; the last row has no weekday
- `size()` counts the rows in each group
- `count()` counts the non-missing values in each column

```
print(sales_na.groupby('weekday').size())
print(sales_na.groupby('weekday').count())
```

```
weekday
Mon    2
Sun    2
dtype: int64
         city  bread  butter
weekday
Mon         2      1       2
Sun         2      2       2
```

```
   weekday    city  bread  butter
0      Sun  Austin  139.0      20
1      Sun  Dallas  237.0      45
2      Mon  Austin  326.0      70
3      Mon  Dallas    NaN      98
4      NaN  Austin  210.0      33
```

- Monday has two rows but one bread value
- `size().reset_index(name='n')` turns the counts into a table with a column `n`

SpiralTrain developer courses

Missing Group Keys

- `groupby()` drops rows whose key is missing (`dropna=True`)
- `dropna=False` keeps them as a separate `NaN` group
- Compare the group total with `len(sales_na)` to detect dropped rows :

```
print(sales_na.groupby('weekday').size().sum())
print(len(sales_na))
print(sales_na.groupby('weekday', dropna=False).size())
```

```
4
5
weekday
Mon    2
Sun    2
NaN    1
dtype: int64
```

demo10
Missing Group Keys

SpiralTrain developer courses

Unobserved Categories

- `observed=False` lists every category, including empty ones
- `observed=True` lists only categories present in the data
- pandas 3 defaults to `observed=True`; pandas 2 defaults to `False`
- An empty group sums to 0, which looks like a measured zero
- Pass `observed` explicitly:

```
cities = ['Austin', 'Dallas', 'Houston']
sales_na['city'] = pd.Categorical(sales_na['city'], categories=cities)
print(sales_na.groupby('city', observed=False)['bread'].sum())
```

```
city
Austin     675.0
Dallas     237.0
Houston      0.0
Name: bread, dtype: float64
```

SpiralTrain developer courses

Fuel Economy Data

```
auto = pd.read_csv('auto-mpg.csv')
print(auto[['mpg', 'cyl', 'weight', 'yr', 'origin', 'name']].head())
print(auto.shape)
```

```
    mpg  cyl  weight  yr origin                       name
0  18.0    8    3504  70     US  chevrolet chevelle malibu
1  15.0    8    3693  70     US          buick skylark 320
2  18.0    8    3436  70     US         plymouth satellite
3  16.0    8    3433  70     US              amc rebel sst
4  17.0    8    3449  70     US                ford torino
(392, 9)
```

- The rest of the module uses `auto-mpg.csv` : 392 cars from model years 1970 to 1982
- `mpg` is miles per gallon, `cyl` the number of cylinders, `weight` in pounds
- `yr` is the model year (70 means 1970) ; `origin` is US, Europe or Asia

demo12
Auto table

Mean MPG by Year

```
print(auto.groupby('yr')['mpg'].mean())
```

```
yr
70    17.689655
71    21.111111
72    18.714286
73    17.100000
74    22.769231
75    20.266667
76    21.573529
77    23.375000
78    24.061111
79    25.093103
80    33.803704
81    30.185714
82    32.000000
Name: mpg, dtype: float64
```

- An aggregation returns one row per group :
  - Thirteen model years give thirteen means
- Mean fuel economy rose from about 18 mpg in 1970 to about 32 mpg in 1982
- Later slides narrow the selection :
  - The mean per year for Chevrolet cars only

demo13
Mean-MPG-by-year

Rank the Groups

```
by_origin = (auto.groupby('origin')['mpg']
             .agg(['size', 'mean']).round(1))
print(by_origin.sort_values('mean', ascending=False))
print(auto.groupby('yr')['mpg'].mean()
      .nlargest(3).round(1))
print(auto['origin'].value_counts())
```

```
        size  mean
origin
Asia      79  30.5
Europe    68  27.6
US       245  20.0
yr
80    33.8
82    32.0
81    30.2
Name: mpg, dtype: float64
origin
US        245
Asia       79
Europe     68
Name: count, dtype: int64
```

- `value_counts()` is a shortcut for `groupby(column).size()`, sorted by count

- Group results are ordinary `Series` and `DataFrame` objects :
  - `sort_values` and `nlargest` rank the groups
- Report the group size beside a mean : Asia's mean rests on 79 cars, the US mean on 245

demo14
Rank the groups

SpiralTrain developer courses

groupby Object

```
splitting = auto.groupby('yr')
print(type(splitting))
print(type(splitting.groups))
print(splitting.groups.keys())
print(list(splitting.groups.values())[:2])
```

```
<class 'pandas.api.typing.DataFrameGroupBy'>
<class 'pandas.io.formats.printing.PrettyDict'>
dict_keys([70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82])
[RangeIndex(start=0, stop=29, step=1), RangeIndex(start=29, stop=56, step=1)]
```

- `groupby` returns a `DataFrameGroupBy` object ; no computation happens yet
- `groups` maps each key to the index labels of its rows :
  - The 1970 cars are rows 0 to 28, the 1971 cars rows 29 to 55
- The object can be aggregated, transformed, filtered or iterated

demo15
group-by Object

Iterating over Groups

```
splitting = auto.groupby('yr')
for group_name, group in splitting:
    print(group_name, group['mpg'].mean())
```

```
70 17.689655172413794
71 21.11111111111111
72 18.714285714285715
73 17.1
74 22.76923076923077
75 20.266666666666666
76 21.573529411764707
77 23.375
78 24.061111111111114
79 25.09310344827585
80 33.803703703703704
81 30.18571428571429
82 32.0
```

- Iterating over a `groupby` object yields pairs :
  - The group key, here the model year
  - A `DataFrame` with the rows of that group
- The loop gives the same means as `groupby('yr')['mpg'].mean()`
- A loop is slower ; use it for logic that no aggregation expresses

demo16
group-by Object Iteration

Selecting Groups in a Loop

```
for group_name, group in splitting:
    chevy = group['name'].str.contains('chevrolet')
    avg = group.loc[chevy, 'mpg'].mean()
    print(group_name, avg)
```

```
70 15.666666666666666
71 20.25
72 15.333333333333334
73 14.833333333333334
74 18.666666666666668
75 17.666666666666668
76 23.25
77 20.25
78 23.233333333333334
79 21.666666666666668
80 30.05
81 23.5
82 29.0
```

- Inside the loop each group is an ordinary `DataFrame` :
  - A Boolean mask filters the Chevrolet rows
  - `.loc[mask, 'mpg']` selects their fuel economy
- Chevrolets score below the yearly average in most years

demo17
group-by Object Iteration and Filtering

Groups in a Comprehension

```
chevy_means = {
    year: group.loc[group['name'].str.contains('chevrolet'), 'mpg'].mean()
    for year, group in splitting
}
print(pd.Series(chevy_means))
```

```
70    15.666667
71    20.250000
72    15.333333
73    14.833333
74    18.666667
75    17.666667
76    23.250000
77    20.250000
78    23.233333
79    21.666667
80    30.050000
81    23.500000
82    29.000000
dtype: float64
```

- A dictionary comprehension collects one result per group :
  - The keys become the index of the `Series`
- The result can be compared or plotted like any `Series`

demo18
group-by Object Comprehension

Boolean groupby

```
chevy = auto['name'].str.contains('chevrolet')
print(auto.groupby(['yr', chevy])['mpg']
      .mean().loc[70:76])
```

```
yr  name
70  False    17.923077
    True     15.666667
71  False    21.260870
    True     20.250000
72  False    19.120000
    True     15.333333
73  False    17.500000
    True     14.833333
74  False    23.304348
    True     18.666667
75  False    20.555556
    True     17.666667
76  False    21.350000
    True     23.250000
Name: mpg, dtype: float64
```

- A Boolean `Series` can serve as a group key :
  - `True` groups the Chevrolets, `False` all other cars
  - The level takes the name of the `Series`, here `name`
- One `groupby` replaces the loop and the comprehension
- Both groups appear side by side for each year

demo19
Boolean groupby

The apply Method

```
def chevy_mean(group):
    chevy = group['name'].str.contains('chevrolet')
    return group.loc[chevy, 'mpg'].mean()

print(auto.groupby('yr')[['name', 'mpg']]
      .apply(chevy_mean).head(4))

def heaviest(group):
    return group.nlargest(1, 'weight')

print(auto.groupby('origin')[['name', 'weight']]
      .apply(heaviest))
```

```
yr
70    15.666667
71    20.250000
72    15.333333
73    14.833333
dtype: float64
                                name  weight
origin
Asia   208       toyota mark ii    2930
Europe 209   mercedes-benz 280s    3820
US     43   pontiac safari (sw)    5140
```

- `apply(f)` calls `f` once per group with that group's rows as a `DataFrame`
- A scalar result gives one value per group ; a `DataFrame` result keeps its rows
- Select the needed columns before `apply` ; prefer `agg` or `transform` when they can express it

demo20
groupby apply

The filter Method

```
counts = auto['cyl'].value_counts().sort_index()
print(counts)

common = auto.groupby('cyl').filter(
    lambda group: len(group) >= 10)
print(common['cyl'].value_counts().sort_index())
print(len(auto), len(common))
```

```
cyl
3      4
4    199
5      3
6     83
8    103
Name: count, dtype: int64
cyl
4    199
6     83
8    103
Name: count, dtype: int64
392 385
```

- `filter(f)` keeps or drops whole groups : `f` returns `True` or `False` per group
- The result has the original rows of the kept groups, not one row per group
- Here the 3- and 5-cylinder groups are too small to compare ; 7 cars are dropped

demo21
groupby filter

Groupby and Transformation

- **z-score definition :**
  - Number of standard deviations a data point is from the mean
  - `series.std()` is the sample standard deviation (`ddof=1`)

```
auto = pd.read_csv('auto-mpg.csv')
print(auto[['mpg', 'cyl', 'yr', 'name']].head())

def zscore(series):
    return (series - series.mean()) / series.std()
print(zscore(auto['mpg']).head())
```

```
    mpg  cyl  yr                       name
0  18.0    8  70  chevrolet chevelle malibu
1  15.0    8  70          buick skylark 320
2  18.0    8  70         plymouth satellite
3  16.0    8  70              amc rebel sst
4  17.0    8  70                ford torino
```

```
0   -0.697747
1   -1.082115
2   -0.697747
3   -0.953992
4   -0.825870
Name: mpg, dtype: float64
```

- **The 18 mpg Chevrolet lies 0.7 standard deviations below the mean of all cars**

demo22
Z-Score

SpiralTrain developer courses

Standard Deviation Conventions

- `Series.std()` divides by n - 1 by default (`ddof=1`, sample)
- `scipy.stats.zscore()` divides by n by default (`ddof=0`, population)
- The results differ ; state the convention and pass `ddof` to match :

```
print(auto['mpg'].std(), auto['mpg'].std(ddof=0))
from scipy import stats
print(stats.zscore(auto['mpg'])[:2])
print(stats.zscore(auto['mpg'], ddof=1)[:2])
```

```
7.8050074865717995 7.795045762682584
[-0.69863841 -1.08349824]
[-0.69774672 -1.08211534]
```

SpiralTrain developer courses

Missing Values and Small Groups

- `mean()` and `std()` skip missing values by default
- With `NaN` in the input, `stats.zscore()` propagates `NaN` by default :
  - `nan_policy='omit'` computes using the non-missing values
- One value has no sample standard deviation ; its z-score is `NaN`
- Identical values have zero standard deviation and `NaN` z-scores
- Check group sizes before standardizing within groups :

```
by_yr_cyl = auto.groupby(['yr', 'cyl'])['mpg']
print((by_yr_cyl.size() == 1).sum())              # 8
print(by_yr_cyl.transform(zscore).isna().sum())   # 8
```

- A z-score describes position within a group ; it does not indicate an error or a cause

demo23
Z-Score Conventions

MPG z-score by Year

- `transform` applies a function to each group and returns one value per row :
  - The result has the same index as the original frame
  - Each car is compared with the cars of its own model year

```
print(auto.groupby('yr')['mpg']
      .transform(zscore).head())
```

```
0    0.058125
1   -0.503753
2    0.058125
3   -0.316460
4   -0.129168
Name: mpg, dtype: float64
```

- The 18 mpg Chevrolet of 1970 :
  - **Below the mean of all years : -0.70**
  - **Slightly above the 1970 mean : 0.06**
- Cars got more efficient over the years ; a per-year score removes that trend

demo24
Z-Score per year

Transform and Retain Row Metadata

```
groups = auto.groupby('yr')['mpg']
mean = groups.transform('mean')
std = groups.transform('std', ddof=1)
result = auto[['yr', 'name']].copy()
result['mpg_z'] = (auto['mpg'] - mean) / std.where(std > 0)
print(result[['mpg_z', 'yr', 'name']].head())
```

```
      mpg_z  yr                       name
0  0.058125  70  chevrolet chevelle malibu
1 -0.503753  70          buick skylark 320
2  0.058125  70         plymouth satellite
3 -0.316460  70              amc rebel sst
4 -0.129168  70                ford torino
```

- `transform` retains the original row index, so the result lines up with `name`
- `std.where(std > 0)` keeps the values where the condition holds and puts `NaN` elsewhere :
  - `s.where(condition, other)` puts `other` there instead
- This uses the sample standard deviation :
  - Zero variance or a one-row group gives a missing score instead of a useful z-score

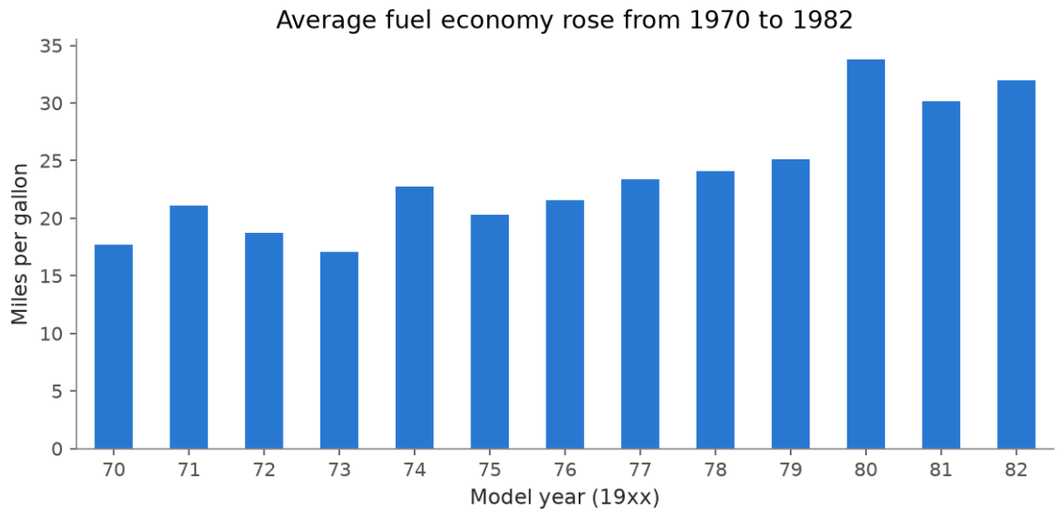
demo25
apply-transformation-and-aggregation

Plotting a Grouped Result

```
import matplotlib.pyplot as plt

mean_mpg = auto.groupby('yr')['mpg'].mean()

fig, ax = plt.subplots(figsize=(8, 4))
mean_mpg.plot.bar(ax=ax, rot=0)
ax.set(title='Average fuel economy rose '
             'from 1970 to 1982',
       xlabel='Model year (19xx)',
       ylabel='Miles per gallon')
fig.tight_layout()
```



- A grouped result plots directly with `.plot` ; the group keys label the bars
- `rot=0` keeps the year labels horizontal
- Module 9 explains `fig` , `ax` and the plotting options

demo26
Plot a grouped result

SpiralTrain developer courses

Summary : Grouping Data

- `groupby` splits rows by key, applies a function and combines the results
- Aggregate with `sum`, `mean`, `size`, `count`, `agg` lists, dictionaries and named aggregation
- `size` counts rows, `count` non-missing values ; missing keys are dropped unless `dropna=False`
- Categorical keys : pass `observed` explicitly
- Rank group results with `sort_values` and `nlargest`, and report group sizes
- `transform` returns one value per row, `filter` keeps whole groups, `apply` runs any function per group
- A group result plots directly with `.plot`

exercise8

- NumPy
- Pandas Intro
- Selecting and Filtering
- Cleaning and Transforming
- Advanced Indexing
- Rearranging Data
- Combining Data
- Grouping Data
- **Matplotlib**
- Time Series
- Integration Case
- Scikit-Learn Essentials
- Data Formats and Tools

APPENDIX

- Reading Other Sources

- What is Matplotlib?
- Figure, Axes and Axis
- Two Interfaces
- A Line Plot
- Colors, Markers and Line Styles
- Several Lines and a Legend
- Subplots
- Limits, Ticks and Scales
- Text and Annotations
- Scatter Plots
- Histograms
- Box Plots
- Bar Charts
- Pie Charts
- Dates on the Axis
- Contour Plots
- 3D Plots
- Styles
- Saving Figures
- Plotting with pandas
- pandas Plot Kinds
- Several Columns at Once
- One Panel per Column
- Seaborn
- Axes-Level and Figure-Level Functions
- Distributions
- Estimates and Group Sizes
- Fitted Lines
- Facets and Themes
- Chart Checklist

SpiralTrain developer courses

What is Matplotlib?

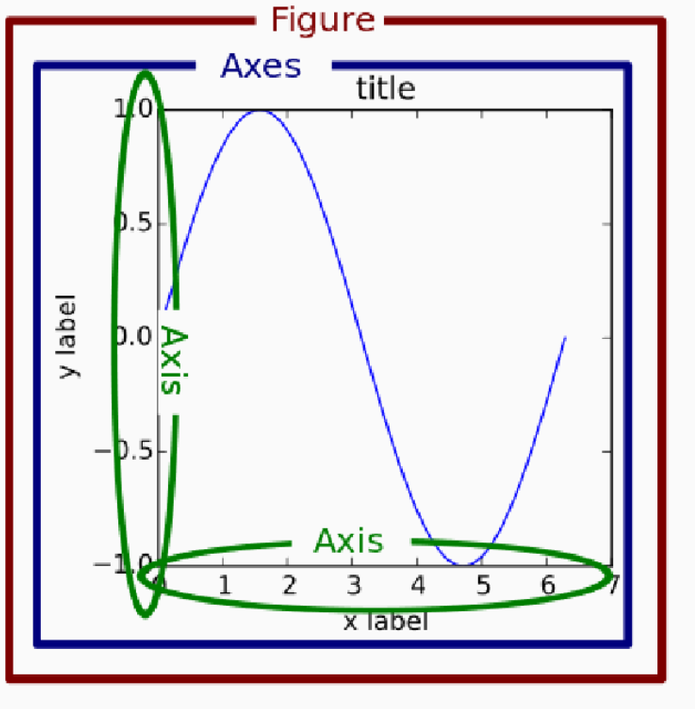
- **Python plotting library at https://matplotlib.org/ :**
  - Started by John D. Hunter in 2003 as a MATLAB-like plotting tool
  - Produces publication-quality figures as PNG, SVG and PDF
  - Built on NumPy ; accepts arrays, lists and pandas columns
- **Matplotlib is the base of the Python plotting stack :**
  - pandas `.plot()` draws with Matplotlib
  - Seaborn adds statistical charts on top of Matplotlib
- **Works in every environment :**
  - Jupyter notebooks show figures below the cell
  - Scripts open a window with `plt.show()`
  - GUI toolkits and web apps can embed figures

```
import matplotlib.pyplot as plt
```

Figure, Axes and Axis

- **`Figure` is the whole image :**
  - Holds one or more `Axes`, plus figure titles and legends
  - Saved to a file with `fig.savefig()`
- **`Axes` is one plot area :**
  - Holds the lines, bars and points, the title and the axis labels
  - Most plotting methods belong to an `Axes` : `ax.plot()`, `ax.bar()`
- **`Axis` is one number line of an `Axes` :**
  - An `Axes` has an x-axis and a y-axis
  - Each sets its limits, ticks and tick labels

```
fig, ax = plt.subplots()          # one Axes
fig, axes = plt.subplots(2, 2)    # 2x2 grid of Axes
```

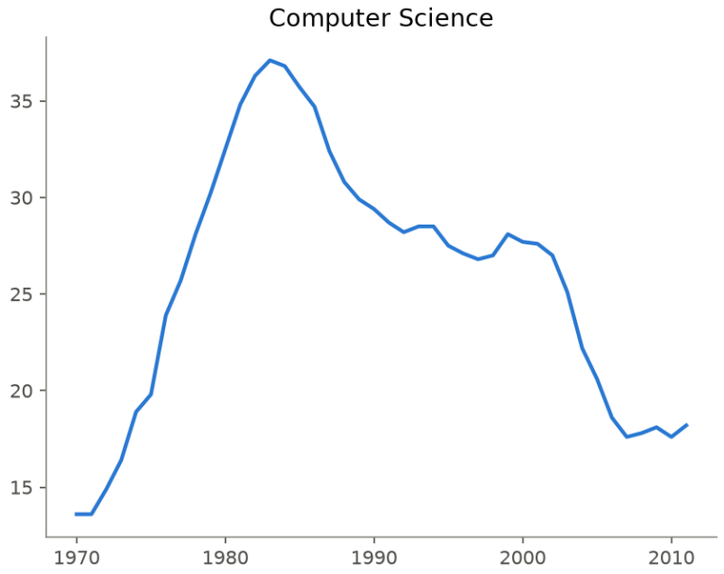


Two Interfaces

```
degrees = pd.read_csv('women-degrees.csv')

# pyplot : functions act on the "current" Axes
plt.plot(degrees['year'], degrees['computer_science'])
plt.title('Computer Science')
plt.show()

# object-oriented : methods on explicit objects
fig, ax = plt.subplots()
ax.plot(degrees['year'], degrees['computer_science'])
ax.set_title('Computer Science')
plt.show()
```



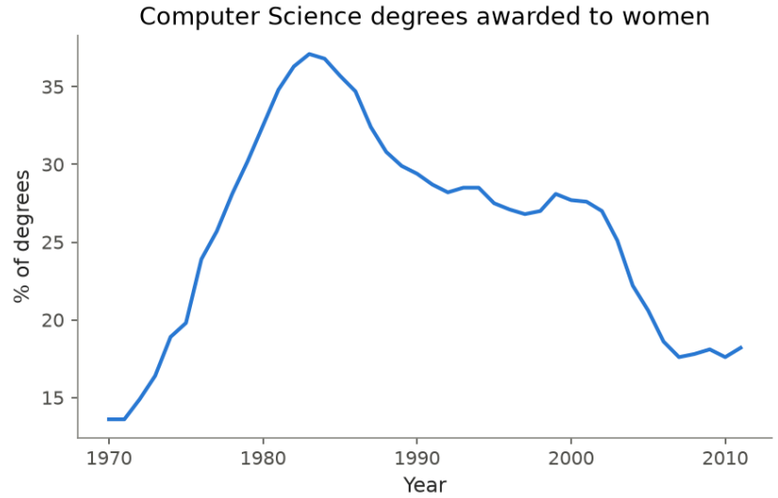
- **Both draw the same chart ; the object-oriented style says which `Axes` gets each call**
- **This course uses `fig, ax = plt.subplots()` and `ax.` methods :**
  - Matplotlib's documentation recommends it for anything beyond a quick plot
  - It scales to several plots per figure ; `plt` stays for `subplots`, `show` and styles

demo01
Two interfaces

A Line Plot

```
degrees = pd.read_csv('women-degrees.csv')

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(degrees['year'], degrees['computer_science'])
ax.set(title='Computer Science degrees awarded to women',
       xlabel='Year', ylabel='% of degrees')
fig.tight_layout()
plt.show()
```



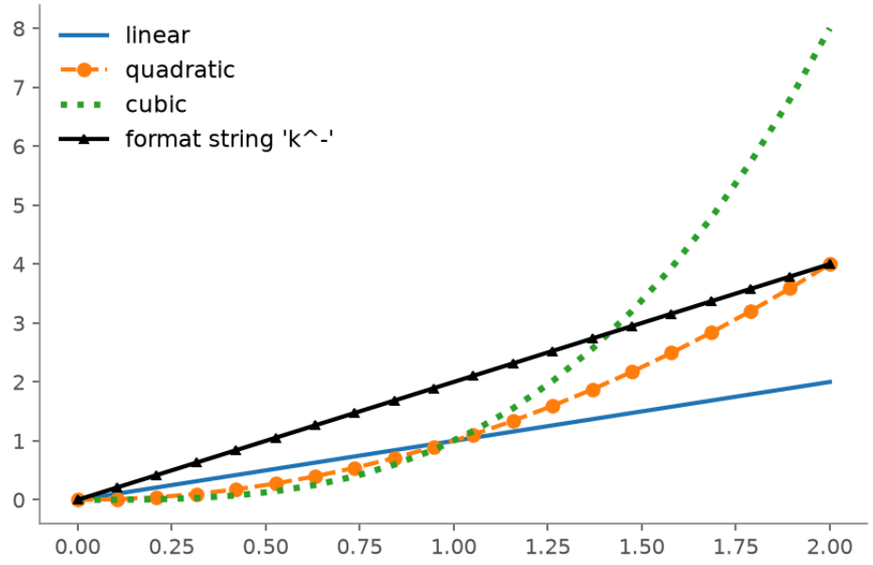
- `women-degrees.csv` : percentage of US bachelor's degrees awarded to women, per field, 1970–2011
- `ax.plot(x, y)` draws a line through the points ; with only `y` , x is 0, 1, 2 ...
- `ax.set(...)` sets several properties at once : `title` , `xlabel` , `ylabel` , `xlim` , `ylim`
- `figsize=(width, height)` in inches ; `fig.tight_layout()` keeps labels inside the figure
- In Jupyter the figure appears below the cell ; a script needs `plt.show()`

demo02
Line plot

Colors, Markers and Line Styles

SpiralTrain
developer courses

```
x = np.linspace(0, 2, 20)
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(x, x, color='tab:blue', linestyle='-',
        label='linear')
ax.plot(x, x**2, color='tab:orange', linestyle='--',
        marker='o', label='quadratic')
ax.plot(x, x**3, color='tab:green', linestyle=':',
        linewidth=3, label='cubic')
ax.plot(x, 2 * x, 'k^-', markersize=4,
        label="format string 'k^-'")
ax.legend()
```

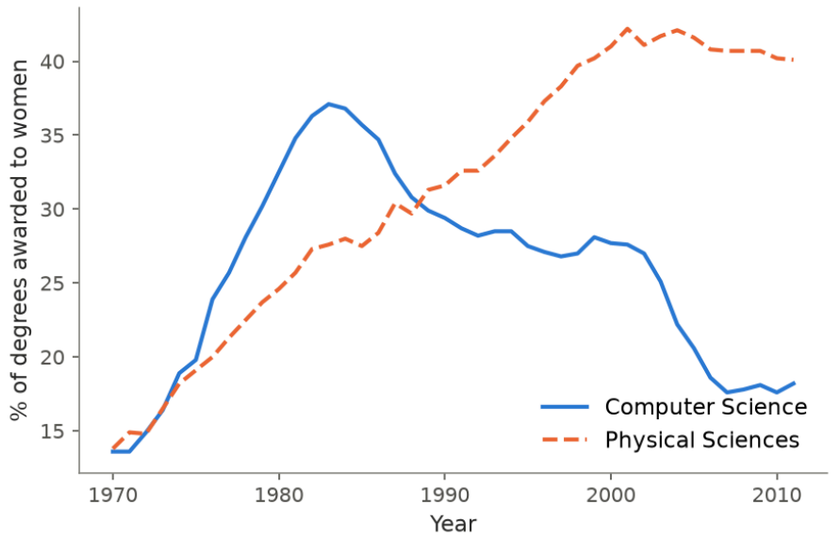


- `color` : `'tab:blue'`, `'black'`, `'#2a78d6'` ; `linestyle` : `'-'`, `'--'`, `':'`, `'-.'`
- `marker` : `'o'`, `'s'`, `'^'`, `'x'`, `'.'` ; `linewidth`, `markersize` and `alpha` take numbers
- A format string such as `'k^-'` combines color, marker and line style in one short code

demo03
Line styles

Several Lines and a Legend

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(degrees['year'], degrees['computer_science'],
        label='Computer Science')
ax.plot(degrees['year'], degrees['physical_sciences'],
        label='Physical Sciences', linestyle='--')
ax.set(xlabel='Year',
       ylabel='% of degrees awarded to women')
ax.legend(loc='lower right')
```

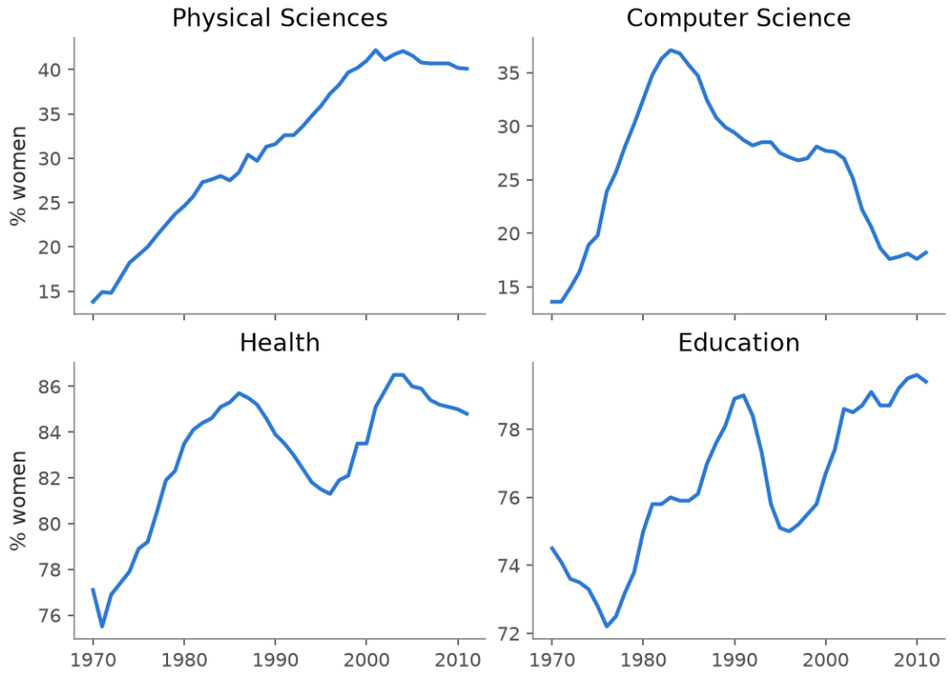


- Each `ax.plot` call adds a line to the same `Axes`
- `label=` names a line ; `ax.legend()` draws the legend from the labels
- `loc=` places the legend : `'best'`, `'upper left'`, `'lower center'` and so on
- Different line styles keep the lines apart in grayscale and for color-blind readers

demo04
Legend

Subplots

```
fields = ['physical_sciences', 'computer_science',
          'health', 'education']
fig, axes = plt.subplots(2, 2, figsize=(7, 5),
                         sharex=True,
                         layout='constrained')
for ax, field in zip(axes.flat, fields):
    ax.plot(degrees['year'], degrees[field])
    ax.set_title(field.replace('_', ' ').title())
axes[0, 0].set_ylabel('% women')
axes[1, 0].set_ylabel('% women')
```



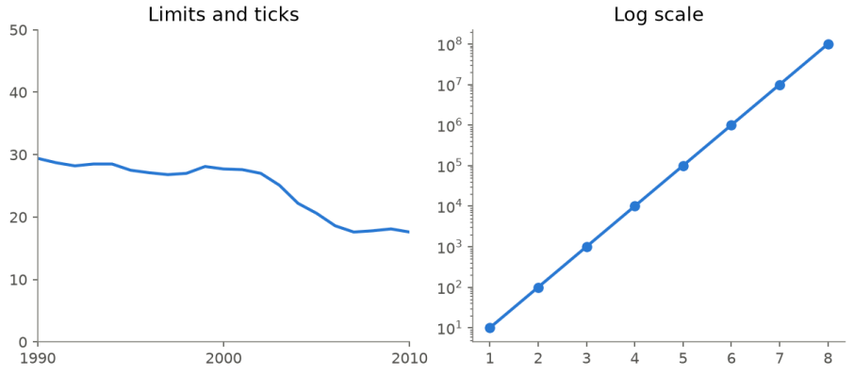
- `plt.subplots(rows, columns)` returns the figure and an array of `Axes`
- `axes[0, 1]` is the first row, second column ; `axes.flat` visits them in order
- `sharex=True` gives every row the same x-axis
- `layout='constrained'` spaces the panels ; `fig.tight_layout()` is the older alternative

demo05
Subplots

Limits, Ticks and Scales

```
fig, (left, right) = plt.subplots(
    1, 2, figsize=(8, 3.5), layout='constrained')
left.plot(degrees['year'], degrees['computer_science'])
left.set_xlim(1990, 2010)
left.set_ylim(0, 50)
left.set_xticks([1990, 2000, 2010])

x = np.arange(1, 9)
right.plot(x, 10.0 ** x, marker='o')
right.set_yscale('log')
```



- `set_xlim` and `set_ylim` set the range ; `ax.axis((xmin, xmax, ymin, ymax))` sets both at once
- `set_xticks` chooses where tick marks go
- `set_yscale('log')` shows growth by factors : each step up is ten times more

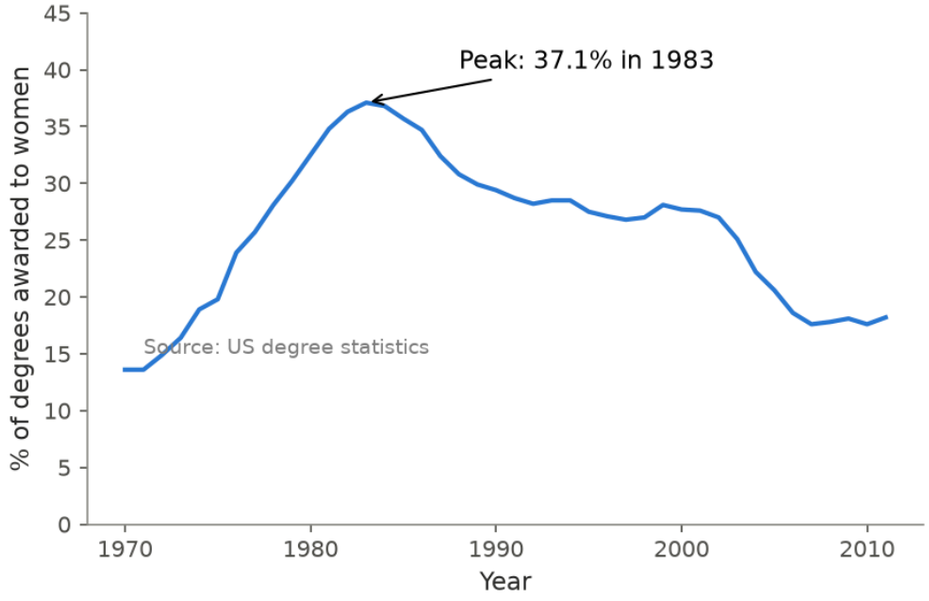
demo06
Limits and scales

Text and Annotations

SpiralTrain
developer courses

```
cs = degrees['computer_science']
peak_year = degrees.loc[cs.idxmax(), 'year']

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(degrees['year'], cs)
ax.annotate(f'Peak: {cs.max()}% in {peak_year}',
            xy=(peak_year, cs.max()),
            xytext=(peak_year + 5, cs.max() + 3),
            arrowprops={'arrowstyle': '->',
                        'color': 'black'})
ax.text(1971, 15, 'Source: US degree statistics',
        fontsize=9, color='gray')
```



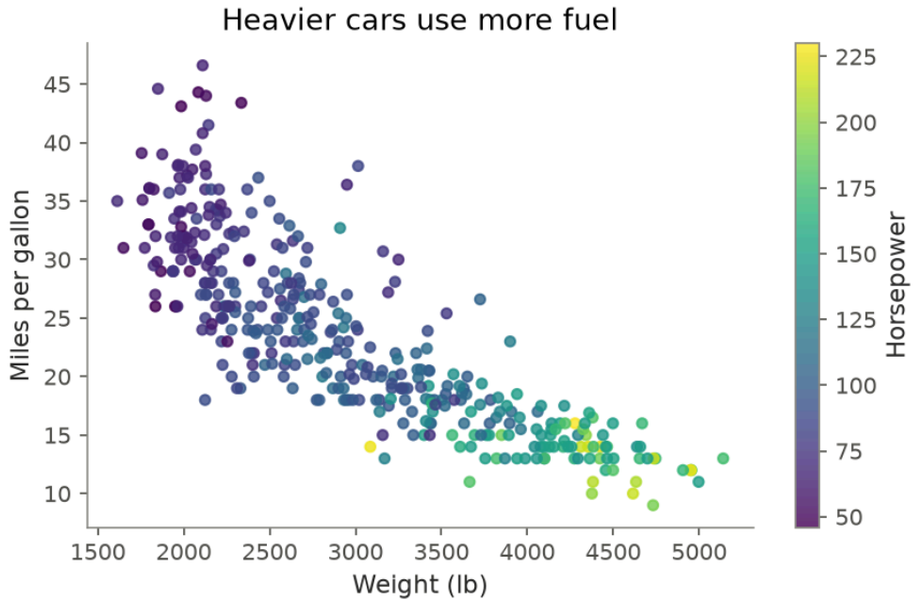
- `ax.annotate(text, xy=..., xytext=...)` points an arrow from the text to the point `xy`
- `ax.text(x, y, text)` places text at data coordinates
- Text options : `fontsize`, `color`, `fontweight`, `ha` (alignment), `rotation`, `bbox`
- Text between `$` signs is rendered as maths : `'$\\sigma^2$'`

demo07
Annotate

Scatter Plots

```
auto = pd.read_csv('auto-mpg.csv')

fig, ax = plt.subplots(figsize=(6, 4))
points = ax.scatter(auto['weight'], auto['mpg'],
                    c=auto['hp'], cmap='viridis',
                    s=20, alpha=0.8)
fig.colorbar(points, ax=ax, label='Horsepower')
ax.set(title='Heavier cars use more fuel',
       xlabel='Weight (lb)',
       ylabel='Miles per gallon')
```

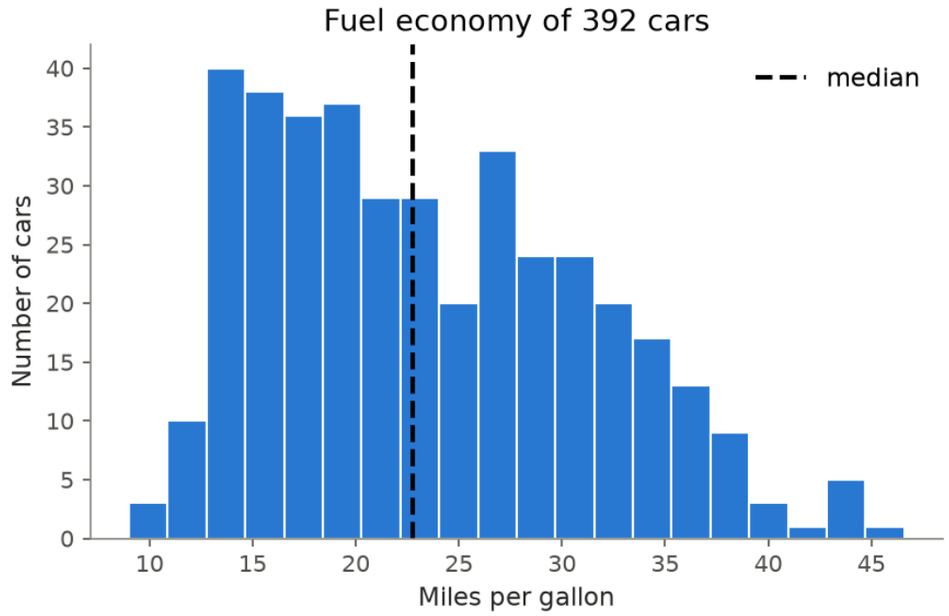


- `ax.scatter(x, y)` draws one marker per row : the relation between two variables
- `c=` colors each point by a third variable ; `cmap` chooses the color scale
- `s=` sets the marker size, `alpha=` the transparency for overlapping points
- `fig.colorbar` explains the colors

demo08
Scatter

Histograms

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(auto['mpg'], bins=20, edgecolor='white')
ax.axvline(auto['mpg'].median(), color='black',
           linestyle='--', label='median')
ax.set(title='Fuel economy of 392 cars',
       xlabel='Miles per gallon',
       ylabel='Number of cars')
ax.legend()
```



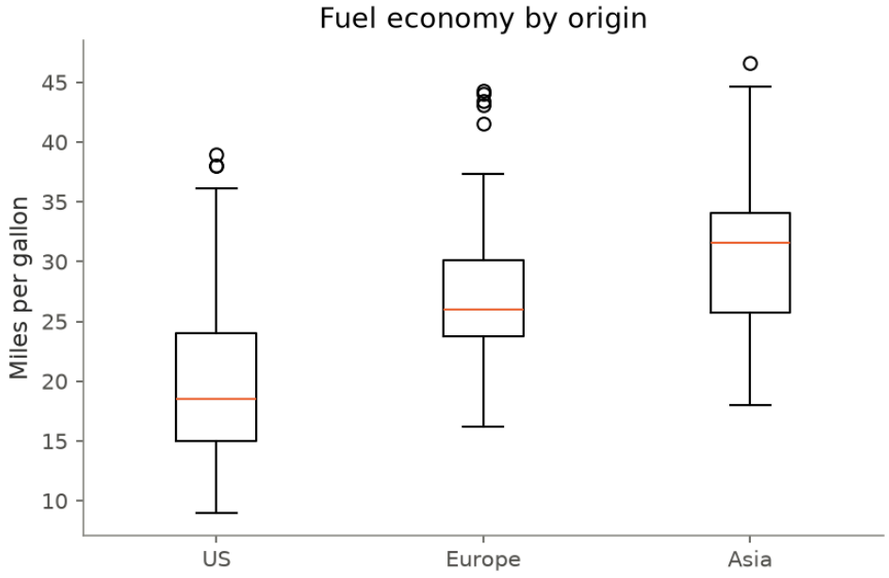
- `ax.hist(values, bins=n)` counts the values in `n` equal intervals and draws one bar per interval
- The number of bins changes the shape you see ; try a few values
- `ax.axvline(x)` and `ax.axhline(y)` draw reference lines across the plot

demo09
Histogram

Box Plots

```
origins = ['US', 'Europe', 'Asia']
groups = [auto.loc[auto['origin'] == origin, 'mpg']
          for origin in origins]

fig, ax = plt.subplots(figsize=(6, 4))
ax.boxplot(groups, tick_labels=origins)
ax.set(title='Fuel economy by origin',
       ylabel='Miles per gallon')
```



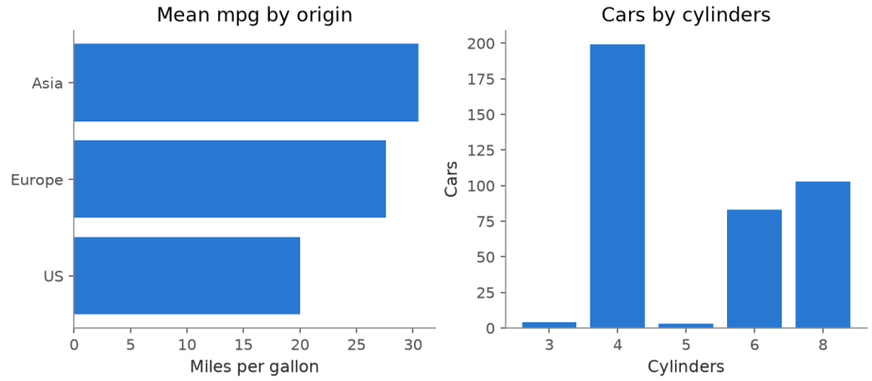
- **A box plot summarizes each group in one shape :**
  - The box spans the middle half of the values, from the 25% to the 75% quartile
  - The line inside is the median ; whiskers reach to 1.5 box lengths, points beyond are outliers
- **`ax.boxplot` takes a list of groups ; `tick_labels` names them**

demo10
Box plot

Bar Charts

```
mean_mpg = (auto.groupby('origin')['mpg'].mean()
             .sort_values())
counts = auto['cyl'].value_counts().sort_index()

fig, (left, right) = plt.subplots(
    1, 2, figsize=(8, 3.5), layout='constrained')
left.barh(mean_mpg.index, mean_mpg.to_numpy())
left.set(title='Mean mpg by origin',
         xlabel='Miles per gallon')
right.bar(counts.index.astype(str), counts.to_numpy())
right.set(title='Cars by cylinders',
          xlabel='Cylinders', ylabel='Cars')
```



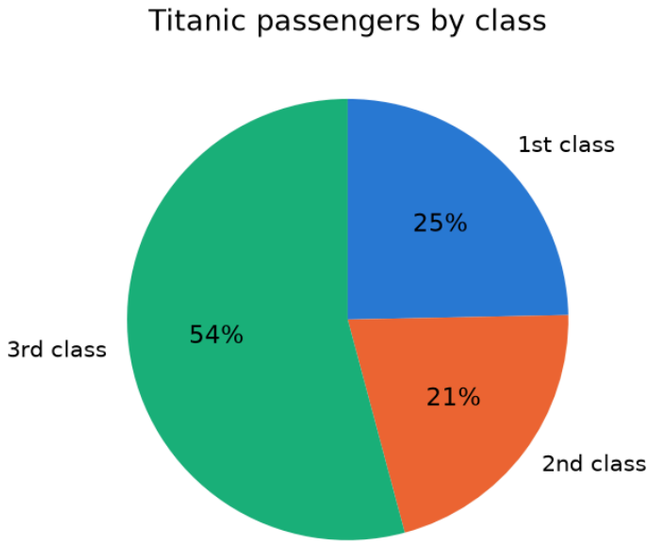
- `ax.bar(labels, heights)` draws vertical bars, `ax.barh` horizontal bars
- Sort the bars by value unless the categories have a natural order ; bars start at zero

demo11
Bar

Pie Charts

```
titanic = pd.read_csv('titanic.csv')
counts = titanic['pclass'].value_counts().sort_index()

fig, ax = plt.subplots(figsize=(4.5, 4.5))
ax.pie(counts.to_numpy(),
       labels=['1st class', '2nd class', '3rd class'],
       autopct='%1.0f%%', startangle=90,
       counterclock=False)
ax.set_title('Titanic passengers by class')
```



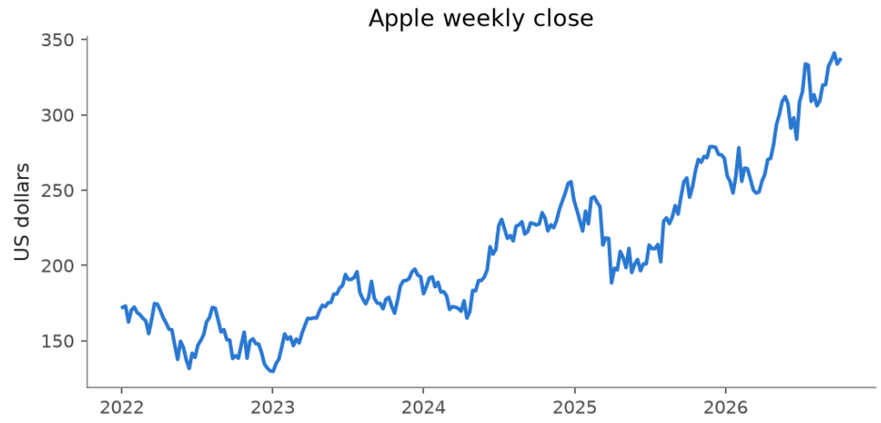
- `autopct` writes the percentage on each slice
- `startangle=90` and `counterclock=False` start at the top and go clockwise
- A pie only works for a few parts of one whole ; compare values with bars

demo12
Pie

Dates on the Axis

```
import matplotlib.dates as mdates

aapl = pd.read_csv('AAPL.csv', parse_dates=['Date'])
fig, ax = plt.subplots(figsize=(7, 3.5))
ax.plot(aapl['Date'], aapl['Close'])
locator = mdates.AutoDateLocator()
ax.xaxis.set_major_locator(locator)
ax.xaxis.set_major_formatter(
    mdates.ConciseDateFormatter(locator))
ax.set(title='Apple weekly close',
       ylabel='US dollars')
```



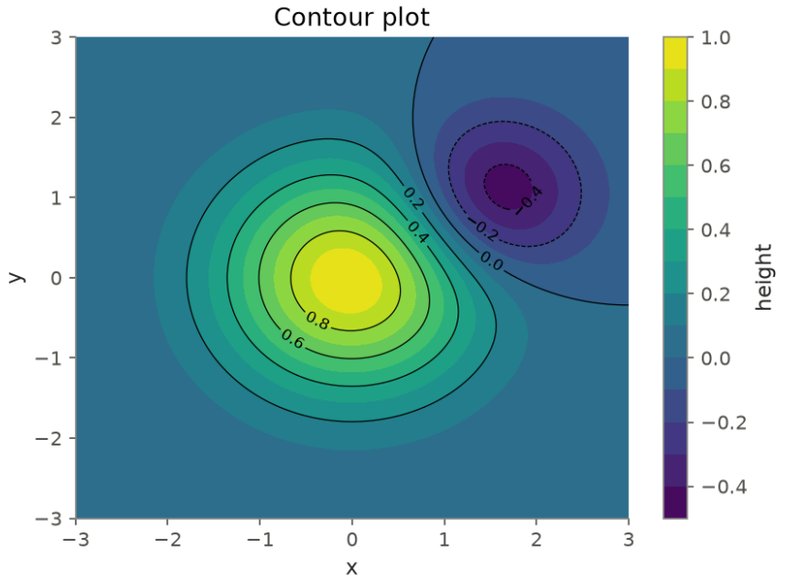
- `ax.plot` accepts parsed dates ; parse them with `parse_dates` or `pd.to_datetime`
- `ConciseDateFormatter` writes short, non-repeating date labels

demo13
Dates

Contour Plots

```
x = np.linspace(-3, 3, 200)
y = np.linspace(-3, 3, 200)
X, Y = np.meshgrid(x, y)
Z = (np.exp(-(X**2 + Y**2) / 2)
     - 0.6 * np.exp(-((X - 1.5)**2 + (Y - 1)**2)))

fig, ax = plt.subplots(figsize=(6, 4.5))
filled = ax.contourf(X, Y, Z, levels=15, cmap='viridis')
lines = ax.contour(X, Y, Z, levels=8, colors='black',
                   linewidths=0.6)
ax.clabel(lines, fontsize=8)
fig.colorbar(filled, ax=ax, label='height')
```



- A contour plot shows a value over a 2-D grid, like height lines on a map
- `np.meshgrid` turns two 1-D ranges into 2-D grids of x and y coordinates
- `contourf` fills the bands, `contour` draws the lines, `clabel` writes their values

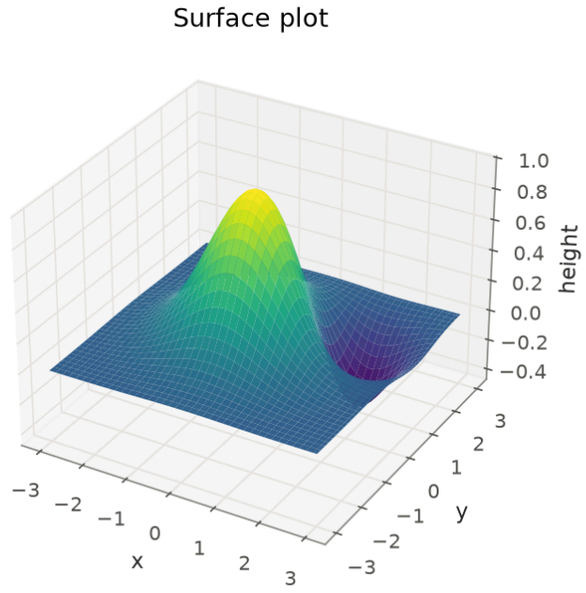
demo14
Contour

3D Plots

SpiralTrain
developer courses

```
x = np.linspace(-3, 3, 80)
y = np.linspace(-3, 3, 80)
X, Y = np.meshgrid(x, y)
Z = (np.exp(-(X**2 + Y**2) / 2)
     - 0.6 * np.exp(-((X - 1.5)**2 + (Y - 1)**2)))

fig, ax = plt.subplots(figsize=(6, 4.5),
                       subplot_kw={'projection': '3d'})
ax.plot_surface(X, Y, Z, cmap='viridis',
                edgecolor='none')
ax.set(title='Surface plot', xlabel='x', ylabel='y',
       zlabel='height')
```



- `subplot_kw={'projection': '3d'}` creates a 3D `Axes`
- `plot_surface`, `plot_wireframe`, `scatter` and `plot` draw in three dimensions
- In an interactive window the plot can be rotated with the mouse
- On paper, a contour plot or a heat map is usually easier to read

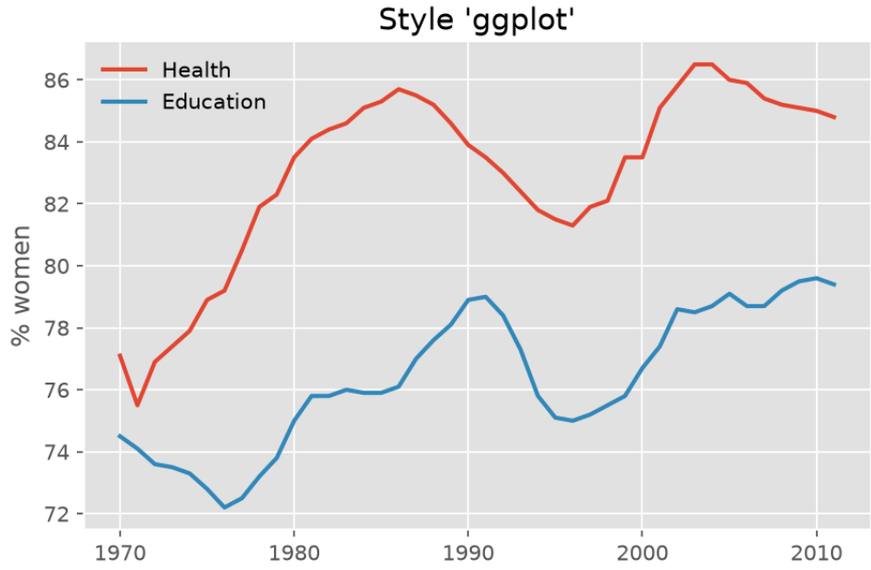
demo15
Surface 3D

Styles

```
print(plt.style.available[:6])

with plt.style.context('ggplot'):
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.plot(degrees['year'], degrees['health'],
            label='Health')
    ax.plot(degrees['year'], degrees['education'],
            label='Education')
    ax.set(title="Style 'ggplot'", ylabel='% women')
    ax.legend()
```

```
['Solarize_Light2', 'bmh', 'classic',
 'dark_background', 'fast', 'fivethirtyeight']
```



- A style sheet sets colors, fonts and backgrounds for every figure
- `plt.style.use('ggplot')` switches for the rest of the session
- `with plt.style.context(...)` applies a style only inside the block

demo16
Styles

Saving Figures

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(degrees['year'], degrees['computer_science'])
ax.set(title='Computer Science', ylabel='% women')

fig.savefig('degrees.png', dpi=200, bbox_inches='tight')
fig.savefig('degrees.svg')
fig.savefig('degrees.pdf')
plt.show()
plt.close(fig)
```

```
degrees.png 50530 bytes
degrees.svg 21977 bytes
degrees.pdf 11309 bytes
```

- The file extension chooses the format : PNG for slides and web, SVG and PDF scale without blur
- `dpi` sets the PNG resolution ; `bbox_inches='tight'` trims empty borders
- Save before `plt.show()` : after a script's window is closed, the figure can be empty
- `plt.close(fig)` frees the memory when a loop makes many figures

demo17
Saving

Plotting with pandas

```
aapl = pd.read_csv('AAPL.csv', parse_dates=['Date'],
                   index_col='Date')

fig, ax = plt.subplots(figsize=(7, 3.5))
aapl['Close'].plot(ax=ax)
ax.set(title='Apple weekly close', xlabel='',
       ylabel='US dollars')
```

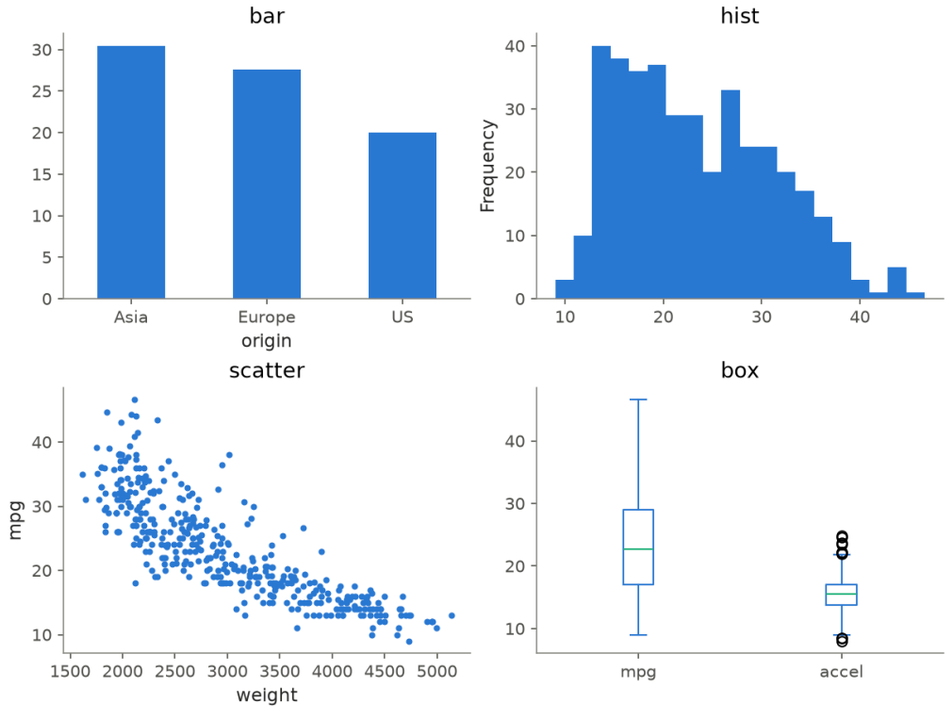


- `Series.plot()` and `DataFrame.plot()` draw with Matplotlib
- The index becomes the x-axis ; dates are formatted automatically
- `ax=` draws on an `Axes` you created ; without it pandas makes a new figure
- `.plot()` returns the `Axes`, so every `ax.` method still works afterwards

demo18
pandas line

pandas Plot Kinds

```
fig, axes = plt.subplots(2, 2, figsize=(8, 6),
                         layout='constrained')
(auto.groupby('origin')['mpg'].mean()
     .plot.bar(ax=axes[0, 0], rot=0, title='bar'))
auto['mpg'].plot.hist(ax=axes[0, 1], bins=20,
                      title='hist')
auto.plot.scatter(x='weight', y='mpg', s=8,
                  ax=axes[1, 0], title='scatter')
auto[['mpg', 'accel']].plot.box(ax=axes[1, 1],
                                title='box')
```



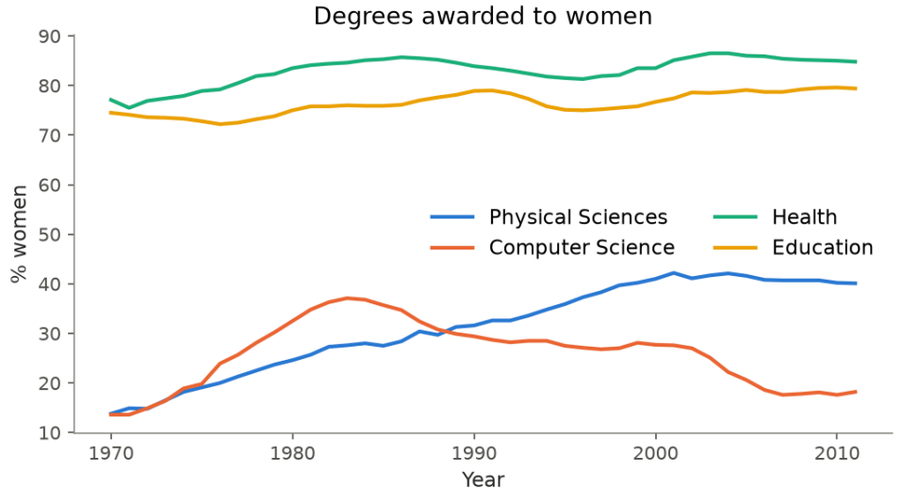
Method	Chart
`.plot()`, `.plot.line()`	line
`.plot.bar()`, `.plot.barh()`	bars
`.plot.hist()`, `.plot.box()`	distributions
`.plot.scatter(x, y)`	two columns
`.plot.area()`, `.plot.pie()`	parts of a whole

demo19
pandas kinds

Several Columns at Once

```
by_year = degrees.set_index('year')

fig, ax = plt.subplots(figsize=(7, 4))
by_year.plot(ax=ax)
ax.set(title='Degrees awarded to women',
       xlabel='Year', ylabel='% women')
ax.legend(['Physical Sciences', 'Computer Science',
           'Health', 'Education'], ncols=2)
```

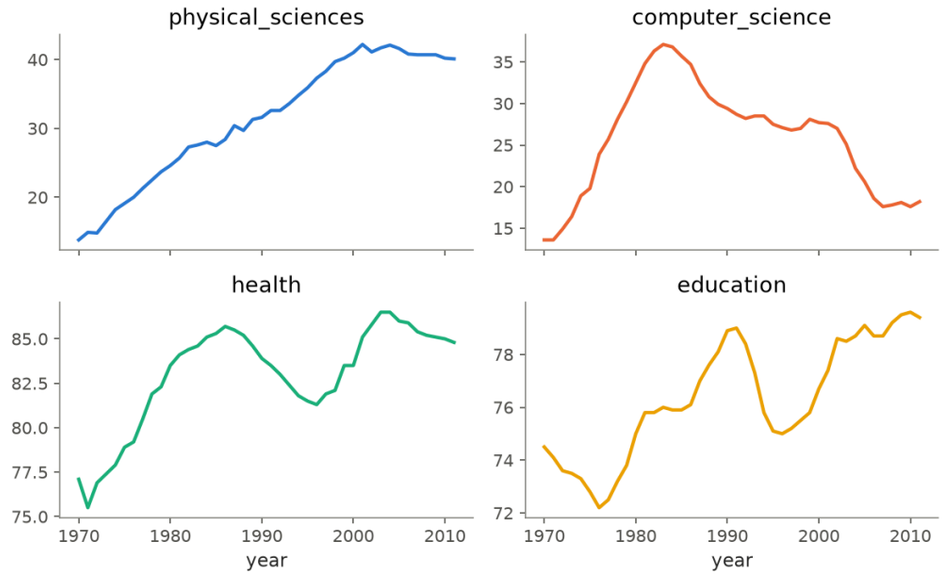


- `DataFrame.plot()` draws one line per column, with a legend
- Wide data with one column per series plots directly

demo20
pandas several columns

One Panel per Column

```
axes = by_year.plot(subplots=True, layout=(2, 2),
                    figsize=(8, 5), sharex=True,
                    legend=False,
                    title=list(by_year.columns))
fig = axes[0, 0].figure
fig.tight_layout()
```



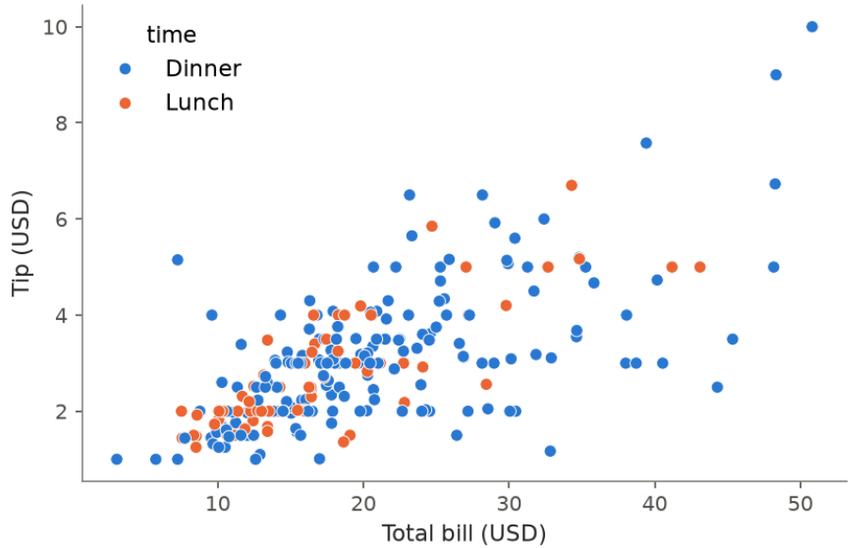
- `subplots=True` draws each column on its own `Axes`
- `layout=(rows, columns)` arranges the panels
- The result is the array of `Axes`; `.figure` gives their `Figure`
- Separate panels compare shapes when the series have different ranges

demo21
pandas subplots

Seaborn

```
import seaborn as sns

tips = pd.read_csv('tips.csv')
fig, ax = plt.subplots(figsize=(6, 4))
sns.scatterplot(data=tips, x='total_bill', y='tip',
                hue='time', ax=ax)
ax.set(xlabel='Total bill (USD)', ylabel='Tip (USD)')
```



- **Seaborn makes statistical charts from a DataFrame :**
  - `data=` is the frame ; `x=` , `y=` and `hue=` name its columns
  - Each row is one observation (tidy data)
- **`tips.csv` : 244 restaurant bills with tip, day, time and sex**

demo22
Seaborn tidy data

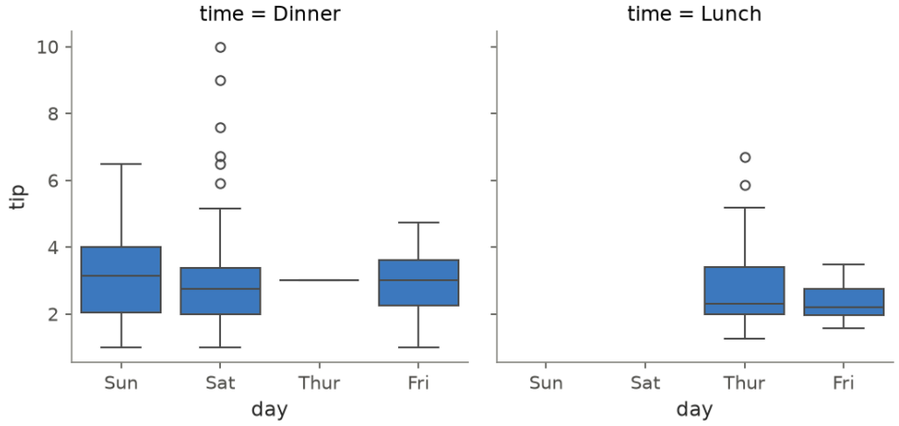
Axes-Level and Figure-Level Functions

SpiralTrain
developer courses

```
fig, ax = plt.subplots()
sns.boxplot(data=tips, x='day', y='tip', ax=ax)

g = sns.catplot(data=tips, x='day', y='tip',
                col='time', kind='box', height=3.5)
print(type(g).__name__)
```

```
FacetGrid
```

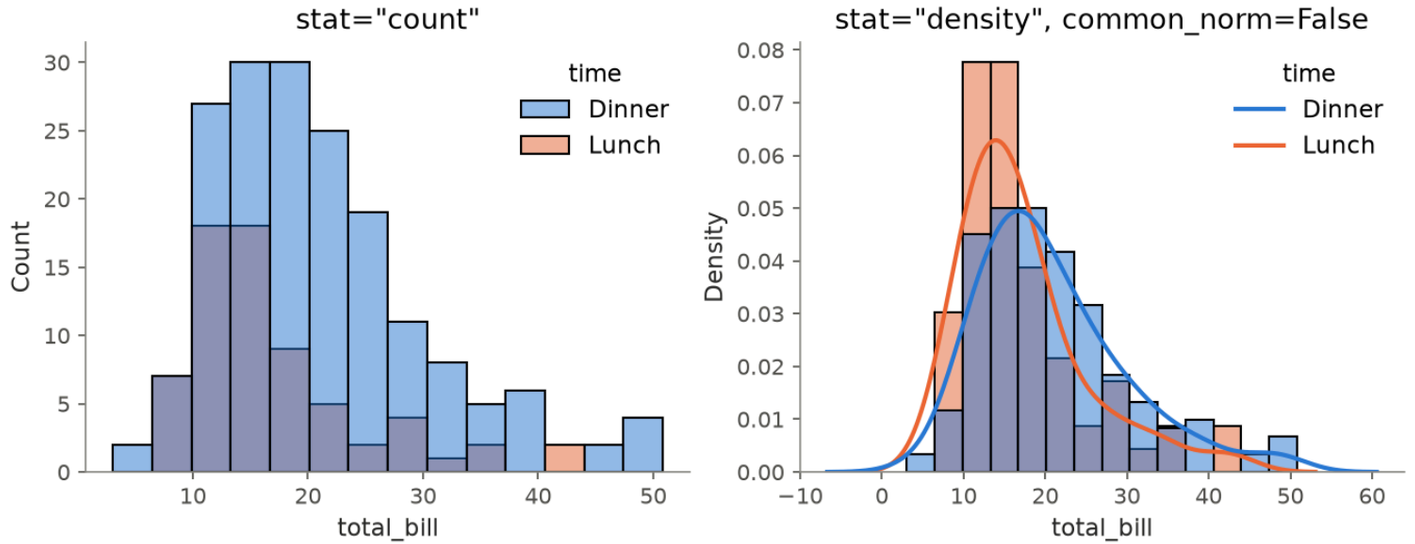


- Axes-level functions draw on one `Axes` and accept `ax=` :
  - `scatterplot`, `histplot`, `kdeplot`, `boxplot`, `barplot`, `regplot`
- Figure-level functions make a whole figure and return a `FacetGrid` :
  - `relplot`, `displot`, `catplot`, `lmplot`

demo23
Seaborn levels

Distributions

```
fig, axes = plt.subplots(1, 2, figsize=(9, 3.5), layout='constrained')
sns.histplot(data=tips, x='total_bill', hue='time', stat='count', ax=axes[0])
sns.histplot(data=tips, x='total_bill', hue='time', stat='density',
             common_norm=False, ax=axes[1])
sns.kdeplot(data=tips, x='total_bill', hue='time', common_norm=False, ax=axes[1])
```



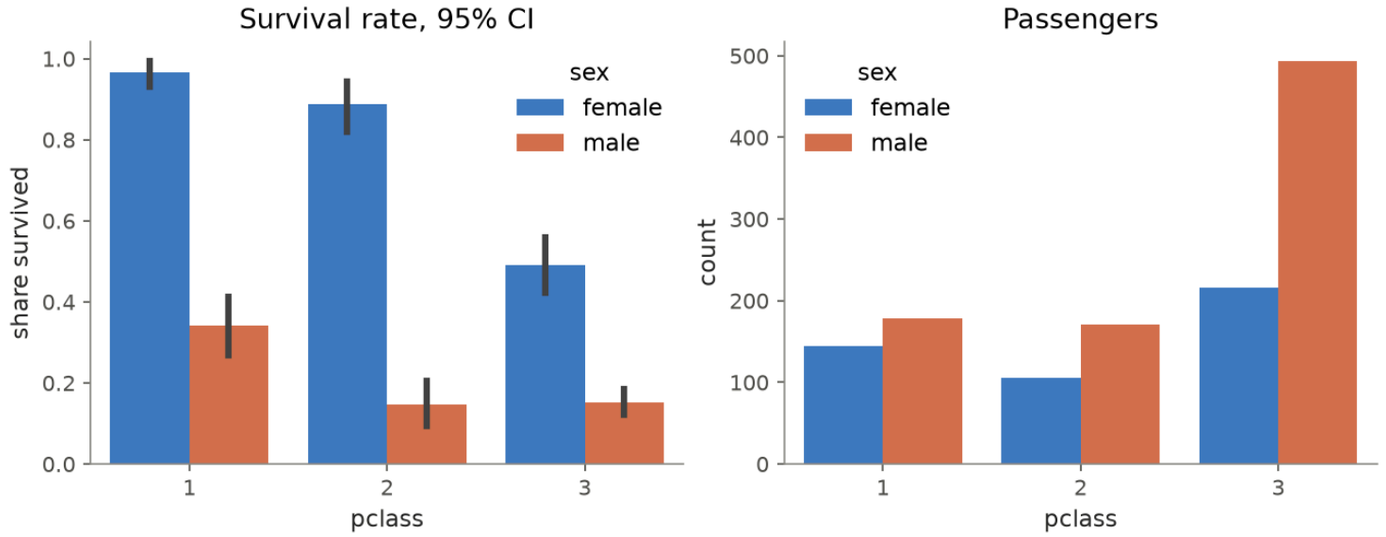
- `stat='count'` shows how many bills ; `stat='density'` makes each area sum to 1
- `common_norm=False` scales each `hue` group on its own, so lunch and dinner compare by shape
- `kdeplot` draws a smoothed density curve

demo24
Seaborn distributions

SpiralTrain developer courses

Estimates and Group Sizes

```
fig, axes = plt.subplots(1, 2, figsize=(9, 3.5), layout='constrained')
sns.barplot(data=titanic, x='pclass', y='survived', hue='sex',
            estimator='mean', errorbar=('ci', 95), seed=0, ax=axes[0])
sns.countplot(data=titanic, x='pclass', hue='sex', ax=axes[1])
```

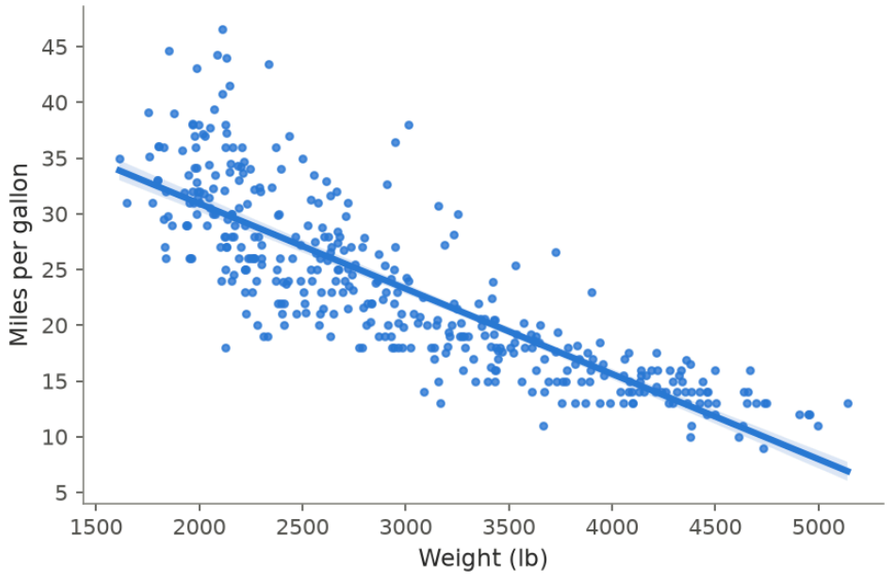


- `barplot` draws an estimate (here the mean of 0/1 : a survival rate) with a 95% interval
- A bar does not show how many rows it summarizes : `countplot` or `groupby(...).size()` does
- First-class women : rate 0.965 of 144 ; third-class men : 0.152 of 493

demo25
Seaborn estimates

Fitted Lines

```
fig, ax = plt.subplots(figsize=(6, 4))
sns.regplot(data=auto, x='weight', y='mpg', seed=0,
            scatter_kws={'s': 10}, ax=ax)
ax.set(xlabel='Weight (lb)',
       ylabel='Miles per gallon')
```

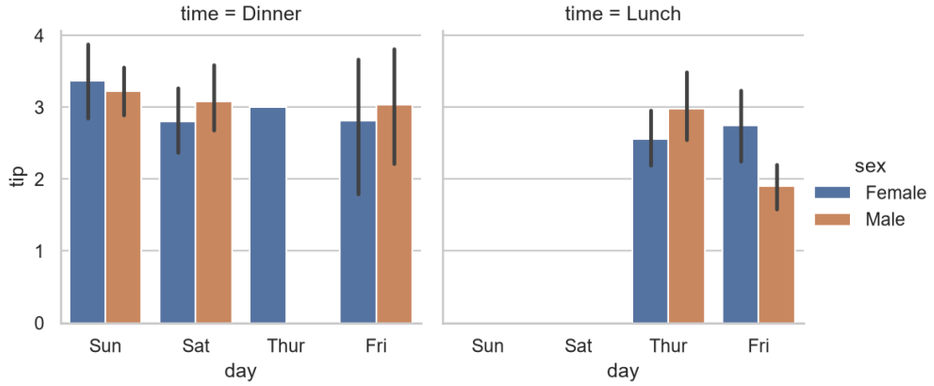


- `regplot` draws the points and a fitted straight line with its confidence band
- `lmplot` is the figure-level version, with `col=` and `row=`
- The line summarizes an association in this data ; it does not show that weight causes fuel use

demo26
Seaborn regplot

Facets and Themes

```
sns.set_theme(style='whitegrid')
g = sns.catplot(data=tips, x='day', y='tip',
                hue='sex', col='time', kind='bar',
                errorbar=('ci', 95), seed=0,
                height=3.5)
g.savefig('seaborn-facets.png')
print(tips.groupby(['time', 'day', 'sex'],
                  observed=True).size())
```



- `col=` and `row=` split the data into panels
- `sns.set_theme()` applies a Seaborn theme to all later figures
- A `FacetGrid` saves with `g.savefig()`
- Thursday dinner holds one bill : that bar is a single tip, not an average

demo27
Seaborn facets

Chart Checklist

- **Choose the chart type for the comparison :**
  - Change over time : line ; comparison of categories : bar ; distribution : histogram or box ; relation : scatter
- **Let the title state the finding : ”Heavier cars use more fuel”, not ”Weight vs mpg”**
- **Label both axes, with units**
- **Bars start at zero ; sort them unless the categories have an order**
- **Use at most a few colors, and add labels or line styles so color is never the only cue**
- **Show how many observations stand behind each bar or mean**
- **One y-axis per chart ; two measures with different units get two charts**

SpiralTrain developer courses

Summary : Matplotlib

- `fig, ax = plt.subplots()` creates the figure ; `ax.` methods draw and label
- Line, scatter, histogram, box, bar, pie, contour and 3D plots share this pattern
- `plt.subplots(rows, columns)` makes grids ; `layout='constrained'` spaces them
- `ax.set` , `ax.legend` , `ax.annotate` , limits, ticks and scales finish a chart
- `fig.savefig()` before `plt.show()`
- pandas `.plot(ax=ax)` charts Series and DataFrames directly
- Seaborn adds statistical charts from tidy DataFrames

- NumPy
- Pandas Intro
- Selecting and Filtering
- Cleaning and Transforming
- Advanced Indexing
- Rearranging Data
- Combining Data
- Grouping Data
- Matplotlib
- ▶ **Time Series**
- Integration Case
- Scikit-Learn Essentials
- Data Formats and Tools

APPENDIX

- Reading Other Sources

- Read Dates into a DatetimeIndex
- Selecting Dates
- Slicing and Converting
- Date Ranges and Gaps
- Reindexing and Filling
- Filling Event Data
- Resampling
- Resampling Frequencies
- Verifying a Resampled Value
- Daily Totals and Chaining
- Empty Intervals
- Resampling by Week
- Binning Events
- Upsampling
- Inspecting a Gap in the Clock
- Conditional Gap Filling
- Rolling Windows
- shift, diff and pct_change
- Plotting a Time Series
- Datetime Parts with .dt
- Counting with Booleans
- Time Zones
- Storing Timestamps in UTC
- Daylight-Saving Transitions

Read Dates into a DatetimeIndex

```
sales = pd.read_csv('sales-feb-2026.csv',
                    parse_dates=['Date'],
                    index_col='Date')
print(sales.head())
sales.info()
```

```
                                Company   Product  Units
Date
2026-02-02 08:30:00               Hooli  Software      3
2026-02-02 21:00:00           Mediacore  Hardware      9
2026-02-03 14:00:00             Initech  Software     13
2026-02-04 15:30:00           Streeplex  Software     13
2026-02-04 22:00:00  Acme Corporation  Hardware     14
<class 'pandas.DataFrame'>
DatetimeIndex: 19 entries, 2026-02-02 08:30:00
  to 2026-02-26 09:00:00
Data columns (total 3 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   Company  19 non-null     str
 1   Product  19 non-null     str
 2   Units    19 non-null     int64
```

- `parse_dates` turns text into timestamps
- `index_col='Date'` makes them the row index : a `DatetimeIndex`
- ISO 8601 text (`2026-02-02 08:30:00`) needs no hints ; other layouts need `date_format` , such as `'%Y%m%d %H:%M'`
- `sales-feb-2026.csv` : an invented log of 19 orders in February 2026

demo01
Read dates

Selecting Dates

```
print(sales.loc['2026-02-19 11:00:00', 'Company'])
print(sales.loc['2026-02-05'])
print(sales.loc['February 5, 2026'].shape, sales.loc['2026-Feb-5'].shape)
print(sales.loc['2026-02'].shape, sales.loc['2026'].shape)
```

```
Mediacore
                            Company   Product  Units
Date
2026-02-05 02:00:00  Acme Corporation  Software     19
2026-02-05 22:00:00             Hooli   Service     10
(2, 3) (2, 3)
(19, 3) (19, 3)
```

- A full timestamp selects one row ; a date selects the whole day
- A partial string selects a whole month or year : `'2026-02'` , `'2026'`
- Other spellings of a date are accepted ; ISO strings are the clearest

demo02
Selecting dates

Slicing and Converting

```
print(sales.loc['2026-02-16':'2026-02-20'])
evening = pd.to_datetime(['2026-02-11 20:00', '2026-02-11 21:00',
                          '2026-02-11 22:00', '2026-02-11 23:00'])
print(evening)
```

```
                     Company   Product  Units
Date
2026-02-16 12:00:00      Hooli  Software     10
2026-02-19 11:00:00  Mediacore  Hardware     16
2026-02-19 16:00:00  Mediacore   Service     10
DatetimeIndex(['2026-02-11 20:00:00', '2026-02-11 21:00:00',
               '2026-02-11 22:00:00', '2026-02-11 23:00:00'],
              dtype='datetime64[us]', freq=None)
```

- Date slices include both end points, like other label slices
- `pd.to_datetime` converts strings to timestamps ; `format=` fixes the layout

demo03
Slicing and converting

Date Ranges and Gaps

```
evening = pd.date_range('2026-02-11 20:00', periods=4, freq='h')
print(evening)

expected = pd.date_range(weather.index.min(), weather.index.max(), freq='h')
missing = expected.difference(weather.index)
print(len(expected), len(weather))
print(missing)
```

```
DatetimeIndex(['2026-02-11 20:00:00', '2026-02-11 21:00:00',
               '2026-02-11 22:00:00', '2026-02-11 23:00:00'],
              dtype='datetime64[us]', freq='h')
8760 8759
DatetimeIndex(['2010-03-14 03:00:00'], dtype='datetime64[us]', freq='h')
```

- `pd.date_range(start, end or periods, freq=)` builds a regular sequence of timestamps
- `weather` : hourly Austin readings for 2010, read with `date_format='%Y%m%d %H:%M'`
- `expected.difference(actual)` lists the timestamps that have no row

demo04
Date range and gaps

Reindexing and Filling

```
print(sales.reindex(evening))
print(sales.reindex(evening, method='ffill'))
```

```
                     Company   Product  Units
2026-02-11 20:00:00  Initech  Software    7.0
2026-02-11 21:00:00      NaN       NaN    NaN
2026-02-11 22:00:00      NaN       NaN    NaN
2026-02-11 23:00:00    Hooli  Software    4.0
```

```
                     Company   Product  Units
2026-02-11 20:00:00  Initech  Software      7
2026-02-11 21:00:00  Initech  Software      7
2026-02-11 22:00:00  Initech  Software      7
2026-02-11 23:00:00    Hooli  Software      4
```

- `reindex` places the rows on new timestamps ; timestamps without a row get `NaN`
- `method='ffill'` copies the previous row forward, `method='bfill'` the next row backward

demo05
Reindex and fill

Filling Event Data

- **`ffill` and `bfill` copy an adjacent row into each new timestamp :**
  - The 21:00 and 22:00 rows on the previous slide are invented, not recorded sales
  - The fill shows the mechanics ; it is not a valid sales record
- **Count or sum recorded events per interval with `resample` :**
  - No rows means zero events only if the log is complete for that period
  - Otherwise an empty interval is unknown
- **Forward fill suits a quantity that persists between readings :**
  - A temperature, a stock level or a price
  - Gap filling on a temperature series follows later

Resampling

- `resample(rule)` groups rows into time intervals, then aggregates each interval :
  - Works like `groupby`, with intervals as the groups
  - `mean()`, `sum()`, `count()`, `agg([...])` and other aggregations
- Downsampling : to a slower frequency, such as hourly to daily
- Upsampling : to a faster frequency, such as daily to hourly

```
daily_mean = sales['Units'].resample('D').mean()
print(daily_mean.head(8))
```

```
Date
2026-02-02     6.0
2026-02-03    13.0
2026-02-04    13.5
2026-02-05    14.5
2026-02-06     NaN
...
```

- Select the numeric column first : the mean of a text column raises a `TypeError`

demo06
Resample mean

Resampling Frequencies

Rule	Meaning
`'min'`	minute
`'h'`	hour
`'D'`	calendar day
`'B'`	business day
`'W'`	week ending Sunday
`'ME'`, `'MS'`	month end, start
`'QE'`, `'QS'`	quarter end, start
`'YE'`, `'YS'`	year end, start

- A number multiplies a frequency :
  - `'15min'`, `'4h'`, `'2W'`
- A suffix anchors a frequency :
  - `'W-MON'` : weeks ending Monday
  - `'QE-JUN'` : quarters of a year ending in June
- pandas 3 rejects the old rules `'T'`, `'H'`, `'M'`, `'Q'`, `'A'` and `'Y'`
- The same rules work in `pd.date_range(freq=...)`

Verifying a Resampled Value

```
print(daily_mean.loc['2026-02-02'])
print(sales.loc['2026-02-02', 'Units'])
print(sales.loc['2026-02-02', 'Units'].mean())
```

```
6.0
Date
2026-02-02 08:30:00    3
2026-02-02 21:00:00    9
Name: Units, dtype: int64
6.0
```

- Compare one interval with the rows it summarizes
- The two sales on 2 February, 3 and 9 units, give the daily mean 6.0

demo07
Check a resampled value

Daily Totals and Chaining

```
daily_sum = sales['Units'].resample('D').sum()
print(daily_sum.head(8))
print(sales['Units'].resample('D').sum().max())
print(sales['Units'].resample('D').sum().idxmax())
```

- `resample('D').sum()` gives the units sold per day ; a day without sales sums to 0
- Each method works on the result of the previous one
- `max()` finds the best day's total, `idxmax()` its date

```
Date
2026-02-02    12
2026-02-03    13
2026-02-04    27
2026-02-05    29
2026-02-06     0
2026-02-07     1
2026-02-08     0
2026-02-09    26
Freq: D, Name: Units, dtype: int64
29
2026-02-05 00:00:00
```

demo08
Resample sum

Empty Intervals

```
daily = sales['Units'].resample('D')
print(daily.agg(['count', 'sum', 'mean']).loc['2026-02-05':'2026-02-08'])
print(sales['Units'].resample('D').sum(min_count=1).loc['2026-02-05':'2026-02-08'])
```

```
            count  sum  mean
Date
2026-02-05      2   29  14.5
2026-02-06      0    0   NaN
2026-02-07      1    1   1.0
2026-02-08      0    0   NaN
```

```
Date
2026-02-05    29.0
2026-02-06     NaN
2026-02-07     1.0
2026-02-08     NaN
Freq: D, Name: Units, dtype: float64
```

- An empty day has `count` 0, `sum` 0 and `mean` `NaN`
- In a complete sales log, 0 units is correct ; in a sensor log an empty interval is unknown
- `sum(min_count=1)` returns `NaN` for an empty interval ; report `count` beside an aggregate

demo09
Empty intervals

Resampling by Week

```
print(sales.resample('W').count())
print(sales['Units'].resample('2W').sum())
```

- `'W'` makes weekly bins ; `count()` counts the rows in each week
- `'2W'` makes two-week bins
- Weekly and monthly-end bins carry the label of their last day
- `'D'` and `'h'` bins carry the label of their start

```
            Company  Product  Units
Date
2026-02-08        8        8      8
2026-02-15        4        4      4
2026-02-22        5        5      5
2026-03-01        2        2      2
Date
2026-02-08    82
2026-02-22    79
2026-03-08    14
Freq: 2W-SUN, Name: Units, dtype: int64
```

demo10
Resample week

Binning Events

```
two_days = sales.loc['2026-02-04':'2026-02-05',
                     'Units']
print(two_days)
print(two_days.resample('4h').sum())
```

```
Date
2026-02-04 15:30:00    13
2026-02-04 22:00:00    14
2026-02-05 02:00:00    19
2026-02-05 22:00:00    10
Name: Units, dtype: int64
```

```
Date
2026-02-04 12:00:00    13
2026-02-04 16:00:00     0
2026-02-04 20:00:00    14
2026-02-05 00:00:00    19
2026-02-05 04:00:00     0
2026-02-05 08:00:00     0
2026-02-05 12:00:00     0
2026-02-05 16:00:00     0
2026-02-05 20:00:00    10
Freq: 4h, Name: Units, dtype: int64
```

- Irregular events become a regular series : each 4-hour bin sums the sales inside it
- A bin without sales gets 0, which is correct for a complete sales log

demo11
Binning events

Upsampling

```
daily = (weather['Temperature'].resample('D')
         .mean().head(2).round(1))
print(daily.resample('6h').asfreq())
print(daily.resample('6h').ffill())
print(daily.resample('6h').interpolate())
```

```
Date
2010-01-01 00:00:00    49.3
2010-01-01 06:00:00     NaN
2010-01-01 12:00:00     NaN
2010-01-01 18:00:00     NaN
2010-01-02 00:00:00    49.8
...
2010-01-01 06:00:00    49.3
2010-01-01 12:00:00    49.3
...
2010-01-01 06:00:00    49.425
2010-01-01 12:00:00    49.550
2010-01-01 18:00:00    49.675
...
```

- Upsampling creates timestamps that have no observation
- `asfreq()` leaves them missing
- `ffill()` repeats the last value
- `interpolate()` draws a straight line between values
- Each filled value is an estimate ; choose the rule that fits how the quantity behaves

demo12
Upsampling

Inspecting a Gap in the Clock

```
hourly = weather['Temperature'].asfreq('h')
print(hourly.isna().sum())
print(hourly.loc['2010-03-14 01:00':'2010-03-14 05:00'])
```

```
1
Date
2010-03-14 01:00:00    55.7
2010-03-14 02:00:00    54.8
2010-03-14 03:00:00     NaN
2010-03-14 04:00:00    53.9
2010-03-14 05:00:00    53.4
Freq: h, Name: Temperature, dtype: float64
```

- `asfreq('h')` puts the series on a regular hourly grid ; a missing hour becomes `NaN`
- The gap falls on the day the US clocks moved forward : check the file's clock before filling

demo13
Clock gap

Conditional Gap Filling

```
print(hourly.ffill()
      .loc['2010-03-14 02:00':'2010-03-14 04:00'])
```

```
Date
2010-03-14 02:00:00    54.8
2010-03-14 03:00:00    54.8
2010-03-14 04:00:00    53.9
```

```
print(hourly.interpolate(method='time')
      .loc['2010-03-14 02:00':'2010-03-14 04:00']
```

```
Date
2010-03-14 02:00:00    54.80
2010-03-14 03:00:00    54.35
2010-03-14 04:00:00    53.90
```

- `ffill` repeats the last reading ; `interpolate(method='time')` draws a line between readings
- **Both assume a missing reading on a regular clock :**
  - A clock-change artifact is not a missing measurement
  - A filled value is an assumption : report the method and how many values were filled

demo14
Gap filling

Rolling Windows

SpiralTrain
developer courses

```
aapl = pd.read_csv('AAPL.csv', parse_dates=['Date'], index_col='Date')
close = aapl['Close'].tail(8)
table = pd.DataFrame({'close': close,
                      'rolling': close.rolling(window=4).mean(),
                      'partial': close.rolling(window=4, min_periods=2).mean(),
                      'centered': close.rolling(window=3, center=True).mean()})
print(table.round(2))
```

```
              close
Date
2026-08-17  309.35
2026-08-24  319.70
2026-08-31  319.97
2026-09-07  332.27
2026-09-14  336.13
...
```

- `rolling(window=4).mean()` averages each value with the three before it : a moving average
- The first values have too few neighbours : `NaN`, unless `min_periods` allows fewer
- `center=True` puts the window around each value instead of before it
- Also `.sum()`, `.max()`, `.std()` ; a window like `'30D'` uses time instead of a count

demo15
Rolling

shift, diff and pct_change

```
close = aapl['Close'].tail(5)
print(pd.DataFrame({'close': close, 'previous': close.shift(1),
                    'change': close.diff(),
                    'pct': close.pct_change().mul(100).round(2)}))
```

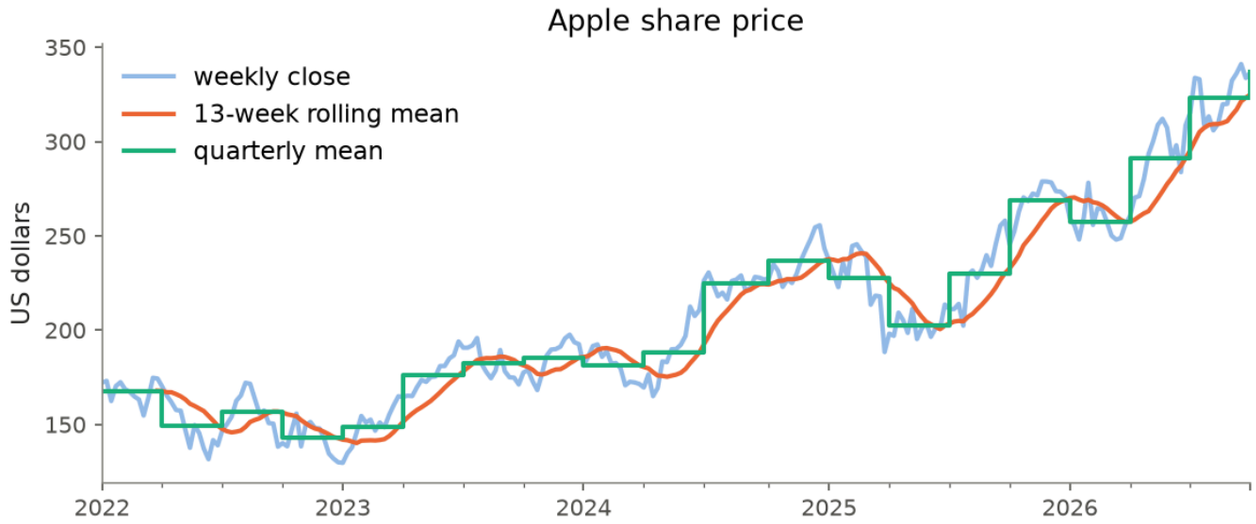
```
              close  previous  change   pct
Date
2026-09-07  332.27       NaN     NaN   NaN
2026-09-14  336.13    332.27    3.86  1.16
2026-09-21  341.07    336.13    4.94  1.47
2026-09-28  333.69    341.07   -7.38 -2.16
2026-10-05  336.64    333.69    2.95  0.88
```

- `shift(1)` moves every value one row later : each row sees the previous value
- `diff()` is the change from the previous row ; `pct_change()` the relative change
- The first row has no predecessor : `NaN`
- Use them on a sorted index ; `shift(-1)` looks one row ahead

demo16
shift, diff, pct_change

Plotting a Time Series

```
close = aapl['Close']
fig, ax = plt.subplots(figsize=(8, 3.5))
close.plot(ax=ax, alpha=0.5, label='weekly close')
close.rolling(window=13).mean().plot(ax=ax, label='13-week rolling mean')
close.resample('QS').mean().plot(ax=ax, drawstyle='steps-post', label='quarterly mean')
ax.set(title='Apple share price', xlabel='', ylabel='US dollars')
ax.legend()
```



- A `DatetimeIndex` becomes a formatted date axis ; overlay smoothed versions to show the trend

demo17
Plot a time series

Datetime Parts with .dt

SpiralTrain
developer courses

```
sales = pd.read_csv('sales-feb-2026.csv', parse_dates=['Date'])
print(sales['Date'].dt.hour.head(3).tolist())
print(sales['Date'].dt.day_name().head(3).tolist())
print(sales['Date'].dt.strftime('%d %b %H:%M').head(3).tolist())
print(sales.groupby(sales['Date'].dt.day_name())['Units'].sum()
      .sort_values(ascending=False).head(3))
```

```
[8, 21, 14]
['Monday', 'Monday', 'Tuesday']
['02 Feb 08:30', '02 Feb 21:00', '0
Date
Thursday     59
Monday       48
Wednesday    48
Name: Units, dtype: int64
```

- Without `index_col`, the dates are an ordinary column with the `.dt` accessor
- `.dt.hour`, `.dt.day`, `.dt.month`, `.dt.year`, `.dt.dayofweek` (Monday is 0), `.dt.day_name()`
- `.dt.strftime(format)` formats dates as text : `%d` day, `%b` month name, `%H:%M` time

demo18
dt accessor

Counting with Booleans

```
is_ware = sales['Product'].str.contains('ware')
print(True + True, True + False)   # 2 1
print(is_ware.sum(), len(sales))   # 14 19
print(is_ware.mean().round(3))     # 0.737
```

- In arithmetic, `True` counts as 1 and `False` as 0
- `sum()` of a Boolean `Series` counts the `True` values : 14 of 19 orders are software or hardware
- `mean()` gives the share of `True` values
- `.str.contains` is one of the text methods from Module 4

demo19
Boolean arithmetic

Time Zones

```
central = sales['Date'].dt.tz_localize('America/Chicago')
print(central.head(3))
eastern = central.dt.tz_convert('America/New_York')
print(eastern.head(3))
print(sales['Date'].iloc[7], '->', eastern.iloc[7])
```

```
0   2026-02-02 08:30:00-06:00
1   2026-02-02 21:00:00-06:00
2   2026-02-03 14:00:00-06:00
Name: Date, dtype: datetime64[us, America/Chicago]
0   2026-02-02 09:30:00-05:00
...
2026-02-07 23:00:00 -> 2026-02-08 00:00:00-05:00
```

- `tz_localize` attaches a time zone to naive timestamps ; the clock time stays
- `tz_convert` converts to another zone ; the instant stays
- 23:00 in Chicago on 7 February is already 8 February in New York

Storing Timestamps in UTC

```
sales = pd.read_csv('sales-feb-2026.csv', parse_dates=['Date'], index_col='Date')
print(sales.index.is_monotonic_increasing, sales.index.has_duplicates)
utc = sales.index.tz_localize('America/Chicago').tz_convert('UTC')
print(utc[:3])
```

```
True False
DatetimeIndex(['2026-02-02 14:30:00+00:00', '2026-02-03 03:00:00+00:00',
               '2026-02-03 20:00:00+00:00'],
              dtype='datetime64[us, UTC]', name='Date', freq=None)
```

- Sort the index before slicing a time range ; `sort_index()` fixes the order
- Decide whether a repeated timestamp is a second event or a duplicate record
- UTC has no daylight-saving changes : store UTC, convert for display

Daylight-Saving Transitions

SpiralTrain
developer courses

```
times = pd.Series(pd.to_datetime(['2026-03-08 02:30', '2026-11-01 01:30']))
print(times.dt.tz_localize('America/Chicago', nonexistent='shift_forward',
                           ambiguous='NaT'))
try:
    times.dt.tz_localize('America/Chicago')
except Exception as error:
    print(type(error).__name__, error)
```

```
0   2026-03-08 03:00:00-05:00
1                         NaT
dtype: datetime64[us, America/Chicago]
ValueError 2026-03-08 02:30:00 is a nonexistent time due to daylight savings time.
Try using the 'nonexistent' argument.
```

- 02:30 on 8 March 2026 does not exist in Chicago ; 01:30 on 1 November occurs twice
- `tz_localize` raises an error unless a policy is given : `nonexistent=` and `ambiguous=`
- `ambiguous='infer'` works for an ordered series that passes through the repeated hour

demo22
DST transitions

SpiralTrain developer courses

Summary : Time Series

- `parse_dates` and `index_col` give a `DatetimeIndex` ; select by date strings and slices
- `pd.date_range` builds regular timestamps ; `difference` finds gaps ; `reindex` aligns to them
- `resample` aggregates by interval ; report `count` beside aggregates of event data
- Upsampling and gap filling are estimates : `asfreq` , `ffill` , `interpolate`
- `rolling` for moving windows ; `shift` , `diff` and `pct_change` for changes
- `.dt` for parts and formatting ; `tz_localize` and `tz_convert` for time zones
- Exercise 10 counts scheduled flights : define the population before counting

exercise10
Time Series

- NumPy
- Pandas Intro
- Selecting and Filtering
- Cleaning and Transforming
- Advanced Indexing
- Rearranging Data
- Combining Data
- Grouping Data
- Matplotlib
- Time Series
- **Integration Case**
- Scikit-Learn Essentials
- Data Formats and Tools

APPENDIX

- Reading Other Sources

- Integration Case
- Case Definition and Scope
- Data Provenance
- Data Quality Issues
- Inspecting the Raw Files
- Constants
- Loading with Types
- Selecting the Death Counts
- Mapping Age Codes to Labels
- Banding the Population
- Checking the Bands
- Building the Country and Age Grid
- Missing Counts
- Interpreting the Result
- Comparison with Published Rates
- Charting the Rates
- Reading the Chart
- Reviewing Generated Code
- Checking the Grain
- Checking the Age Mapping
- Reconciling the Totals
- Data Sharing and Privacy
- Limitations and Reproducibility
- A Local Streamlit App
- Exercise 11

SpiralTrain developer courses

Integration Case

- **The course closes with one complete analysis :**
  - Two public Eurostat tables : road deaths and population
  - A checked summary, a chart and stated limitations
- **It uses the techniques of the earlier modules :**
  - Typed loading, filtering and text labels
  - Grouping into age bands and joining at the right grain
  - Rates with matching denominators and visible gaps
- **Twelve numbered steps ; every slide shows all the code of its step**
- **At the end you audit a proposal from an ”AI assistant” against your own results**

Case Definition and Scope

- **Research question : in 2024, how many road deaths per million inhabitants were recorded in each age group, across 33 European countries?**
  - The EU27 countries plus Switzerland, Iceland, Liechtenstein, Norway, Montenegro and Serbia
- **The two inputs :**
  - Deaths : counts by country, age group, sex and road user
  - Population : on 1 January 2024, by single year of age
- **Consequences for the analysis :**
  - Observation unit : one country and age group, 33 × 7 cells
  - Denominator : the population of the countries that report the group
  - Unknown ages : counted, never shared out over the age groups

SpiralTrain developer courses

Data Provenance

- **Record where each table comes from before interpreting it :**
  - Eurostat `tran_sf_roadus` (road deaths) and `demo_pjan` (population)
  - Retrieved 10 October 2026 ; Eurostat updated them on 25 August and 25 September
  - Deaths during 2024 ; population on 1 January 2024
- **Retrieval date, release date and reference period are three different dates**
- **Reuse is permitted with acknowledgement ; the files are kept unchanged**
- **Queries, fetch script and definitions :** `datasets/eurostat-road-safety-README.md`
- **A road death means death within 30 days of an injury accident**

SpiralTrain developer courses

Data Quality Issues

- **None of these defects was planted ; all come from the published files :**
  - `TOTAL` rows and the `EU27_2020` aggregate sit beside their parts
  - Counts (`NR`) and rates (`P_MHAB`) share one value column
  - Missing data are absent rows : Liechtenstein (LI) and Montenegro (ME) record no ages
  - Deaths use age bands ; population uses single years of age
  - Some population values are flagged estimated or provisional
- **Summing every row double counts : `EU27_2020` equals the sum of its 27 members**
- **No value is empty, so `isna()` finds nothing missing :**
  - Absent rows only show up once the expected grid of cells is built

Inspecting the Raw Files

```
raw = pd.read_csv(
    'eurostat-road-deaths-2023-2024.csv')
print(raw.shape)
print(raw[['unit', 'sex', 'age', 'pers_cat', 'geo',
           'TIME_PERIOD', 'OBS_VALUE']].head(3))
print(raw['unit'].unique().tolist())
print(raw['age'].unique().tolist())
print(raw['geo'].nunique(),
      'EU27_2020' in set(raw['geo']))
```

```
(12254, 12)
  unit sex    age pers_cat geo  TIME_PERIOD  OBS
0   NR   F  TOTAL     DRIV  AT         2023
1   NR   F  TOTAL     DRIV  AT         2024
2   NR   F  TOTAL     DRIV  BE         2023
['NR', 'P_MHAB']
['TOTAL', 'UNK', 'Y15-17', 'Y18-24', 'Y25-49',
 'Y50-64', 'Y_GE65', 'Y_LT15']
34 True
```

- One value per unit, sex, age, road user (`pers_cat`), country (`geo`) and year
- Two units : counts `NR` and rates per million `P_MHAB`
- Age codes are bands ; `UNK` means unknown age
- 34 geos : 33 countries plus the `EU27_2020` aggregate

Constants

```
AGE_CODES = {'Y_LT15': 'Under 15', 'Y15-17': '15-17', 'Y18-24': '18-24',
             'Y25-49': '25-49', 'Y50-64': '50-64', 'Y_GE65': '65+',
             'UNK': 'Unknown age'}
LABELS = list(AGE_CODES.values())
BINS = [0, 15, 18, 25, 50, 65, float('inf')]
print(LABELS)
```

```
['Under 15', '15-17', '18-24', '25-49', '50-64', '65+', 'Unknown age']
```

- `AGE_CODES` maps Eurostat's age codes to readable labels
- `LABELS` lists the seven age groups in order
- `BINS` are the band edges for single years of age : 0 to 14, 15 to 17, 18 to 24, 25 to 49, 50 to 64, 65+
- Writing the constants down first makes the rules visible and reusable

SpiralTrain developer courses

Loading with Types

```
names = {'TIME_PERIOD': 'year', 'OBS_VALUE': 'value', 'OBS_FLAG': 'flag'}
deaths = pd.read_csv('eurostat-road-deaths-2023-2024.csv',
    usecols=['unit', 'sex', 'age', 'pers_cat', 'geo', 'TIME_PERIOD', 'OBS_VALUE'],
    dtype={'TIME_PERIOD': 'Int64', 'OBS_VALUE': 'Int64'}).rename(columns=names)
population = pd.read_csv('eurostat-population-2024.csv',
    usecols=['sex', 'age', 'geo', 'TIME_PERIOD', 'OBS_VALUE', 'OBS_FLAG'],
    dtype={'TIME_PERIOD': 'Int64', 'OBS_VALUE': 'Int64'}).rename(columns=names)
print(deaths.shape, population.shape)
print(deaths['value'].isna().sum(), population['value'].isna().sum())
print(population['flag'].value_counts())
```

```
(12254, 7) (10500, 6)
0 0
flag
ep    411
p     309
e     306
```

- `usecols` reads only the needed columns ; codes stay text, numbers become `Int64`
- Flags : `e` estimated, `p` provisional ; the values are still used, but worth mentioning

demo01
Case steps

Selecting the Death Counts

```
counts = deaths.loc[deaths['unit'].eq('NR') & deaths['pers_cat'].eq('TOTAL')
                    & deaths['year'].eq(2024) & deaths['sex'].eq('T')
                    & deaths['geo'].ne('EU27_2020')]
COUNTRIES = sorted(counts['geo'].unique())
totals = counts.loc[counts['age'].eq('TOTAL')].set_index('geo')['value']
counts = counts.loc[counts['age'].ne('TOTAL')]
print(len(COUNTRIES), len(counts), totals.sum())   # 33 219 20875
```

- Counts only (`NR`), all road users (`TOTAL`), 2024, both sexes (`T`), countries only
- `totals` keeps each country's published total for a later check
- 219 age rows, not 33 × 7 = 231 : some cells are absent
- 20,875 road deaths in the 33 countries

Mapping Age Codes to Labels

```
counts = counts.assign(age_group=counts['age'].map(AGE_CODES))
counts = counts[['geo', 'age_group', 'value']].rename(columns={'value': 'deaths'})
print(counts.head(4))
print(counts['age_group'].isna().sum())
```

```
     geo    age_group  deaths
6744  AT  Unknown age       1
6746  BE  Unknown age      12
6748  BG  Unknown age       0
6750  CH  Unknown age       0
0
```

- `map(AGE_CODES)` turns each code into its label
- No missing labels : every code in the file is in the dictionary
- `assign` adds a column and returns a new frame

Banding the Population

```
people = population.loc[population['sex'].eq('T') & population['geo'].ne('EU27_2020')]
ages = people.loc[~people['age'].isin(['TOTAL', 'UNK'])]
years = ages['age'].replace({'Y_LT1': 'Y0', 'Y_OPEN': 'Y100'}).str[1:].astype(int)
ages = ages.assign(age_group=pd.cut(years, BINS, right=False, labels=LABELS[:-1]))
bands = (ages.groupby(['geo', 'age_group'], observed=True)['value'].sum()
         .reset_index(name='population'))
bands['age_group'] = bands['age_group'].astype(str)
print(bands.head(3))
```

```
  geo age_group  population
0  AT  Under 15     1315821
1  AT     15-17      264406
2  AT     18-24      686721
```

- Population ages are `Y_LT1` (under 1), `Y1` … `Y99`, `Y_OPEN` (100+) : make them 0 … 100
- `.str[1:]` drops the `Y` ; `astype(int)` fails loudly on an unexpected label
- `pd.cut(..., right=False)` puts 15 in 15 to 17 and 65 in 65+ : 101 ages become 6 bands

Checking the Bands

```
published = people.loc[people['age'].eq('TOTAL')].set_index('geo')['value']
banded = bands.groupby('geo')['population'].sum()
print((banded == published).all())
print(bands.loc[bands['geo'].eq('NL') & bands['age_group'].eq('Under 15'),
                'population'].iloc[0])
```

```
True
2714040
```

- Each country's six bands add up to its published total population
- A hand check : the Netherlands' under-15 population is `Y_LT1` plus `Y1` … `Y14`
- A check that would fail on a lost age group is stronger than trusting the code

Building the Country and Age Grid

```
cells = pd.MultiIndex.from_product([COUNTRIES, LABELS], names=['geo', 'age_group'])
table = cells.to_frame(index=False)
table = table.merge(bands, how='left', validate='one_to_one')
table = table.merge(counts, how='left', validate='one_to_one', indicator=True)
print(len(table), table['_merge'].value_counts().to_dict())
recorded = table.groupby('geo')['deaths'].sum().loc[totals.index]
print((recorded == totals).all(), recorded.sum())
```

```
231 {'both': 219, 'left_only': 12, 'right_only': 0}
True 20875
```

- `from_product` builds every country × age group pair : the 231 cells exist before any data arrive
- Left merges keep every cell ; `validate` checks one row per cell on both sides
- `indicator` shows 12 cells without a death count
- Each country’s deaths add up to its published total : no death was lost or doubled

Missing Counts

```
missing = table.loc[table['deaths'].isna()]
print(missing['geo'].value_counts().to_dict())
print(table.loc[table['geo'].isin(['LI', 'ME']) & table['age_group'].eq('Unknown age'),
                ['geo', 'deaths']])
```

```
{'LI': 6, 'ME': 6}
    geo  deaths
132  LI       3
160  ME      75
```

- Liechtenstein and Montenegro report all their deaths as unknown age : 3 and 75
- A missing count is not zero deaths :
  - Filling with 0 keeps their population in the denominator
  - Every rate would become too low
- Unknown age has deaths but no population, so it gets no rate

Summary and Rates

```
table['reported_population'] = table['population'].where(table['deaths'].notna())
grouped = table.groupby('age_group', sort=False)
summary = pd.DataFrame({
    'countries': grouped.size(),
    'reported': grouped['deaths'].count(),
    'deaths': grouped['deaths'].sum(),
    'population': grouped['reported_population'].sum(min_count=1),
})
summary['missing'] = summary['countries'] - summary['reported']
summary['per_million'] = (summary['deaths'] / summary['population'] * 1_000_000).round(1)
```

- `reported_population` keeps a population only where a death count exists :
  - Countries without a count do not enter the denominator
- `size` counts all cells, `count` only the cells with a count
- `sum(min_count=1)` keeps the unknown-age population missing instead of 0
- `sort=False` keeps the groups in grid order

SpiralTrain developer courses

Interpreting the Result

age_group	countries	reported	missing	deaths	population	per_million
Under 15	33	31	2	405	68964305	5.9
15-17	33	31	2	475	15120594	31.4
18-24	33	31	2	2518	35122676	71.7
25-49	33	31	2	6448	151599137	42.5
50-64	33	31	2	4461	98658481	45.2
65+	33	31	2	6385	101525865	62.9
Unknown age	33	33	0	183	<NA>	<NA>

- The deaths add up to 20,875, the countries' published totals
- Read each rate with its counts :
  - 18-24 has the highest rate per inhabitant
  - 25-49 and 65+ have far more deaths
- Even if all 183 unknown-age deaths were 65+, that rate would stay below 18 to 24

Comparison with Published Rates

```
rates = deaths.loc[deaths['unit'].eq('P_MHAB') & deaths['sex'].eq('T')
                   & deaths['pers_cat'].eq('TOTAL') & deaths['year'].eq(2024)
                   & deaths['age'].isin(list(AGE_CODES)[:-1])]
rates = rates.assign(age_group=rates['age'].map(AGE_CODES))
mine = table.assign(rate=table['deaths'] / table['population'] * 1_000_000)
both = mine.merge(rates, on=['geo', 'age_group'], validate='one_to_one')
print((both['rate'].round() == both['value']).sum(), 'of', len(both), 'country rates match')

expected = pd.read_csv('road-deaths-expected-summary.csv').set_index('age_group')
print((expected['deaths'] == summary['deaths']).all())
```

```
186 of 186 country rates match
True
```

- Eurostat publishes its own rates per million per country and age band : all 186 agree after rounding
- `road-deaths-expected-summary.csv` was computed separately with SQL in DuckDB : the deaths agree too

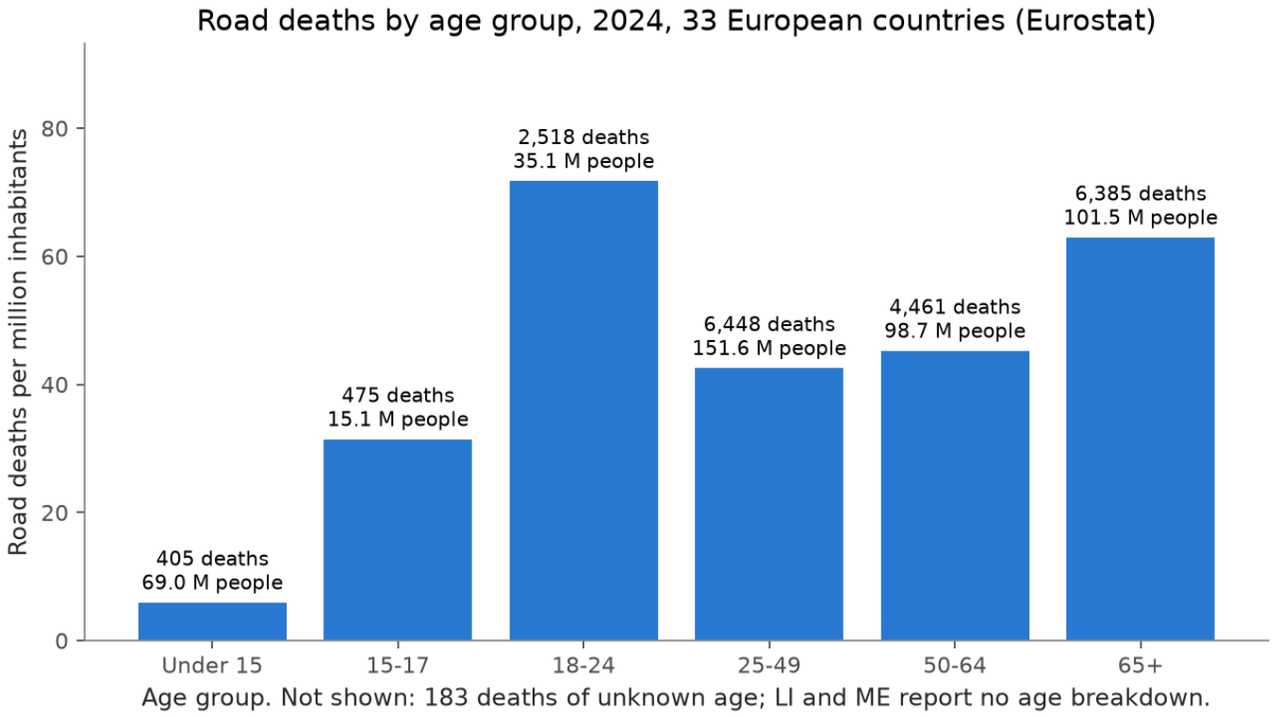
SpiralTrain developer courses

Charting the Rates

```
shown = summary.loc[summary['per_million'].notna()]
fig, ax = plt.subplots(figsize=(8, 4.5), layout='constrained')
bars = ax.bar(shown.index, shown['per_million'])
labels = [f'{deaths:,} deaths\n{people / 1e6:.1f} M people'
          for deaths, people in zip(shown['deaths'], shown['population'])]
ax.bar_label(bars, labels=labels, fontsize=9, padding=3)
unknown = summary.loc['Unknown age', 'deaths']
ax.set(ylim=(0, shown['per_million'].max() * 1.3),
       ylabel='Road deaths per million inhabitants',
       xlabel=f'Age group. Not shown: {unknown} deaths of unknown age; '
              f'LI and ME report no age breakdown.',
       title='Road deaths by age group, 2024, 33 European countries (Eurostat)')
fig.savefig('road-deaths-by-age.png', dpi=160)
plt.show()
```

- Unknown age has no rate, so it gets no bar ; the axis label says so
- `ax.bar_label` writes a label on each bar ; `ylim` leaves room for the labels
- Save the figure before `plt.show()`

Reading the Chart



- One bar per age group, starting at zero
- Each bar shows its deaths and population
- The axis label names what is not shown : unknown ages and the two countries without age detail
- A reader sees the size of each group as well as its rate

Reviewing Generated Code

```
rows = deaths.loc[deaths['year'].eq(2024) & deaths['sex'].eq('T')
                  & deaths['pers_cat'].eq('TOTAL') & deaths['age'].ne('TOTAL')]
years = pd.to_numeric(population['age'].str[1:], errors='coerce')
bands = population.assign(age=pd.cut(years, BINS, right=False, labels=CODES))
bands = (bands.loc[bands['sex'].eq('T')]
         .groupby(['geo', 'age'], observed=True, as_index=False)['value'].sum())
joined = rows.merge(bands.astype({'age': 'str'}), on=['geo', 'age'],
                    suffixes=('', '_population'))
answer = joined.groupby('age').agg(deaths=('value', 'sum'),
                                   population=('value_population', 'sum'))
answer['per_million'] = (answer['deaths'] / answer['population'] * 1_000_000).round(1)
```

- Written by the course team in the style of an AI assistant : it runs and looks plausible
- It reports 52.9 per million for 18 to 24, where we found 71.7
- `CODES` are the six band codes `Y_LT15` … `Y_GE65` ; the other names come from Step 3
- Before accepting it, ask which rows, ages and countries it describes

demo02
Audit

Checking the Grain

```
print(joined['unit'].unique().tolist(), 'EU27_2020' in set(joined['geo']))
print(joined.duplicated(['geo', 'age']).sum(), 'repeated cells')
```

```
['NR', 'P_MHAB'] True
192 repeated cells
```

- The proposal never filters `unit` : rates per million are added to the death counts
- It keeps the `EU27_2020` aggregate : 27 countries are counted twice
- 192 country × age cells appear more than once
- Repair : one row per country and age group, counts only, countries only (Step 4)

Checking the Age Mapping

```
lost = population.loc[years.isna(), 'age'].unique().tolist()
print(lost)   # ['TOTAL', 'UNK', 'Y_LT1', 'Y_OPEN']
```

- `pd.to_numeric(..., errors='coerce')` turns every label it cannot read into `NaN`
- `TOTAL` and `UNK` should go ; `Y_LT1` and `Y_OPEN` should not :
  - Babies under 1 and people aged 100+ vanish from the population without any error
- Repair : map `Y_LT1` and `Y_OPEN` explicitly and use `astype(int)`, which fails loudly (Step 6)
- Then check the bands against the published totals (Step 7)

SpiralTrain developer courses

Reconciling the Totals

```
all_countries = set(deaths['geo']) - {'EU27_2020'}
print(sorted(all_countries - set(joined['geo'])))   # ['LI', 'ME']
print('UNK' in set(joined['age']))                  # False
```

- **The default inner join keeps only rows that match on both sides :**
  - Liechtenstein and Montenegro disappear, because they have no age bands
  - The 183 unknown-age deaths disappear, because unknown age has no population
- **The total no longer reconciles with the published 20,875**
- **Repair : a left join onto the full grid keeps the gaps visible (Steps 8 to 9)**

Data Sharing and Privacy

- **Sending data to an AI assistant or another external tool is a sharing decision :**
  - Follow the data owner's rules and the approved-tool policy
  - Check for personal or confidential data, retention and credentials
- **Prefer sharing a schema plus a small invented example**
- **Aggregates can still identify people when counts are small :**
  - Liechtenstein recorded 3 road deaths in 2024
- **These public Eurostat tables can be shared without such concerns**

SpiralTrain developer courses

Limitations and Reproducibility

- **State the limitations with the answer :**
  - Missing counts are not zeros ; unknown ages are not shared out
  - Rates per inhabitant are not risk per kilometre travelled
  - An association with age does not establish a cause
- **Make the answer reproducible :**
  - Keep the raw CSVs unchanged ; write the summary and chart to separate files
  - Restart the kernel and run all cells in order : no leftover state
  - Record the environment : `uv sync --frozen` uses the committed lock

SpiralTrain developer courses

A Local Streamlit App

```
cd datasets
uv run --project .. --frozen streamlit run ../demos/11-integration-case/app.py
```

- Streamlit turns a Python script into a local web app
- The app repeats Steps 3 to 10 for a chosen sex and set of countries :
  - Shows the recorded deaths, the summary table and a bar chart
  - Downloads the selected cells as CSV
- Every widget change reruns the script ; `@st.cache_data` avoids reloading the CSVs each time
- Try only `LI` and `ME` : every death is of unknown age, so no rate can be shown

SpiralTrain developer courses

Exercise 11

- **Repeat the case in the exercise notebook, step by step**
- **Audit the assistant proposal with your own three checks**
- **Optional, or as homework :**
  - Repeat the analysis for women and for men separately
  - Write 6 to 10 sentences on the result and its limitations

- NumPy
- Pandas Intro
- Selecting and Filtering
- Cleaning and Transforming
- Advanced Indexing
- Rearranging Data
- Combining Data
- Grouping Data
- Matplotlib
- Time Series
- Integration Case
- ▶ **Scikit-Learn Essentials**
- Data Formats and Tools

APPENDIX

- Reading Other Sources

- Scikit-Learn Essentials
- The Estimator API
- Train and Test Data
- Classification
- Pipelines and Cross-Validation
- Clustering without Labels
- Common Modelling Errors
- Further Study

SpiralTrain developer courses

Scikit-Learn Essentials

- **scikit-learn is Python's standard library for machine learning :**
  - Regression : predict a number, such as fuel economy
  - Classification : predict a category, such as survived or not
  - Clustering : find groups without known labels
  - Tools to prepare data, split it and score models
- **It works directly with pandas DataFrames and NumPy arrays**
- **This module is a short tour on data you already know :**
  - `auto-mpg.csv` for regression and clustering
  - `titanic.csv` for classification

```
import sklearn
print(sklearn.__version__)   # 1.9.1 in the course environment
```

The Estimator API

```python
from sklearn.linear_model import LinearRegression

auto = pd.read_csv('auto-mpg.csv')
X = auto[['weight']]
y = auto['mpg']

model = LinearRegression()
model.fit(X, y)
print(model.coef_, model.intercept_)
# [-0.00764734] 46.21652454901758

new_cars = pd.DataFrame({'weight': [2500, 4000]})
print(model.predict(new_cars).round(1))   # [27.1 15.6]
```

- Every model follows the same pattern :
  - Create it, with its settings
  - `fit(X, y)` learns from the data
  - `predict(X)` applies it to new rows
- `X` is a table of features : a `DataFrame`, even with one column
- `y` is the target : one value per row
- Learned values end with `_` : here 0.0076 mpg less per pound

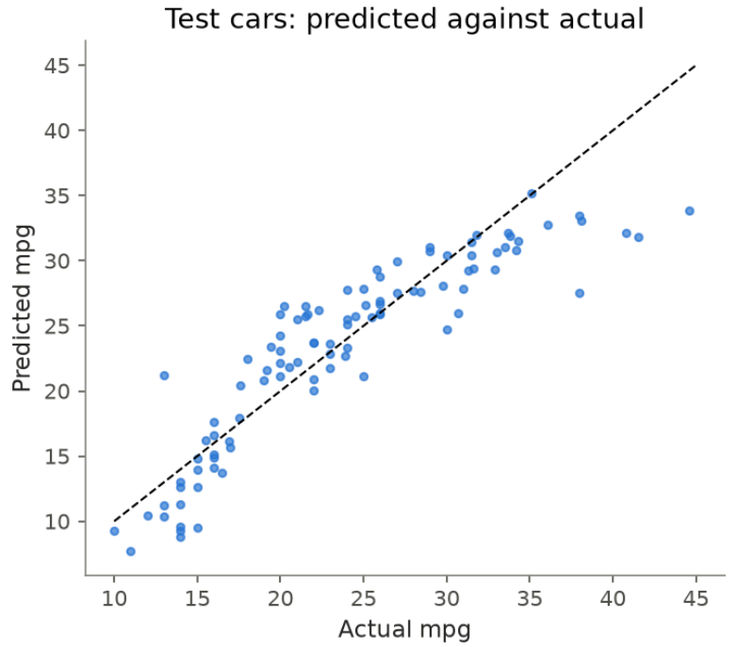
demo01
Estimator API

Train and Test Data

```
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split

X = auto[['weight', 'hp', 'yr']]
y = auto['mpg']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0)

model = LinearRegression().fit(X_train, y_train)
predicted = model.predict(X_test)
print(round(mean_absolute_error(y_test, predicted), 2),
      round(r2_score(y_test, predicted), 3))   # 2.61 0.801
```



- A model is judged on rows it has not seen : keep a test set apart
- Mean absolute error : predictions are 2.6 mpg off on average ; R² : 80% of the variation explained

demo02
Train and test

Classification

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

titanic = pd.read_csv('titanic.csv').dropna(subset=['age', 'fare'])
X = titanic[['pclass', 'age', 'fare']].assign(male=titanic['sex'].eq('male').astype(int))
y = titanic['survived']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

model = LogisticRegression(max_iter=1000).fit(X_train, y_train)
predicted = model.predict(X_test)
print(accuracy_score(y_test, predicted), 1 - y_test.mean())
print(confusion_matrix(y_test, predicted))
```

```
0.79 0.592
[[130  25]
 [ 30  77]]
```

- Logistic regression predicts a category ; `predict_proba` gives probabilities
- Base rate : always guessing ”died” is right 59% of the time
- Confusion matrix : 25 predicted survivors died, 30 survivors were missed

demo03
Logistic regression

SpiralTrain developer courses

Pipelines and Cross-Validation

```
prepare = ColumnTransformer([
    ('numbers', make_pipeline(SimpleImputer(strategy='median'), StandardScaler()),
     ['pclass', 'age', 'fare']),
    ('categories', make_pipeline(SimpleImputer(strategy='most_frequent'),
                                 OneHotEncoder(handle_unknown='ignore')),
     ['sex', 'embarked']),
])
model = make_pipeline(prepare, LogisticRegression(max_iter=1000))
print(cross_val_score(model, X, y, cv=5).round(3))
folds = KFold(n_splits=5, shuffle=True, random_state=0)
print(cross_val_score(model, X, y, cv=folds).round(3))
```

```
[0.515 0.836 0.809 0.725 0.678]
[0.794 0.786 0.782 0.756 0.778]
```

- A pipeline bundles preparation and model : fill gaps, scale numbers, one-hot encode text
  - Imported from `sklearn.compose`, `.impute`, `.pipeline`, `.preprocessing`, `.model_selection`
- Cross-validation scores the pipeline on five different splits
- `titanic.csv` is sorted by class : unshuffled folds give very different scores

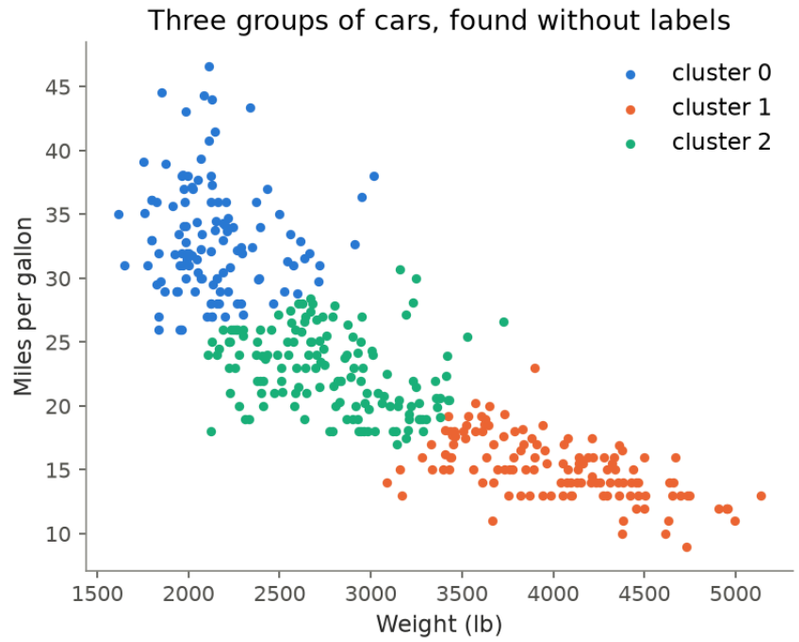
demo04 Pipeline

Clustering without Labels

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

features = StandardScaler().fit_transform(
    auto[['weight', 'mpg']])
auto['cluster'] = KMeans(
    n_clusters=3, n_init=10, random_state=0
).fit_predict(features)
print(auto.groupby(['cluster', 'origin'])
      .size().unstack(fill_value=0))
```

```
origin   Asia  Europe   US
cluster
0          56      32   31
1           0       2  123
2          23      34   91
```



- KMeans groups similar rows ; there is no `y`
- Scale first, so pounds and mpg weigh equally ; you choose the number of clusters

demo05
KMeans

Common Modelling Errors

- **Leakage : information from the test set or the future reaches the training data**
  - Fit scalers and imputers inside a pipeline, never on all data first
- **A high accuracy can match the base rate : always compare with the simplest guess**
- **Sorted or grouped data : shuffle folds, or keep related rows in the same fold**
- **A model finds associations, not causes : heavier cars use more fuel, but the model does not say why**
- **Test data is used once at the end ; tuning on it turns it into training data**

SpiralTrain developer courses

Further Study

- **The workflow from this course carries over :**
  - Clean and check the data first ; a model cannot repair a bad join
  - Keep missing values visible and decide how to handle them
- **Next steps in scikit-learn :**
  - More models : decision trees, random forests, gradient boosting
  - Model selection : `GridSearchCV` to tune settings with cross-validation
  - Feature engineering and evaluation metrics per problem
- **Documentation and tutorials : https://scikit-learn.org/stable/user_guide.html**
- **The SpiralTrain course *Python Machine Learning* continues from here**

- NumPy
- Pandas Intro
- Selecting and Filtering
- Cleaning and Transforming
- Advanced Indexing
- Rearranging Data
- Combining Data
- Grouping Data
- Matplotlib
- Time Series
- Integration Case
- Scikit-Learn Essentials
- ▶ **Data Formats and Tools**

APPENDIX

- Reading Other Sources

- Data Formats and Tools
- The Orders Ledger
- Typed CSV Ingestion
- Arrow-Backed dtypes
- Solution with pandas
- Parquet
- Arrow and Parquet
- SQL with DuckDB
- How DuckDB Reads Parquet
- Polars Expression Basics
- Polars Lazy Queries
- Choosing a Tool
- Compiling Loops with Numba
- Exercise 13

Data Formats and Tools

- **An optional module : the same summary computed with several tools**
  - pandas reads a small order ledger with deliberate types
  - Parquet stores the typed table and restores it
  - DuckDB computes the summary in SQL
  - Polars computes it with expressions and a lazy query
  - Each result matches the pandas result
- **Formats :**
  - CSV : plain text, every type decided when reading
  - Parquet : a typed, columnar file format built on Apache Arrow

The Orders Ledger

```
order_id,ordered_at,region,status,units
0001,2026-06-01,North,shipped,4
0002,2026-06-02,South,shipped,6
0003,2026-06-03,North,shipped,
0004,2026-06-04,South,cancelled,9
0005,2026-06-05,North,shipped,3
0006,2026-05-31,North,shipped,100
0007,2026-06-07,,shipped,5
0008,2026-06-08,South,shipped,2
0009,2026-07-01,South,shipped,200
0010,2026-06-10,North,shipped,0
0011,2026-06-11,East,shipped,
0012,2026-06-12,South,shipped,1
```

- `orders.csv` : 12 invented orders
- Task : June shipped units and order counts per region
  - Only orders with a known region and known units
  - An empty field means unknown ; zero is a recorded quantity
- Points to watch :
  - IDs with leading zeros
  - Two unknown quantities, one unknown region, one zero
  - May, July and cancelled orders

Typed CSV Ingestion

```
orders = pd.read_csv(
    'orders.csv',
    dtype={'order_id': str, 'units': 'Int64',
           'region': 'category',
           'status': 'category'},
    parse_dates=['ordered_at'],
    date_format='%Y-%m-%d',
)
print(orders.dtypes)

inferred = pd.read_csv('orders.csv')
print(inferred['order_id'].head(3).tolist(),
      inferred['units'].dtype)
```

```
order_id                 str
ordered_at    datetime64[us]
region                 category
status                 category
units                     Int64
dtype: object
[1, 2, 3] float64
```

- `str` keeps `0001`; inferred IDs become the numbers 1, 2, 3
- `Int64` keeps unknown units as `<NA>`; inferred `units` turns into floats
- `category` stores the few region and status values once

demo01
Typed ingestion

Arrow-Backed dtypes

```
arrow = pd.read_csv('orders.csv', dtype={'order_id': 'string[pyarrow]'},
                    dtype_backend='pyarrow')
print(arrow.dtypes)
print(arrow['units'].isna().sum())
```

```
order_id               string
ordered_at    string[pyarrow]
region        string[pyarrow]
status        string[pyarrow]
units          int64[pyarrow]
dtype: object
2
```

- `dtype_backend='pyarrow'` stores every inferred column as an Arrow type
- Arrow integers hold missing values without turning into floats
- Dates still need `parse_dates`; without it `ordered_at` stays text

demo02
Arrow dtypes

SpiralTrain developer courses

Solution with pandas

```
june_shipped = (orders['ordered_at'].ge('2026-06-01')
                & orders['ordered_at'].lt('2026-07-01')
                & orders['status'].eq('shipped'))
selected = orders.loc[june_shipped & orders['region'].notna() & orders['units'].notna()]
print(june_shipped.sum(), len(selected))

answer = (selected.groupby('region', observed=True)
          .agg(units=('units', 'sum'), orders=('units', 'size'))
          .reset_index())
print(answer)
```

```
9 6
  region  units  orders
0  North      7       3
1  South      9       3
```

- `ge` and `lt` (≥ and <) give the half-open interval 1 June up to 1 July
- 9 June orders were shipped ; 6 have a known region and known units
- `observed=True` leaves out the `East` category, whose only order has unknown units

demo03
pandas answer

Parquet

```
orders.to_parquet('orders.parquet', index=False)
restored = pd.read_parquet('orders.parquet')
print(restored.dtypes)

print(pq.read_schema('orders.parquet'))
subset = pd.read_parquet('orders.parquet',
                         columns=['region', 'units'])
print(subset.columns.tolist())
```

```
order_id                 str
ordered_at    datetime64[us]
region                category
status                category
units                    Int64
dtype: object
order_id: large_string
ordered_at: timestamp[us]
region: dictionary<values=string, indices=int8
status: dictionary<values=string, indices=int8
units: int64
['region', 'units']
```

- `import pyarrow.parquet as pq` reads the stored schema
- `columns=` reads only the named columns
- Parquet stores the types with the data : no `dtype` or `parse_dates` needed when reading back
- Columns are stored and compressed separately, so a reader can skip unneeded columns

demo04
Parquet

SpiralTrain developer courses

Arrow and Parquet

- **Apache Arrow is a typed, columnar in-memory format :**
  - pandas, Polars and DuckDB can all exchange Arrow data
  - A shared format makes moving data between tools cheap
- **Parquet is the matching file format :**
  - Stores column types, categories and missing values
  - Compresses each column separately
  - Lets readers load only the columns and row groups a query needs
- **Other formats keep their place :**
  - CSV for plain-text exchange, Excel for people
  - JSON for nested records from web services

SQL with DuckDB

```
con = duckdb.connect()
con.register('orders', orders)
SQL = '''
SELECT region, CAST(SUM(units) AS INTEGER) AS units, COUNT(*) AS orders
FROM {source}
WHERE ordered_at >= DATE '2026-06-01' AND ordered_at < DATE '2026-07-01'
  AND status = 'shipped' AND region IS NOT NULL AND units IS NOT NULL
GROUP BY region ORDER BY region
'''
print(con.execute(SQL.format(source='orders')).df())
print(con.execute(SQL.format(source="read_parquet('orders.parquet')")).df())
```

```
  region  units  orders
0  North      7       3
1  South      9       3
```

- DuckDB is an SQL database that runs inside Python ; `duckdb.connect()` needs no server
- `register` makes a DataFrame available as a table ; `read_parquet(...)` queries the file directly
- Both queries print the same answer as pandas

demo05 DuckDB

How DuckDB Reads Parquet

```
plan = con.execute('EXPLAIN ' + SQL.format(
    source="read_parquet('orders.parquet')"
)).fetchone()[1]
print(plan)
```

- `EXPLAIN` shows the query plan instead of running the query
- The scan reads two of the five columns : `region` and `units` (the projection)
- Filters run during the scan, so rows outside June are skipped early
- On a large file this is where the time is saved

```
┌────────────────────────────┐
│        READ_PARQUET        │
│        Projections:        │
│           region           │
│           units            │
│          Filters:          │
│ ordered_at>='2026-06-01 00 │
│   :00:00'::TIMESTAMP AND   │
│  ordered_at<'2026-07-01 00 │
│     :00:00'::TIMESTAMP     │
│      status='shipped'      │
│    (region IS NOT NULL)    │
│    (units IS NOT NULL)     │
└────────────────────────────┘
```

demo05
DuckDB

Polars Expression Basics

```python
import polars as pl

orders = pl.read_csv('orders.csv',
    schema_overrides={'order_id': pl.String},
    try_parse_dates=True)
result = (orders
    .filter(pl.col('status') == 'shipped')
    .filter(pl.col('region').is_in(['North',
                                    'South']))
    .with_columns(pl.col('units').cast(pl.Float64)
                  .alias('units_f'))
    .select('order_id', 'region', 'units_f'))
print(result.head(4))
```

```
shape: (4, 3)
┌──────────┬────────┬─────────┐
│ order_id ┆ region ┆ units_f │
│ ---      ┆ ---    ┆ ---     │
│ str      ┆ str    ┆ f64     │
╞══════════╪════════╪═════════╡
│ 0001     ┆ North  ┆ 4.0     │
│ 0002     ┆ South  ┆ 6.0     │
│ 0003     ┆ North  ┆ null    │
│ 0005     ┆ North  ┆ 3.0     │
└──────────┴────────┴─────────┘
```

- Polars is a DataFrame library without a row index ; `pl.col(name)` refers to a column
- `filter` keeps rows, `select` picks columns, `with_columns` adds or replaces columns
- `is_in` tests membership, `cast` changes the type, `alias` names the result ; missing is `null`

demo06
Polars basics

SpiralTrain developer courses

Polars Lazy Queries

```
query = (pl.scan_parquet('orders.parquet')
         .filter((pl.col('ordered_at') >= pl.lit(date(2026, 6, 1)))
                 & (pl.col('ordered_at') < pl.lit(date(2026, 7, 1)))
                 & (pl.col('status') == 'shipped')
                 & pl.col('region').is_not_null() & pl.col('units').is_not_null())
         .group_by('region')
         .agg(pl.col('units').sum(), pl.len().alias('orders'))
         .sort('region'))
print(query.explain())
print(query.collect())
```

```
SORT BY [col("region")]
  AGGREGATE ... BY [col("region")]
      Parquet SCAN [orders.parquet]
      PROJECT 4/5 COLUMNS
      SELECTION: ...
```

- `scan_parquet` describes the input ; nothing is read until `collect()`
- `explain()` shows the optimized plan : four of five columns, filters inside the scan
- `group_by` does not keep any order, so `sort` is explicit ; the result matches pandas

demo07
Polars lazy

Choosing a Tool

- **The tools work together in one project :**
  - pandas : flexible inspection, cleaning and plotting
  - DuckDB : SQL over DataFrames and files, without a server
  - Polars : fast expressions and lazy queries on large files
  - Parquet : the typed file format they all read
- **For data that hardly fits in memory :**
  - Read only the needed columns
  - Let the reader filter during the scan
  - Read CSV in chunks when partial results can be combined : sums and counts combine, averages of averages do not
- **Compare the answers before comparing the speed**

Compiling Loops with Numba

```python
from numba import njit

def sum_squares(values):
    total = 0.0
    for value in values:
        total += value * value
    return total

compiled = njit(sum_squares)
values = np.random.default_rng(0).random(1_000_000)
compiled(values)  # the first call compiles
```

```
333560.616 333560.616 333560.616
Python loop: 76.24 ms
Numba: 0.61 ms
NumPy: 2.38 ms
```

- Numba compiles numerical Python loops on NumPy arrays to machine code
- It does not speed up general pandas code
- Install with `uv sync --frozen --all-groups`
- Timings differ per machine ; check that the results agree first

demo08
Numba (optional)

SpiralTrain developer courses

Exercise 13

- **The same workflow on real data : Eurostat road deaths, 12,254 rows**
  - Typed load with coded categories
  - Parquet round trip, schema and column selection
  - One SQL query and one Polars query, both equal to pandas
  - Reflection on totals, absent rows and tools
- **Notebook :** `exercises/13-data-formats-and-tools/exercise.ipynb`

- NumPy
- Pandas Intro
- Selecting and Filtering
- Cleaning and Transforming
- Advanced Indexing
- Rearranging Data
- Combining Data
- Grouping Data
- Matplotlib
- Time Series
- Integration Case
- Scikit-Learn Essentials
- Data Formats and Tools

APPENDIX

- ▶ **Reading Other Sources**
  - Reading Other Sources
  - Several Files at Once
  - Excel Workbooks
  - JSON Records
  - SQL Databases

Reading Other Sources

- **pandas has a reader for each common source :**
  - `pd.read_csv()` for CSV and other delimited text (Module 2)
  - `pd.read_excel()` for Excel workbooks
  - `pd.read_json()` and `pd.json_normalize()` for JSON
  - `pd.read_sql()` for SQL databases
  - `pd.read_parquet()` for typed columnar files (Module 13)
- **Each reader has a matching writer :** `to_excel`, `to_json`, `to_sql`, `to_parquet`
- **Inspect the result as always :** `shape`, `head`, `dtypes`, `isna().sum()`

SpiralTrain developer courses

Several Files at Once

```
from glob import glob

filenames = sorted(glob('sales-*-2026.csv'))
print(filenames)
frames = [pd.read_csv(name, parse_dates=['Date']).assign(source=name)
          for name in filenames]
sales = pd.concat(frames, ignore_index=True)
print(sales['source'].value_counts().sort_index())
```

```
['sales-feb-2026.csv', 'sales-jan-2026.csv', 'sales-mar-2026.csv']
source
sales-feb-2026.csv    20
sales-jan-2026.csv    20
sales-mar-2026.csv    20
Name: count, dtype: int64
```

- `glob` finds the files that match a pattern ; `*` matches any text
- `sorted` fixes the order ; check the number of files before combining
- Read all files first, then call `pd.concat` once ; `source` records each row's file (Module 7)

demo01
Several files

Excel Workbooks

```
march = pd.read_excel('shop.xlsx', sheet_name='March',
                      skiprows=3)
print(march.head(3))

sheets = pd.read_excel('shop.xlsx', sheet_name=None)
print(list(sheets))
stores = pd.read_excel('shop.xlsx', sheet_name='Stores')
print(stores)
```

```
  order_id        date       city product
0     A001  2026-03-02  Amsterdam  Coffee
1     A002  2026-03-02    Utrecht     Tea
2     A003  2026-03-03  Amsterdam  Coffee
['March', 'Stores']
        city manager  opened
0  Amsterdam   Sanne    2019
1  Rotterdam    Daan    2021
2    Utrecht  Fatima    2023
```

- `sheet_name` picks a sheet by name or position ; the default is the first sheet
- `sheet_name=None` reads every sheet into a dictionary of DataFrames
- `skiprows` , `header` , `usecols` and `dtype` work as in `read_csv`
- Reading `.xlsx` needs `openpyxl` , which is in the course environment

demo02
Excel

JSON Records

```
flat = pd.read_json('orders-flat.json')
print(flat)

with open('orders.json') as file:
    records = json.load(file)
orders = pd.json_normalize(records)
print(orders.columns.tolist())
items = pd.json_normalize(records,
                          record_path='items',
                          meta=['order_id'])
print(items)
```

```
  order_id       city  units
0     A001  Amsterdam      4
1     A002    Utrecht      1
2     A003  Amsterdam      2
['order_id', 'items', 'customer.name',
 'customer.city']
  product  units order_id
0  Coffee      3     A001
1    Cake      1     A001
2     Tea      1     A002
3  Coffee      2     A003
```

- `read_json` reads a list of flat records directly
- `json_normalize` flattens nested records : `customer.city` comes from a nested object
- `record_path` turns a nested list into rows ; `meta` copies fields from the parent record

demo03
JSON

SQL Databases

```
import sqlite3

con = sqlite3.connect(':memory:')
pd.read_csv('sales-feb-2026.csv').to_sql('sales', con, index=False)
query = '''
SELECT Company, SUM(Units) AS units, COUNT(*) AS orders
FROM sales GROUP BY Company ORDER BY units DESC
'''
print(pd.read_sql(query, con))
print(pd.read_sql('SELECT * FROM sales WHERE Units > ?', con, params=(15,)))
```

- `sqlite3` ships with Python ; `':memory:'` makes a temporary database, a file name a real one
- `to_sql` writes a frame as a table ; `read_sql` runs a query and returns a DataFrame :
  - Mediacore leads with 45 units in 5 orders
- Pass values with `params=` instead of pasting them into the SQL text
- Other databases need their driver or a SQLAlchemy engine ; the pandas calls stay the same

SpiralTrain developer courses

Summary : Reading Other Sources

- Combine many files with `glob`, a list of frames and one `pd.concat`
- `read_excel` reads one sheet, a list of sheets or all sheets
- `read_json` for flat records, `json_normalize` for nested ones
- `read_sql` runs a query against a database connection
- Whatever the source : look at the raw data, declare types, inspect the result